

Bull Bear

Propozicija

BullBear je brokerska aplikacija koja korisnicima omogućava simulirano trgovanje (tzv. "paper trading") burzovnim instrumentima, poput dionica i ETF-ova. Etimologija naziva počiva na kolokvijalnim nazivima za stanja tržišta kapitala. Medvjed (eng. "bear") simbolizira silazni trend, vrijeme nesigurnosti i povećanoga rizika; dok bik (eng. "bull") predstavlja uzlazni trend popraćen rastom prinosa i povratom osjećaja sigurnosti i povjerenja.

Primarni cilj platforme jest omogućiti korisnicima sigurno testiranje strategija trgovanja bez rizika gubitka stvarnog kapitala, što je odraženo i u samome nazivu, stoga će svaki korisnik započeti s fiksnom količinom početnoga kapitala koja iznosi 10 000 \$.

Razlikovat će se dvije vrste korisnika, obični, te oni s "premium" pretplatom. Standardni će korisnici biti ograničeni gleda broja dnevnih transakcija (npr. 15) i pristupa tržišnim instrumentima (npr. 30 različitih dnevno), dok "premium" korisnici neće imati ta ograničenja, ali će morati plaćati mjesečnu pretplatu putem PayPal ili Stripe servisa.

Korisnici će imati pristup tržišnim podacima s vremenski odgođenim ažuriranjem (obično 10–15 minuta), budući da burzovni servisi naplaćuju pristup podacima u stvarnom vremenu i to i do nekoliko stotina tisuća američkih dolara mjesečno. Takvo ograničenje ne utječe negativno na osnovnu funkcionalnost aplikacije. Aplikacija radi u ciklusima (npr. 10 minuta): tijekom jednog ciklusa podaci vezani za pojedini instrument dohvaćaju se jednom i keširaju. Tek nakon isteka ciklusa ti se podaci mogu ponovno dohvatiti. Primjerice, ako ciklus traje od 00:00 do 00:10 i korisnik u 00:02 zatraži podatke (koji mogu biti stari i do 10 minuta) za Advanced Micro Devices, Inc. (AMD), podaci će se dohvatiti i keširati u bazi; drugi korisnik koji zatraži podatke za AMD u 00:09 dobit će prethodno keširane, a ne nove podatke. Ako treći korisnik zatraži AMD u 00:11 (u sljedećem ciklusu), dohvatit će se novi podaci. Trenutačno smo ograničeni na 60 API poziva u minuti, odnosno na 600 različitih instrumenata unutar jednoga ciklusa. Ukoliko se kvota ispuni, a podatak za određeni instrument je prisutan u keširanim podacima iz prošlih dvaju ciklusa (max. kašnjenje 30 min), koristit će se ti podaci, a inače će se pojaviti upozorenje. Budući da je glavna namjena platforme edukativna, takvo ograničenje ne predstavlja problem. Posljedično, broj korisnika koji može koristi platformu tijekom jednoga ciklusa je uvjetovan tim ograničenjima i njihovim zahtjevima, kao i hardverskom limitacijom poda na kojemu je platforma hostana.

Svaki korisnički profil može biti javan ili privatn. Transakcije javnih profila bit će uračunate u ukupnu ljestvicu, koja prati prinos na različitim vremenskim razinama, uključujući ukupnu, godišnju, mjesečnu, tjednu i dnevnu. Korisnici će moći kreirati i dijeliti vlastite popise omiljenih instrumenata, takozvane "mini fondovi". Također, oni koji koriste stvarne brokerske platforme moći će uvesti prethodne transakcije putem CSV datoteka, pri čemu mogu odabrati hoće li te transakcije biti uključene u javnu ljestvicu ili ostati privatne. Platforma će nuditi pregled uspješnosti, usporedbu prinosa s tržišnim indeksima poput S&P 500, NASDAQ i EURONEXT 100 ,te osnovne pokazatelje rizika, uključujući standardnu devijaciju za volatilnost i beta vrijednost koja prikazuje osjetljivost na tržišne promjene.

Za realizaciju projekta planirane su sljedeće integracije:

- Financijski API-ji: Yahoo Finance API, Finnhub.io, Polygon.io za dohvat tržišnih podataka koji će uključivati sve relevantne parametre instrumenta, kao što su cijena pri otvaranju (open), cijena pri zatvaranju (close), minimalna i maksimalna cijena (low i high), najviša kupovna i najniža prodajna cijena (ask i bid) te ukupni broj kupljenih i prodanih instrumenata (volume).
- Autentifikacija korisnika: Google i Microsoft OAuth (v2)
- plaćanje pretplate: PayPal i Stripe

U sustavu postoje i administratori sustava koji postavljaju cijenu članstva, te upravljaju korisnicima.

Korist projekta

Primarna korist projekta jest upoznavanje članova tima s tehnikama timskoga rada, alatima za suradnju i verzioniranje, CI/CD procesima za automatizaciju razvoja, testiranja i implementacije, te općenito usvajanje dobrih praksi u razvoju softverskih rješenja.

Sekundarna korist projekta jest upoznavanje s osnovama funkcioniranja tržišta kapitala te povećanje razine financijske pismenosti korisnika.

Sama aplikacija predstavlja siguran i edukativan način upoznavanja korisnika s trgovanjem na burzi, financijskim strategijama i tržišnim obrascima — bez rizika gubitka stvarnoga kapitala i bez potrebe za početnim ulaganjima.

Brokerski servisi često nude privremene "paper-trading" opcije, no one su obično vrlo ograničene i često se resetiraju nakon jedne sesije. Primarni cilj takvih platformi nije edukacija korisnika, već njihovo privlačenje prema stvarnim uslugama posredovanja.

Takvim servisima nije u interesu dugoročna profitabilnost korisnika, već naplata provizija za posredovanje u transakcijama. Nerijetko pojednostavljaju složene procese kako bi uslugu učinili privlačnijom osobama s niskom razinom financijske pismenosti.

Za razliku od toga, ova aplikacija djeluje na potpuno suprotnome principu — edukacija korisnika i razumijevanje tržišnih mehanizama predstavljaju njezin **primarni cilj**.

Slična rješenja

Na tržištu postoje platforme poput *Investopedia Simulatora*, *TradingView Paper Trading*a i *eToro Virtual Portfolio*, koje omogućuju simulirano trgovanje. Međutim, većina tih rješenja:

- nije otvorenoga koda
- ima ograničen pristup tržišnim podacima
- ne pruža edukativne materijale ni analitičke prikaze za početnike
- često je povezana s promotivnim interesima brokerskih kuća

Ovaj projekt razlikuje se po tome što naglasak stavlja na edukaciju, transparentnost i proširivost sustava, a ne na komercijalnu komponentu.

Skup korisnika

Aplikacija je namijenjena svima, neovisno o dobi, spolu, materijalnom statusu ili stupnju obrazovanja.

Posebice je korisan osobama koje žele povećati vlastitu financijsku pismenost, te svima koji žele razumjeti funkcioniranje tržišta kapitala i isprobati strategije ulaganja bez stvarnog rizika.

Mogućnost prilagodbe rješenja

Zbog modularne i API-orijentirane arhitekture, sustav se lako može prilagoditi različitim potrebama poput:

- integracije s dodatnim vanjskim servisima
- internacionalizacije za razne jezike
- kreiranje različitih verzija korisničkoga sučelja, koja ovise o razini iskustva korisnika

Opseg zadatka

Projekt uključuje:

1. Razvoj pozadinskog sustava (backend) temeljenog na Django REST Frameworku
2. Izradu baze podataka za pohranu korisničkih računa, transakcija i simuliranih tržišnih vrijednosti
3. Implementaciju osnovnog korisničkog sučelja (REACT web aplikacija)
4. Integraciju s vanjskim API-jima za preuzimanje tržišnih podataka, autentifikacij i plaćanje

Mogućnosti nadogradnje

Budući da je aplikacija temeljena na Django REST API-ju, moguće ju je jednostavno proširiti različitim klijentskim rješenjima: mobilnim aplikacijama, desktop klijentima ili CLI alatima.

Modularna struktura omogućava i jednostavno dodavanje novih značajki, poput:

- trgovanja izvedenicama (opcije, futuresi)
- ulaganja u plemenite metale (zlato, srebro) ili energente (nafta, plin i sl.)
- simulacije tržišta roba (pšenica, kukuruz i sl.)
- sustava notifikacija i preporuka
- naprednih alata za analizu i vizualizaciju portfelja
- te mogućnosti napredne komunikacije među korisnicima, po uzoru na društvene mreže

Uz odgovarajuću infrastrukturu, aplikacija bi se mogla razviti u kompleksnu edukativno-investicijsku platformu, koja spaja praktično iskustvo trgovanja i teorijsko razumijevanje tržišta kapitala.

U ovom poglavlju detaljno razraditi postavljene zahtjeve
Funkcionalni zahtjevi: Opis funkcionalnosti aplikacije (npr., registracija korisnika, prijava slučajeva, pregled statusa pomoći).
Nefunkcionalni zahtjevi: npr. Zahtjevi vezani uz performanse, sigurnost, skalabilnost i korisničko sučelje.

Zahtjeve prikazati standardnim formatom dobro strukturirane tablice zahtjeva koja u SRS dokumentu uobičajeno uključuje stupce: ID zahtjeva, Opis, Prioritet, Izvor i Kriteriji prihvatanja (inačica). Primjer:

Funkcionalni zahtjevi

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
FZ-1	Sustav mora podržavati prijavu i odjavu korisnika.	Visok	Poslovni zahtjevi	Korisnik se može uspješno prijaviti putem Google i/ili MS računa i odjaviti bez pogrešaka.
FZ-2	Sustav mora omogućiti upravljanje profilom i korisničkim podacima.	Srednji	Poslovni zahtjevi	Korisnik može mijenjati osobne podatke (korisničko ime, profilnu fotografiju, BIO), te spremiti izmjene.
FZ-3	Sustav mora dohvaćati cijene i podatke o instrumentima.	Visok	Tehnički zahtjevi	Sustav dohvaća ažurne podatke (cijena, open, close, high, low, volumen, profil korporacije) i koristi caching radi bržeg pristupa i smanjenog opterećenja API-ja.
FZ-4	Sustav mora omogućiti pretraživanje instrumenata po nazivu.	Srednji	Korisnički zahtjevi	Upis djelomičnog ili potpunog naziva instrumenta (dionica/ETF) vraća relevantne rezultate.
FZ-5	Sustav mora omogućiti simuliranu kupnju i prodaju instrumenata .	Visok	Poslovni zahtjevi	Korisnik može simulirano kupovati i prodavati instrumente; sustav bilježi transakcije, ažurira stanje portfelja i prikazuje promjene u stvarnome vremenu.
FZ-6	Sustav mora izračunavati vrijednosti u stvarnome vremenu.	Visok	Tehnički zahtjevi	Promjene cijena instrumenata odmah se reflektiraju na ukupnu vrijednost portfelja.
FZ-7	Sustav mora prikazivati sve relevantne informacije korisniku u grafičkome obliku.	Srednji	Korisnički zahtjevi	Grafovi i pokazatelji dinamički prikazuju tržišne promjene i rezultate korisnika.
FZ-8	Sustav mora omogućiti upravljanje i brisanje portfelja.	Srednji	Poslovni zahtjevi	Korisnik može kreirati, preimenovati i brisati portfelje; sustav automatski ažurira povezane podatke.
FZ-9	Sustav mora omogućiti uvoz i izvoz CSV datoteka s poviješću transakcija.	Nizak	Tehnički zahtjevi	Korisnik može uvesti i izvesti CSV datoteke s točnim i potpunim zapisima o transakcijama (vrijeme, simbol, opis, tip, količina, cijena, klasa).
FZ-10	Sustav mora prikazivati sortiranu ljestvicu javnih korisnika prema prinosu.	Srednji	Korisnički zahtjevi	Rang lista javnih korisnika automatski se sortira prema postotnom prinosu i redovito ažurira.
FZ-11	Sustav mora omogućiti kreiranje i upravljanje Mini-Fondovima.	Srednji	Poslovni zahtjevi	Korisnik može osnovati Mini-Fond (simulirani ETF), dodavati članove i instrumente te pratiti kolektivni prinos.
FZ-12	Sustav mora omogućiti interakciju s drugim korisnicima.	Nizak	Korisnički zahtjevi	Korisnici mogu međusobno komunicirati, komentirati i dijeliti rezultate simulacija unutar sustava putem vanjske integracije.
FZ-13	Sustav mora omogućiti kreiranje i upravljanje Mini-Fondovima.	Srednji	Korisnički zahtjevi	Korisnik može kreirati, uređivati i dijeliti popise omiljenih instrumenata.
FZ-14	Sustav mora omogućiti kupnju i obnovu premium pretplate.	Visok	Tehnička specifikacija	Korisnik može uspješno aktivirati i produžiti premium pretplatu uplatom putem PayPala ili kartičnim plaćanjem.
FZ-15	Sustav mora omogućiti administratorima upravljanje korisnicima i postavljanje cijene pretplate.	Visok	Poslovna pravila	Administrator može mijenjati cijene i blokirati korisnike putem administratorskog sučelja.
FZ-16	Sustav mora omogućiti korisnicima da odaberu je li njihov profil javan ili privatn.	Srednji	Korisnički zahtjevi	Korisnik može promijeniti vidljivost profila; privatni profili nisu uključeni u ljestvicu.
FZ-17	Sustav mora omogućiti usporedbu prinosa korisnika s tržišnim indeksima (S&P 500, NASDAQ, EURONEXT 100).	Nizak	Tehnička specifikacija	Sustav prikazuje grafičku usporedbu prinosa korisnika s indeksima.

ID zahtjeva	Opis	Prioritet	Izvor	Kriteriji prihvatanja
FZ-18	Sustav mora prikazivati pokazatelje rizika, uključujući volatilnost i beta vrijednost.	Nizak	Tehnička specifikacija	U sučelju se prikazuju vrijednosti standardne devijacije i beta koeficijenta.
FZ-19	Sustav mora omogućiti dodavanje i uklanjanje instrumenata s popisa omiljenih instrumenata.	Srednji	Korisnički zahtjevi	Korisnik može dodati instrument u popis omiljenih i ukloniti ga po želji.

Ostali zahtjevi

Nefunkcionalni zahtjevi i zahtjevi domene primjene dopunjuju funkcionalne zahtjeve. Oni opisuju kako se sustav treba ponašati i koja ograničenja treba poštivati (performanse, korisničko iskustvo, pouzdanost, standardi kvalitete, sigurnost...). Primjeri takvih zahtjeva u Vašem projektu mogu biti: podržani jezici korisničkog sučelja, vrijeme odziva, najveći mogući podržani broj korisnika, podržane web/mobilne platforme, razina zaštite (protokoli komunikacije, kriptiranje...) ... Svaki takav zahtjev potrebno je navesti u jednoj ili dvije rečenice.

ID zahtjeva	Opis	Prioritet
NF-1	Sučelje mora biti u potpunosti responzivno i funkcionalno na desktop i mobilnim uređajima.	Visok
NF-2	Prosječni odaziv API poziva mora biti ≤ 500 ms, a nijedan poziv ne smije trajati dulje od 2 s.	Visok
NF-3	Baza podataka mora imati dnevni full backup.	Visok
NF-4	Sustav mora automatski bilježiti greške i kritične događaje uz timestamp i korisnički kontekst.	Visok
NF-5	Sustav mora podržavati internacionalizaciju s minimalno dva jezika (EN i HR).	Visok
NF-6	Kod mora biti dokumentiran i pisan prema PEP8 za Python te TypeScript ESLint za TypeScript.	Visok
NF-7	Sustav mora biti skalabilan na zahtjev.	Visok

Dionici

- Korisnici
- Administratori platforme
- Naručitelj: izv.prof.dr.sc. Vlado Sruk, izv.prof.dr.sc. Alan Jović
- Razvojni tim

Aktori

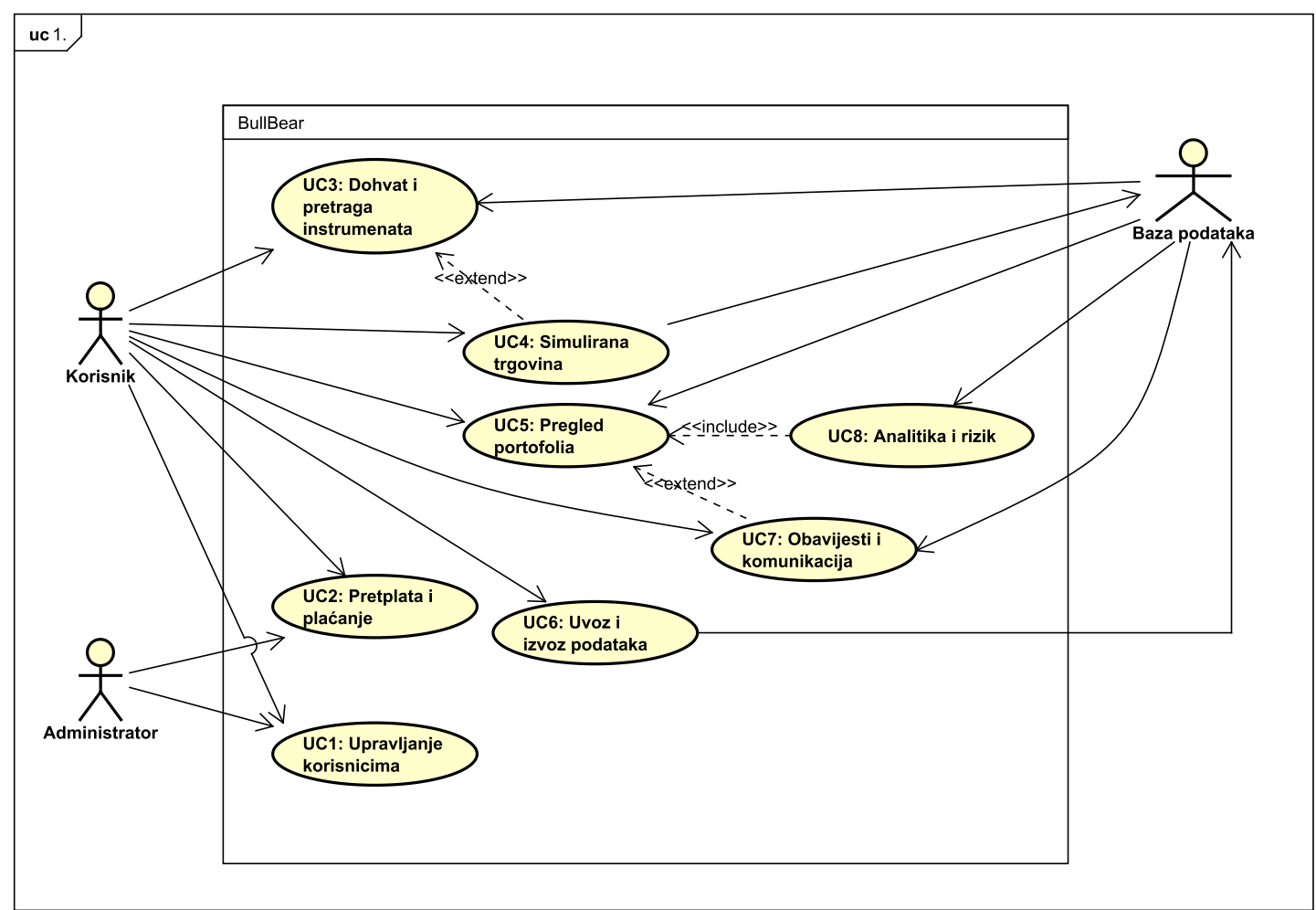
- A0: neprijavljeni korisnik - FZ1
- A1: (obični) prijavljeni korisnik - FZ1–FZ5, FZ7–FZ14, FZ16–FZ19
- A2: (premium) prijavljeni korisnik -FZ1–FZ5, FZ7–FZ14, FZ16–FZ19
- A3: administrator sustava - FZ1, FZ2, FZ15
- A4: vanjski financijski API - FZ3, FZ4
- A5: vanjski autorizacijski API -FZ1
- A6: baza podataka: FZ-1–FZ-19

Obrasci uporabe

Dijagrami obrazaca uporabe

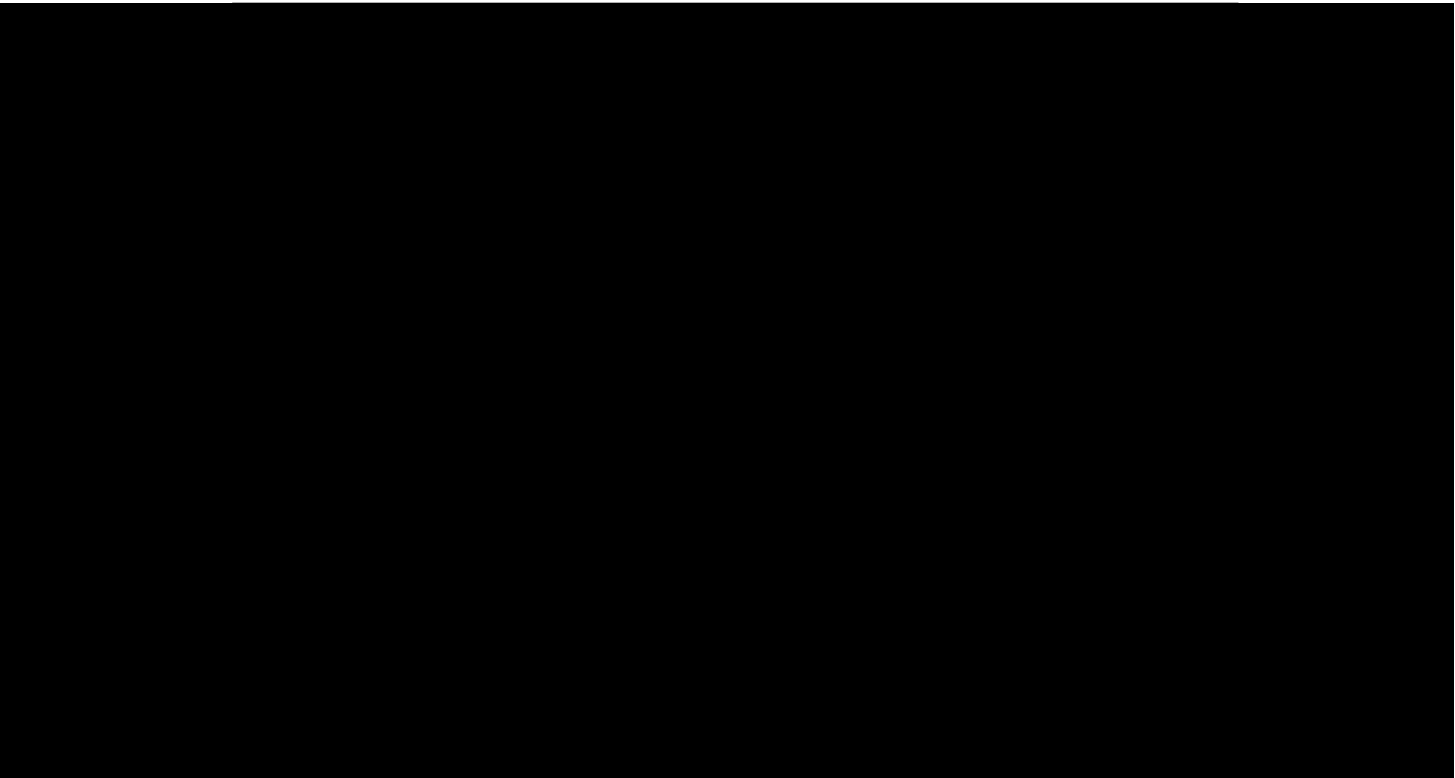
Prikazati odnos aktora i obrazaca uporabe odgovarajućim UML dijagramom. Nije nužno nacrtati sve na jednom dijagramu. Modelirati po razinama apstrakcije i skupovima srodnih funkcionalnosti.

1. Visokorazinski dijagram obrazaca uporabe cijelog sustava



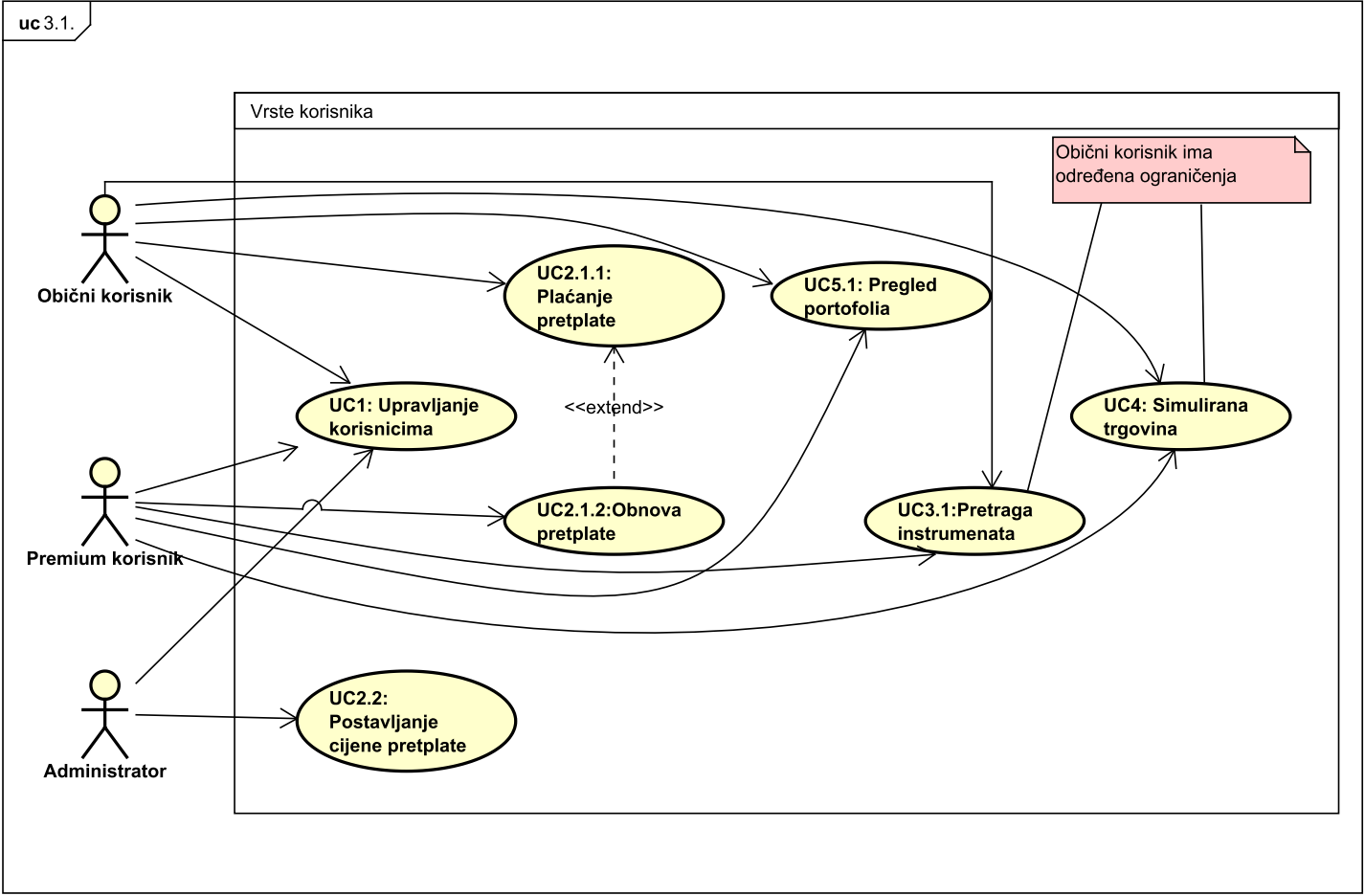
2. dijagram obrazaca uporabe za ključne značajke

2.1: Pregled portofolia



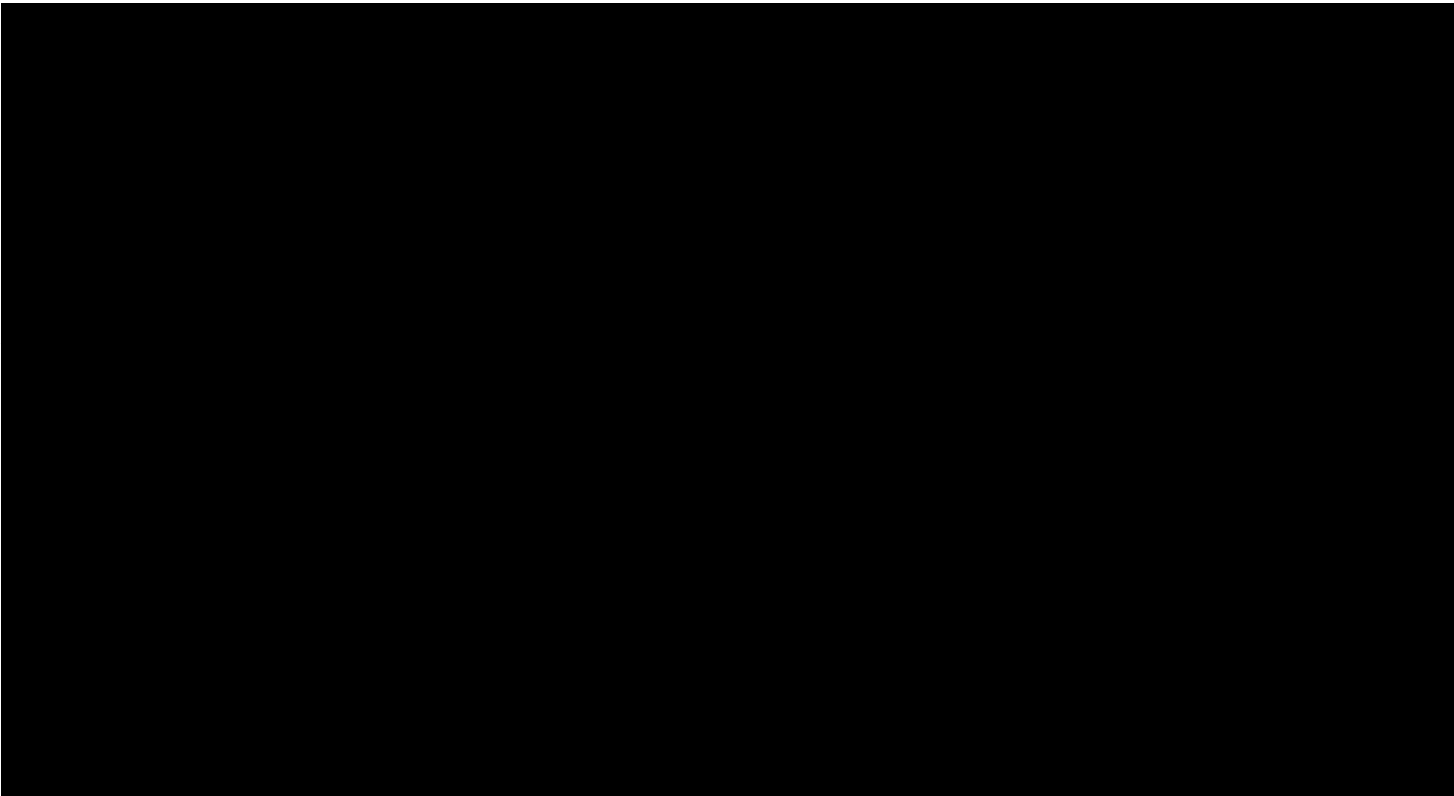
3. dijagram obrazaca uporabe za korisničke uloge

3.1: Vrste Korisnika

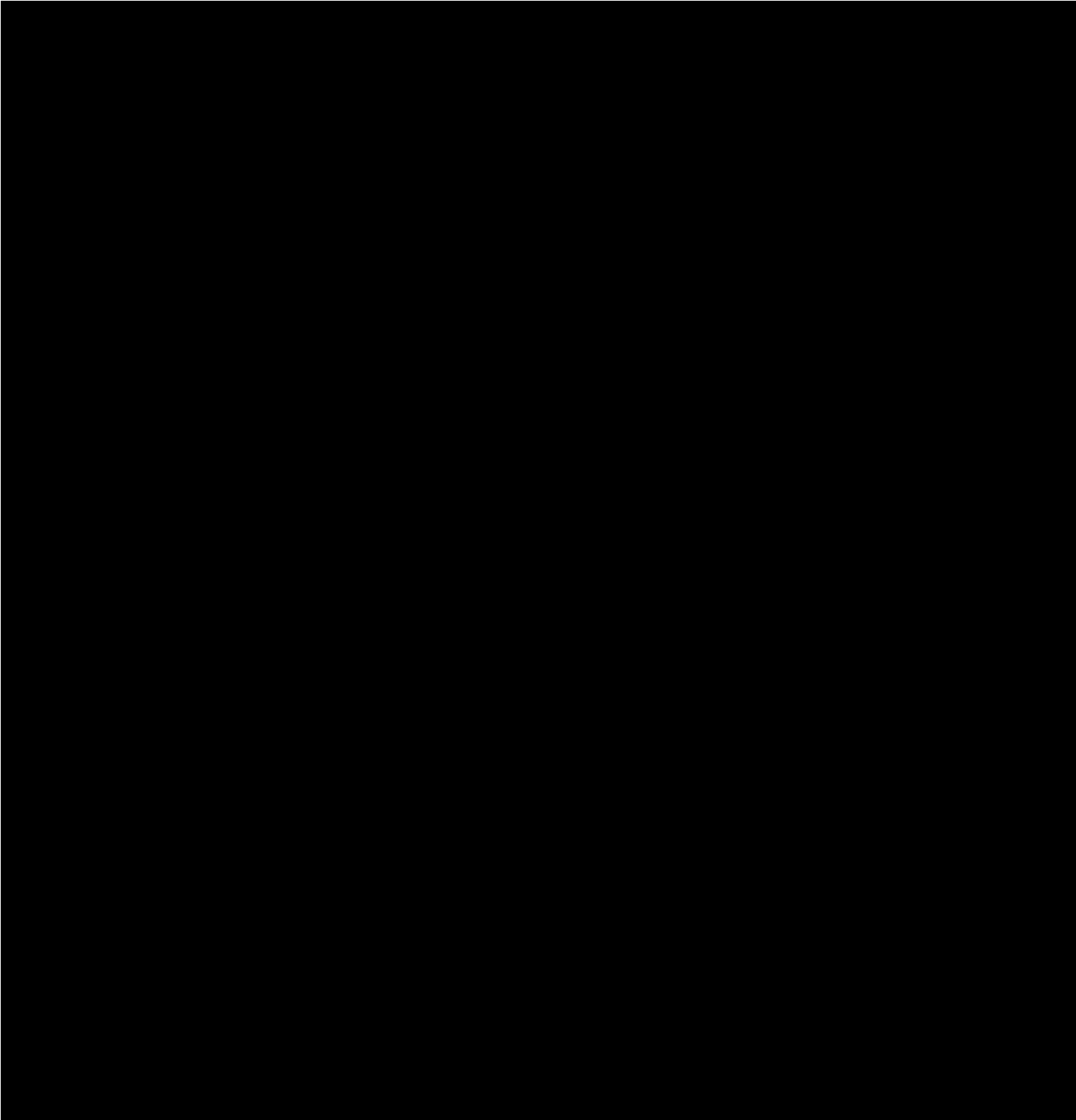


4. dijagram obrazaca uporabe za osnovne poslovne procese

4.1: Simulirana transakcija



5. dijagram obrazaca uporabe za kritične sustave i integracije



Opis obrazaca uporabe

UC1 – Upravljanje korisnicima

- Glavni sudionik: Administrator
- Cilj: kreiranje i brisanje korisničkih računa, zabrana pristupa platformi
- Preduvjet: administrator je prijavljen u sustav te traženi korisnički računi postoje
- Opis osnovnoga tijeka:

1. Administrator otvara sučelje za upravljanje računima
 2. Administrator odabire akciju nad korisničkim računom (stvaranje, brisanje ili zabrana pristupa)
 3. Administrator pohranjuje promjene, čime se mijenja status korisnika
- Opis mogućih odstupanja:

1. Administrator nije prijavljen u sustav
 - 1.a) Administratoru se prikazuje obavijest o pogrešci
 - 1.b) Administratora se preusmjerava na stranicu za prijavu u sustav
2. Traženi korisnički račun ne postoji
 - 2.a) Administrator mora stvoriti račun traženog korisnika

UC2 - Pretplata i Plaćanje

- Glavni sudionici: Korisnik
- Cilj: Plaćanje pretplate
- Preduvjet: Korisnik je prijavljen u sustav
- Opis osnovnoga tijeka:

1. Korisnik otvara obrazac za plaćanje pretplate
2. Korisnik odabire način plaćanja
3. Korisnik je preusmjeren na relevantnu vanjsku integraciju
4. Integracija sustava vraća odgovor o stanju uplate, koji mijenja status korisnika

- Opis mogućih odstupanja:

1. Plaćanje odbijeno
 - 1.a) Korisniku se pokazuje obavijest o pogrešci
 - 1.b) Korisnik se preusmjerava na odabir načina plaćanja
2. Vanjska integracija za plaćanje nije dostupna
 - 2.a) Korisniku se prikazuje obavijest o dostupnosti
 - 2.b) Korisnika se preusmjerava na drugi način plaćanja

UC3 - Dohvat i pretraga instrumenta

- Glavni sudionik: Korisnik
- Cilj: Pretraga instrumenata po nazivu i njihov dohvat iz baze podataka
- Preduvjet: korisnik je prijavljen u sustav, te zapis o traženome instrumentu postoji
- Opis osnovnoga tijeka:

1. Korisnik traži instrument po nazivu unutar dijaloga za pretraživanje
2. REACT frontend šalje RESTful zahtjev backendu s informacijom o traženome instrumentu
3. Django-REST backend vraća odgovor s JSON reprezentacijom instrumenta
4. REACT frontend korisniku prikazuje reprezentaciju instrumenta u vizualnome obliku

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen
 - 1.a) Zahtjev se ne izvršava
2. Sustav ne sadrži zapis o traženome instrumentu
 - 2.a) Korisniku se javlja da traženi instrument ne postoji u sustavu

UC4 - Simulirana trgovina

- Glavni sudionik: Korisnik
- Cilj: kreiranje i izvršavanje naloga za kupoprodajom instrumenata
- Preduvjet: korisnik je prijavljen, a relevantni instrument postoji u sustavu
- Opis osnovnoga tijeka:

1. Korisnik dobiva prikaz relevantnog instrumenta i njegovih metrika
2. Korisnik u sučelju bira između kupnje i prodaje instrumenta te bira željenu količinu
3. Korisnik šalje zahtjev o kupnji ili prodaji instrumenta
4. Zahtjev se obrađuje te korisnik dobiva potvrdu o uspješnoj transakciji, a njegov portfelj i status na računu se ažuriraju u bazi podataka

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen u sustav
 - 1.a) Transakcija se neće izvršiti
2. Traženi instrument ne postoji u sustavu
 - 2.a) Korisnik dobiva obavijest da traženi instrument ne postoji

3. Korisnik nema dovoljno sredstava za kupnju tražene količine instrumenta
 - 3.a) Korisnik dobiva obavijest da stanje na računu ne odgovara traženoj transakciji
 - 3.b) Transakcija se neće izvršiti
4. Korisnik pri prodaji nema traženu količinu instrumenta u svom portfelju
 - 4.a) Korisnik dobiva obavijest da stanje u portfelju ne odgovara traženoj transakciji
 - 4.b) Transakcija se neće izvršiti

UC5 - Pregled portfelja

- Glavni sudionik: Korisnik
- Cilj: Dohvat i prikaz informacija o korisničkom portfelju iz baze podataka
- Preduvjet: Korisnik je prijavljen u sustav te je stvoren portfelj za njegov račun
- Opis osnovnoga tijeka:

1. Korisnik na sučelju bira opciju prikaza svog portfelja
2. REACT frontend šalje RESTful zahtjev backendu s informacijom o traženom portfelju
3. Django-REST backend vraća odgovor s JSON reprezentacijom portfelja i instrumenata vezanih za njega
4. REACT frontend korisniku prikazuje reprezentaciju portfelja u vizualnom obliku

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen
 - 1.a) Zahtjev se neće izvršiti
2. Sustav ne sadrži zapis o traženom portfelju
 - 2.a) Korisniku se javlja da njegov portfelj ne postoji u sustavu

UC6 - Uvoz i izvoz podataka

- Glavni sudionik: korisnik
- Cilj: Ubaciti sadržaj CSV datoteke u korisnički portfelj, te portfelj izvesti u CSV datoteku
- Preduvjet: korisnik prijavljen u sustav
- Opis osnovnoga tijeka:

1. Korisnik u sučelju bira opciju uvoza ili izvoza podataka
2. Za uvoz, korisnik putem sučelja sa svog uređaja prilaže odgovarajuću datoteku CSV formata
3. Sustav čita datoteku i njen sadržaj sprema u korisnički portfelj u bazi podataka
4. Sustav javlja korisniku o uspjehu zahtjeva
5. Za izvoz, sustav dohvaća podatke o korisničkom portfelju iz baze podataka
6. Sustav formatira dohvaćene podatke u odgovarajući CSV format
7. Sustav isporučuje CSV datoteku na korisničko sučelje i javlja o uspjehu zahtjeva

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen u sustav
 - 1.a) Zahtjev se ne izvršava
2. Korisnički portfelj ne postoji
 - 2.a) Zahtjev se ne izvršava
3. Priložena datoteka nije odgovarajućeg formata
 - 3.a) Korisnika se obavještava o neispravnom formatu datoteke
 - 3.b) Korisnika se preusmjerava na sučelje za prijenos datoteke

UC7 - Obavijesti i komunikacija

- Glavni sudionik: Korisnik
- Cilj: Obavijestiti korisnika o vijestima i događajima za traženi instrument
- Preduvjet: Korisnik je prijavljen u sustav, te je traženi instrument u sustavu
- Opis osnovnoga tijeka:

1. Korisnik na sučelju bira prikaz informacija o traženom instrumentu
2. REACT frontend šalje RESTful zahtjev backendu s informacijom o traženom instrumentu
3. Django-REST backend vraća odgovor s JSON reprezentacijom instrumenta i nedavnim ključnim vrijednostima njegovih metrika (najviša i najniža vrijednost, trenutak početka rasta ili pada cijene...)
4. REACT frontend korisniku prikazuje reprezentaciju instrumenta i njegovih vijesti i događaja u vizualnom obliku

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen u sustav
 - 1.a) Zahtjev se neće izvršiti
2. Traženi instrument nema zapis u bazi podataka
 - 2.a) Korisniku se javlja da traženi instrument ne postoji u sustavu

UC8 - Analitika i rizik

- Glavni sudionik: Baza podataka
- Cilj: Prikaz analitičkih podataka i podataka rizika instrumenata u korisničkom portfelju
- Preduvjet: Korisnik je prijavljen u sustav i njegov portfelj sadrži instrumente
- Opis osnovnoga tijeka:

1. REACT frontend šalje RESTful zahtjev backendu s informacijom o traženom portfelju
2. Iz baze podataka se izvlače informacije o stanju instrumenta pri kupnji i o zadnjem keširanom stanju instrumenta
3. Sustav uspoređuje dohvaćene informacije i nad njima obavlja računanje traženih metrika i pokazatelja rizika
4. Django-REST backend vraća odgovor s JSON reprezentacijom portfelja i metrikama instrumenata vezanih za njega

- Opis mogućih odstupanja:

1. Korisnik nije prijavljen u sustav
 - 1.a) Zahtjev se neće izvršiti
2. Korisnički portfelj nije stvoren
 - 2.a) Zahtjev se neće izvršiti
3. Korisnički portfelj je prazan
 - 3.a) Sustav neće vratiti informacije o metrikama instrumenata

UC9 - Periodičko osvježavanje podataka

- Glavni sudionik: Scheduler
- Cilj: Prikupljanje informacija o instrumentima i ažuriranje stanja baze podataka s novim informacijama
- Preduvjet: Sustav povezan s financijskim API-jem za dohvat informacija o instrumentima
- Opis osnovnoga tijeka:

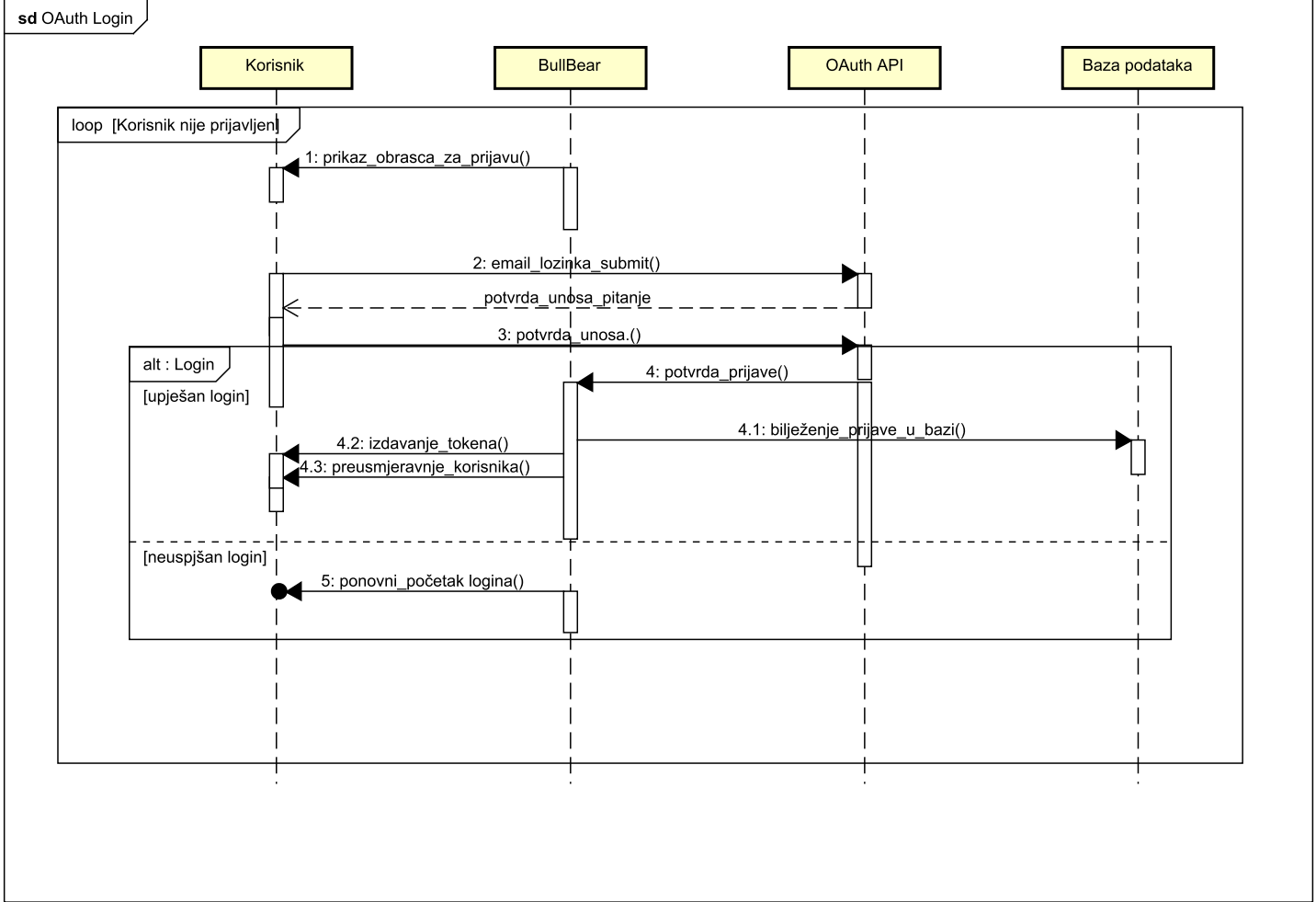
1. U trenutku zadanom po periodu, scheduler šalje zahtjev za dohvatom informacija o instrumentima
2. Financijski API vraća odgovor s JSON reprezentacijom novih podataka o instrumentima
3. Sustav prihvaća odgovor i nove podatke zapisuje u bazu podataka

- Opis mogućih odstupanja:

1. Financijski API nije dostupan
 - 1.a) U bazi podataka ostaju podaci zapisani pri posljednjem keširanju podataka
2. Greškom schedulera nije poslan zahtjev za dohvatom informacija u predviđenom trenutku
 - 2.a) U bazi podataka ostaju podaci zapisani pri posljednjem keširanju podataka

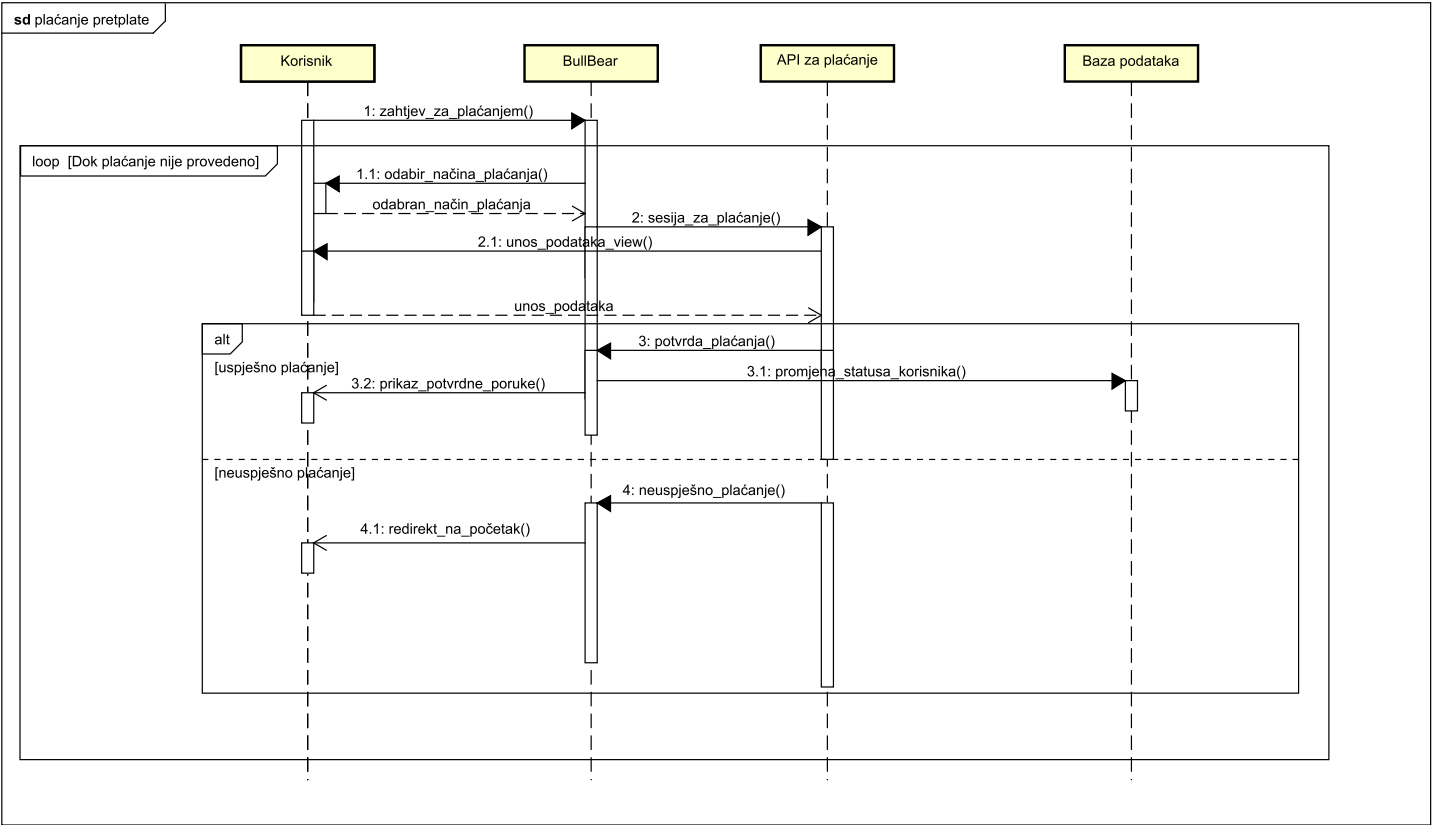
Sekvencijski dijagrami

1: OAuth Login



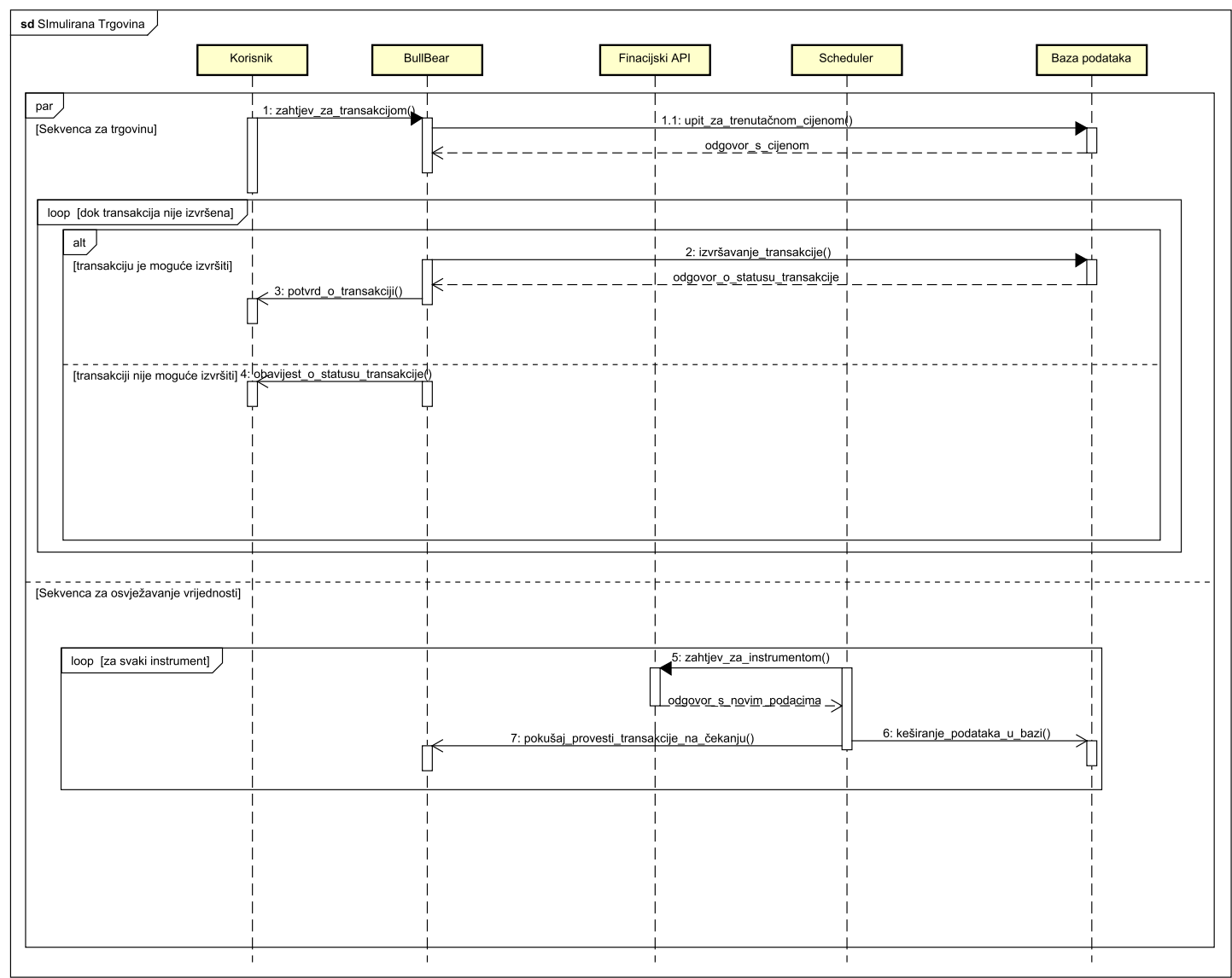
Proces započinje prikazom obrasca za prijavu korisniku; koji ispunjava tražena polja, te potvrđuje unos. U slučaju da su podaci ispravni, OAuth API aplikaciji vraća potvrdni odgovor, generira se pristupni token koji se šalje korisniku, koji biva preusmjeren na glavnu stranicu. Ukoliko to nije slučaj, korisniku se prikazuje pripadna poruka, te ga se preusmjerava na ponovnu prijavu. Proces se ponavlja sve dok se korisnik uspješno ne prijavi ili odustane od postupka.

2: Plaćanje pretplate



Proces započinje korisničkom inicijacijom postupka plaćanja, na što mu sustav vraća odgovor s mogućim načinima plaćanja. Nakon što korisnik odabere odgovarajući način plaćanja, inicira se sesija s odgovarajućim vanjskim API-jem za plaćanje (Stripe/PayPal), koji korisniku prikazuje vlastiti obrazac za unos potrebnih informacija (podaci o kartici ili login u PayPal račun). Nakon što korisnik ispuni tražena polja, vanjski API procesuirá plaćanje, te šalje odgovor aplikaciji. Ukoliko je odgovor potvrđan, mijenja se status korisnika (obični -> premium ili datum isteka pretplate), te se korisniku prikazuje odgovarajuća poruka. Inače se korisnik presumjerava na početak procesa, t.j. odabir načina plaćanja. Proces se ponavlja sve dok se plaćanje ne obavi ili korisnik odustane od procesa.

3: Simulirana trgovina



Dvije se sekvence izvršavaju paralelno. Scheduler konstantno (1 req/s) kontaktira vanjski finacijski API s zahtjevom za trenutačnim stanjem nekoga instrumenta.

Provjera uključenosti ključnih funkcionalnosti u obrasce uporabe

dijagram	FZ
OAuth Login	FZ1
Plaćanje pretplate	FZ1, FZ2
Simulirana trgovina	Fz3, FZ4

Arhitektura sustava

Opis arhitekture

HTTP ZAHTEJEV

HTTP ODGOVOR

Stil arhitekture

Odabrani stil arhitekture je klijent–poslužitelj s elementima uslužno orijentiranog sustava. Frontend (React) djeluje kao klijent, dok Django REST Framework backend predstavlja poslužitelj koji izlaže REST API sučelje putem HTTP protokola. Time se omogućuje potpuno razdvajanje tih dviju komponentni s ciljem lakšega razvoja, boljega ispitivanja usred manje međuovisnosti, lakšega horizontalnoga skaliranja u fazi deploymenta s ciljem uštede, itd. Uporaba REST principa omogućuje jednostavnu, te univerzalnu komunikaciju između različitij dijelova sustava,

Podsustavi

Podsustav	Opis i odgovornost
Frontend (React)	Omogućuje korisničko sučelje, obradu korisničkih akcija, prikaz podataka dobivenih putem REST API-ja. Odgovoran za interaktivnost i validaciju korisničkoga unosa.
Backend (Django REST Framework)	Implementira poslovnu logiku, autentifikaciju, autorizaciju, validaciju i komunikaciju s bazom podataka, te vanjskim servisima. Izlaže API endpoint-e prema van.
Baza podataka (PostgreSQL)	Pohranjuje sve trajne podatke sustava: korisnike, zapise, konfiguracije, keširane podatke o dionicama, itd.
Nginx (reverse proxy)	Posluhuje statičke datoteke i sam React build, te prosljeđuje API zahtjeve prema Django backendu.
Docker okruženje	Svaki podsustav (frontend, backend, ngin) nalazi se u zasebnom kontejneru, čime se postiže izolacija, prenosivost, ali i laka mogućnost skaliranja potrebnih računarnih resursa.

Deployment

Aplikacija će se nalaziti na DigitalOcean droppletu, povezana s GitHub repozorijem putem posebno definiranih akcija. Različiti servisi (AWS, MS Azure, Heroku, itd.) omogućuju iste/slične funkcionalnosti, mogućnosti konekcije, itd.; ali je ova platforma odabrana jer ne ograničava studente u dostupnosti opcija (poput nekih koje studente ograničavaju na single-core instance), te nudi \$200 USD u kreditima za svakoga studenta.

Baza Podataka

Mrežni protokoli

HTTP

Protokol HTTP(S) koristi se za komunikaciju između React frontenda i Django-REST backenda. React aplikacija upućuje RESTful API zahtjeve (GET, POST, PUT, PATCH, DELETE) prema definiranim Django-REST endpointima, dok backend vraća JSON odgovore koje React interpretira i prikazuje korisniku.

ORM

Komunikacija između Django-REST backenda i PostgreSQL baze podataka odvija se putem ORM (Object-Relational Mapping) sloja; ORM prevodi Python kod u SQL naredbe, te prima odgovore iz baze, koje zatim pretvara natrag u Python objekte.

WebSocket

Za real-time funkcionalnosti, poput chat sustava, notifikacija ili dinamičkog ažuriranja podataka, koristi se WebSocket protokol. Django, putem ASGI sučelja i biblioteka poput Django Channels, omogućuje dvosmjernu, trajnu vezu između klijenta i poslužitelja, što omogućuje neprekidnu razmjenu podataka bez potrebe za ponovnim učitavanjem stranice.

ASGI

Komunikacija između web-poslužitelja i Django aplikacije odvija se putem ASGI(Asynchronous Server Gateway Interface) sučelja, koje podržava sinhronu i asinkronu komunikaciju uz posredstvo WebSocket nad-protokola. Time se tvori veza između Django aplikacijskoga sloja i HTTP zahjtevа koje prima sam poslužitelj

Glavni Upravljački Tok

Tok započinje na REACT sučeklju, korisničko interakcijom s nekom vizualnom komponetnom (primjerice, obrascem za kupnju), pri čemu se šalje RESTfull API zahtjev Django-REST backendu putem HTTPS protokola, kojega prihvaća HTTP server. Potom, Django-REST backend prima zahjtev putem ASGI sučelja, gde ga odgovarajući view i serializer, odabarni preko URL uzorka, obrađuju.

Ako je potrebno pristupiti bazi podataka, backend koristi ORM sloj koji pretvara aplikacijski upit u SQL naredbu i prenosi ga PostgreSQL bazi, koja upit izvršava, te vraća rezultat putem ORM sloja, kojai ih prevodi natrah u Python objekte. Potom, serializer te objekte pretvara u JSON format prikladan za prijenos prema klijentu.

Backend potom vraća odgovor koji sadrži JSON podatke putem ASGI sučelja HTTP serveru, koji frontentu vraća taj odgovor putem HTTPS protokola. React aplikacija prima taj odgovor, ažurira stanje ili globalni kontekst te prikazuje dobivene informacije korisniku u web pregledniku. U slučajevima gdje je potrebna stalna komunikacija u stvarnom vremenu, React i Django održavaju trajnu vezu putem WebSocket protokola, što omogućuje dvosmjerni prijenos podataka bez ponovnog učitavanja stranice.

Ciklus se završava povratnom informacijom korisniku; ažuriranim prikazom, porukom o uspjehu ili prikazom novih podataka, čime se zatvara puni krug komunikacije između korisnika, aplikacijskog sloja i baze podataka.

Sklopovsko-programska potpora

Komponenta	Minimalni zahtjev za testiranje	Konačna instanca za predaju
Poslužitelj (backend + nginx)	1 vCPU, 1 GiB RAM, 10 GiB prostora (SSD)	4 vCPU, 8 GiB RAM, 10 GiB prostora (SSD)
Baza podataka	2 GiB RAM, 10 GiB prostora (SSD)	4 GB RAM, 20 GiB prostora (SSD)
Operativni sustav	Ubuntu 24.04. LTS (Server)	Ubuntu 24.04. LTS (Server)

Obrazloženje odabira arhitekture

Odabrani stil arhitekture temelji se na klijent–poslužitelj modelu s elementima „decoupled” pristupa. Frontend (React) je u potpunosti odvojen od backend sustava (Django REST Framework), što omogućuje neovisni razvoj, jednostavnije testiranje i lakšu nadogradnju pojedinih dijelova sustava. Ovakav stupanj razdvajanja smanjuje međuovisnosti između podsustava te povećava fleksibilnost i skalabilnost cijeloga rješenja. REST API komunikacija dodatno osigurava standardizirano i lako proširivo sučelje između komponenti.

Arhitektura je odabrana kako bi zadovoljila zahtjeve modularnosti, održivosti i jednostavne distribucije. Korištenje PostgreSQL baze podataka omogućuje stabilno i performantno upravljanje trajnim podacima, dok Nginx djeluje kao reverse proxy i poslužitelj statičkih datoteka (npr. CSS), čime rasterećuje backend. Cijeli se sustav izvodi kroz tri Docker kontejnera – frontend, backend i Nginx – što pojednostavljuje deployment, izolira procese i omogućuje lakše skaliranje prema potrebi.

- Izbor arhitekture temeljen na principima oblikovanja:
 - Visoka kohezija: svaki podsustav ima jasno definiranu odgovornost (UI, poslovna logika, pohrana podataka, poslužitelj statičkih resursa).
 - Niska povezanost: React i Django REST API komuniciraju isključivo putem HTTP zahtjeva, što omogućuje zamjenu ili nadogradnju jedne strane bez utjecaja na drugu.
 - Fleksibilnost i proširivost: REST API omogućuje jednostavno dodavanje novih endpointa i funkcionalnosti bez promjene postojeće strukture.

- 4. Održivost: jasan razdjeljen dizajn pojednostavljuje održavanje i buduće izmjene.
- 5. Skalabilnost: Docker te odvojen frontend/backend olakšavaju horizontalno skaliranje pojedinih dijelova sustava.

Ovi principi omogućuju da arhitektura zadovolji potrebe projekta uz minimalnu kompleksnost, ali uz jasnu mogućnost nadogradnje u budućnosti.

• Razmatrane alternative

Razmatran je i mikroservisni pristup, koji bi u teoriji pružio još bolju skalabilnost i granularnu kontrolu nad podsustavima. Ipak, takva arhitektura donosi znatnu složenost u pogledu orkestracije, komunikacije između servisa i održavanja. S obzirom na veličinu tima i opseg projekta, mikroservisna arhitektura ocijenjena je kao prekomplikirana i nepotrebno skupa. Stoga je odabran jednostavniji, ali dovoljno fleksibilan klijent–poslužitelj model s jasno odvojenim podsustavima.

Organizacija sustava na visokoj razini

Sustav je organiziran prema klijent–poslužitelj arhitekturi, pri čemu se frontend i backend izvode kao odvojeni, ali međusobno povezani podsustavi. Arhitektura naglašava modularnost i neovisnost komponenti, što olakšava razvoj, testiranje i održavanje.

• Klijent–poslužitelj

Frontend implementiran u Reactu djeluje kao klijent koji šalje HTTP zahtjeve prema backendu. Backend, izgrađen pomoću Django REST Frameworka, djeluje kao poslužitelj i izlaže REST API koji omogućuje dohvat, obradu i izmjenu podataka. Klijent komunicira isključivo preko API endpointa, čime se postiže potpuno “decoupled” interakcija i neovisnost između korisničkog sučelja i poslovne logike.

• Baza podataka

Sustav koristi PostgreSQL, relacijsku bazu podataka smještenu unutar zasebnog Docker kontejnera. Baza podataka služi za trajnu pohranu svih ključnih entiteta sustava (korisnici, zapisi, konfiguracije i ostali poslovni podaci). Backend pristupa bazi preko ORM sloja, čime se osigurava dosljednost, integritet podataka i jednostavno upravljanje upitima.

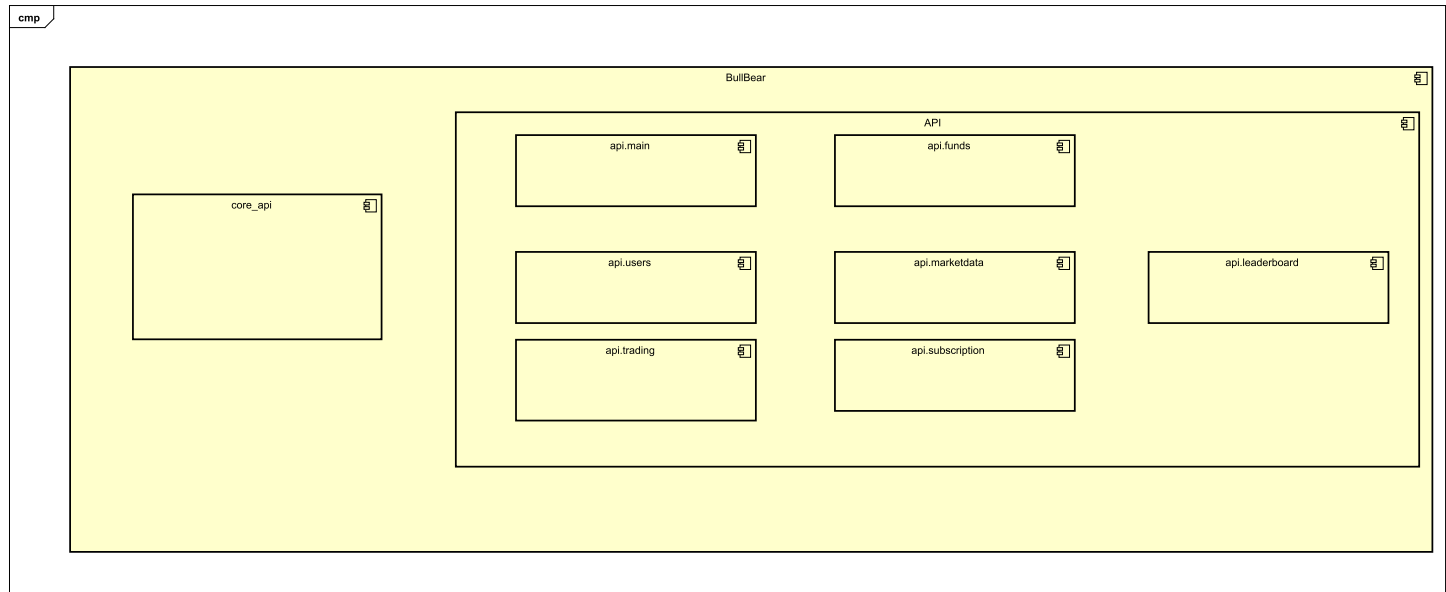
• Datotečni sustav

Statički resursi koji se ne pohranjuju u bazu – poput generiranih datoteka, CSS-a, JavaScript builda i drugih statičkih sadržaja – smješteni su na datotečnom sustavu koji posluhuje Nginx. Nginx djeluje kao reverse proxy koji prosljeđuje API zahtjeve backendu, dok istovremeno preuzima odgovornost za posluživanje statičkih datoteka radi bolje optimizacije performansi.

• Grafičko sučelje

Korisničko sučelje implementirano je kao web aplikacija u Reactu. Ono se izvršava u pregledniku korisnika i prikazuje podatke preuzete putem REST API-ja. GUI je u potpunosti odvojen od backend logike, a integracija se odvija isključivo putem mrežne komunikacije. Takav pristup omogućuje jednostavnu nadogradnju sučelja, brzu iteraciju te mogućnost razvoja alternativnih klijenata (npr. mobilne aplikacije) bez promjene backend dijela sustava.

Organizacija aplikacije



Aplikacija je strukturirana kao skup posve odvojenih, ali povezanih cjelina: Django–REST backend, React frontend i servis za periodičko ažuriranje tržišnih vrijednosti. Takva izolacija omogućuje kvalitetnu implementaciju principa separacije odgovornosti, veću otpornost na pogreške i olakšani razvoj. Svaka cjelina obavlja strogo definiranu ulogu u cjelokupnoj arhitekturi, a međusobna se komunikacija svodi na standardizirane protokole (REST, ORM, ASGI).

Tradicionalne Django aplikacije slijedi obrazac nalik MVC paradigmi (točnije M–Template–C model), međutim u ovoj implementaciji prezentacijski sloj u potpunosti je izdvojen u React sloj, čime se Django backend reducira isključivo na ulogu API poslužitelja. React frontend i Django REST backend međusobno komuniciraju isključivo putem REST API poziva, što omogućuje potpunu neovisnost dvaju slojeva. Takvo strukturiranje omogućuje da se frontend i backend razvijau posve nezavisno, što značajno olakšava testiranje i sam razvoj.

Pomoćni servis za ažuriranje vrijednosti predstavlja dodatnu komponentu koji djeluje nezavisno od glavnih cjelina, pristupajući bazi podataka izravno, zaobilazeći Django–REST sloj, ubrzavajući proces pristupa i smanjući opterećenje samoga sustava.

Backend

Django REST backend dodatno je organiziran u dvije glavne komponentne; core-api i api. Core-api osigurava glavnu konfiguraciju, globalne postavke,i rutiranje prema api podsustavima. Kompleksniji dio strukture nalazi se u modulu api, koji je podijeljen u jasno definirane pod-komponente; api.users, api.trading, api.leaderboard, api.subscription itd.. Svaka od tih cjelina funkcionira kao polu-neovisna jedinica s vlastitim modelima, serializerima, viewovima i rutama. Premda dijele jedinstvenu bazu podataka, te postoji određena komunikacija između njih, u pravilu djeluju kao semi-neovisne cjeline.

Ovakav način organizacije pridonosi jednostavnosti sustava, jednostavnijem otklanjanju pogrešaka te bržem razvoju novih funkcionalnosti. Modularni pristup omogućuje razvoj specifičnim dijelova sustava bez narušavanja stabilnosti ostalih. Nadalje, precizna podjela omogućuje da se pojedine cjeline potencijalno izdvoje u zasebne usluge u kasnijim fazama razvoja, ili se lako zamjene drugima.

Frontend

React frontend strukturiran je modularbno, gdje se veće cjeline (stranice) sastoje od manjih, višekratno upotrebljivih komponenti. Tijekom procesa izgradnje sve se komponentne objedinjene u jedinstvenu SPA strukturu, čime se zadržavaju sve prednosti modernih klijentske aplikacije poput dinamičkoga učitavanje sadržaja bez potreba za ponovnim učitavanjem stranica te mogućnost.

Cjelokupni sustav zasniva se na kombinaciji monolitnog i mikroservisnog pristupa, preuzimajući najbolja svojstva svakoga od tih načina podjele.

- Django dokumentacija, posljednji pristup 14.11.2025: <https://docs.djangoproject.com/en/5.2/>
- Djnago-REST dokumentacija, posljednji pristup 14.11.2025: <https://www.django-rest-framework.org>
- React dokumentacija, posljednji pristup 14.11.2025: <https://react.dev/reference/react>
- JavaScript frameworks and libraries, posljednji pristup 14.11.2025: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries
- Django Web Framework (Python), posljednji pristup 14.11.2025: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django

Baza podataka

Baza je podataka konstruirana izravno iz Modela, slijedeći Django pristup.

Opis tablica

Svaku tablicu je potrebno opisati po zadanom predlošku. Lijevo se nalazi točno ime varijable u bazi podataka, u sredini se nalazi tip podataka, a desno se nalazi opis varijable. Označite primarni ključ i strani ključ.

Ime tablice

LogEntry

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
content_type	ForeignKey(id)	Foreign Key
user	ForeignKey(id)	Foreign Key
action_flag	PositiveSmallIntegerField	
action_time	DateTimeField	
change_message	TextField	
object_id	TextField	

Atribut	Tip Podatka	Opis Varijable
object_repr	CharField	

Permission

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
content_type	ForeignKey(id)	Foreign Key
codename	CharField	
name	CharField	

Group

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
name	CharField	

Site

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
domain	CharField	
name	CharField	

ContentType

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
app_label	CharField	
model	CharField	

|

Session

Atribut	Tip Podatka	Opis Varijable
session_key	CharField	PrimaryKey
expire_date	DateTimeField	
session_data	TextField	

User

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
date_joined	DateTimeField	
email	EmailField	
first_name	CharField	
is_active	BooleanField	
is_staff	BooleanField	

Atribut	Tip Podatka	Opis Varijable
is_superuser	BooleanField	
last_login	DateTimeField	
last_name	CharField	
password	CharField	
username	CharField	

UserProfile

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
user	OneToOneField(id)	One-to-One
avatar_url	URLField	
bio	TextField	

UserPortfolio

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
user	ForeignKey(id)	Foreign Key
balance	DecimalField	
created_at	DateTimeField	
is_active	BooleanField	
name	CharField	
updated_at	DateTimeField	

UserSettings

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
user	OneToOneField(id)	One-to-One
dark_mode	BooleanField	
email_notifications	BooleanField	

UserActivityLog

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
user	ForeignKey(id)	Foreign Key
activity_type	CharField	
details	TextField	
timestamp	DateTimeField	

SubscriptionType

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
description	TextField	

Atribut	Tip Podatka	Opis Varijable
name	CharField	
price	DecimalField	

UserSubscriptionPackage

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
subscription_type	ForeignKey(id)	Foreign Key
duration_days	PositiveIntegerField	
is_active	BooleanField	
price	DecimalField	

UserSubscription

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
package	ForeignKey(id)	Foreign Key
user	ForeignKey(id)	Foreign Key
end_date	DateTimeField	
is_active	BooleanField	
start_date	DateTimeField	

Instrument

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
currency	CharField	
exchange	CharField	
is_active	BooleanField	
name	CharField	
symbol	CharField	
type	CharField	

InstrumentIntervalData

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
instrument	ForeignKey(id)	Foreign Key
close_price	DecimalField	
data_source	CharField	
end_time	DateTimeField	
high_price	DecimalField	
low_price	DecimalField	
open_price	DecimalField	
start_time	DateTimeField	
updated_at	DateTimeField	

Atribut	Tip Podatka	Opis Varijable
volume	BigIntegerField	

PortfolioHolding

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
instrument	ForeignKey (id)	Foreign Key
portfolio	ForeignKey (id)	Foreign Key
added_at	DateTimeField	
average_price	DecimalField	
quantity	DecimalField	

InstrumentQuote

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
ask_price	DecimalField	
ask_size	IntegerField	
bid_price	DecimalField	
bid_size	IntegerField	
currency	CharField	
daily_change	DecimalField	
daily_change_percent	DecimalField	
exchange	CharField	
instrument	CharField	
last_price	DecimalField	
market_state	CharField	
timestamp	DateTimeField	
updated_at	DateTimeField	

TradeTransaction

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
instrument	ForeignKey (id)	Foreign Key
portfolio	ForeignKey (id)	Foreign Key
transaction_type	CharField	

Fond

Atribut	Tip Podatka	Opis Varijable
id	BigAutoField	Primary Key
composition	JSONField	
created_at	DateTimeField	
description	TextField	
is_active	BooleanField	

Atribut	Tip Podatka	Opis Varijable
name	CharField	
updated_at	DateTimeField	

Token

Atribut	Tip Podatka	Opis Varijable
key	CharField	PrimaryKey
user	OneToOneField(id)	One-to-One
created	DateTimeField	

EmailAddress

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
user	ForeignKey(id)	Foreign Key
email	EmailField	
primary	BooleanField	
verified	BooleanField	

EmailConfirmation

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
email_address	ForeignKey(id)	Foreign Key
created	DateTimeField	
key	CharField	
sent	DateTimeField	

SocialApp

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
client_id	CharField	
key	CharField	
name	CharField	
provider	CharField	
provider_id	CharField	
secret	CharField	
settings	JSONField	

SocialAccount

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
user	ForeignKey(id)	Foreign Key
date_joined	DateTimeField	
extra_data	JSONField	

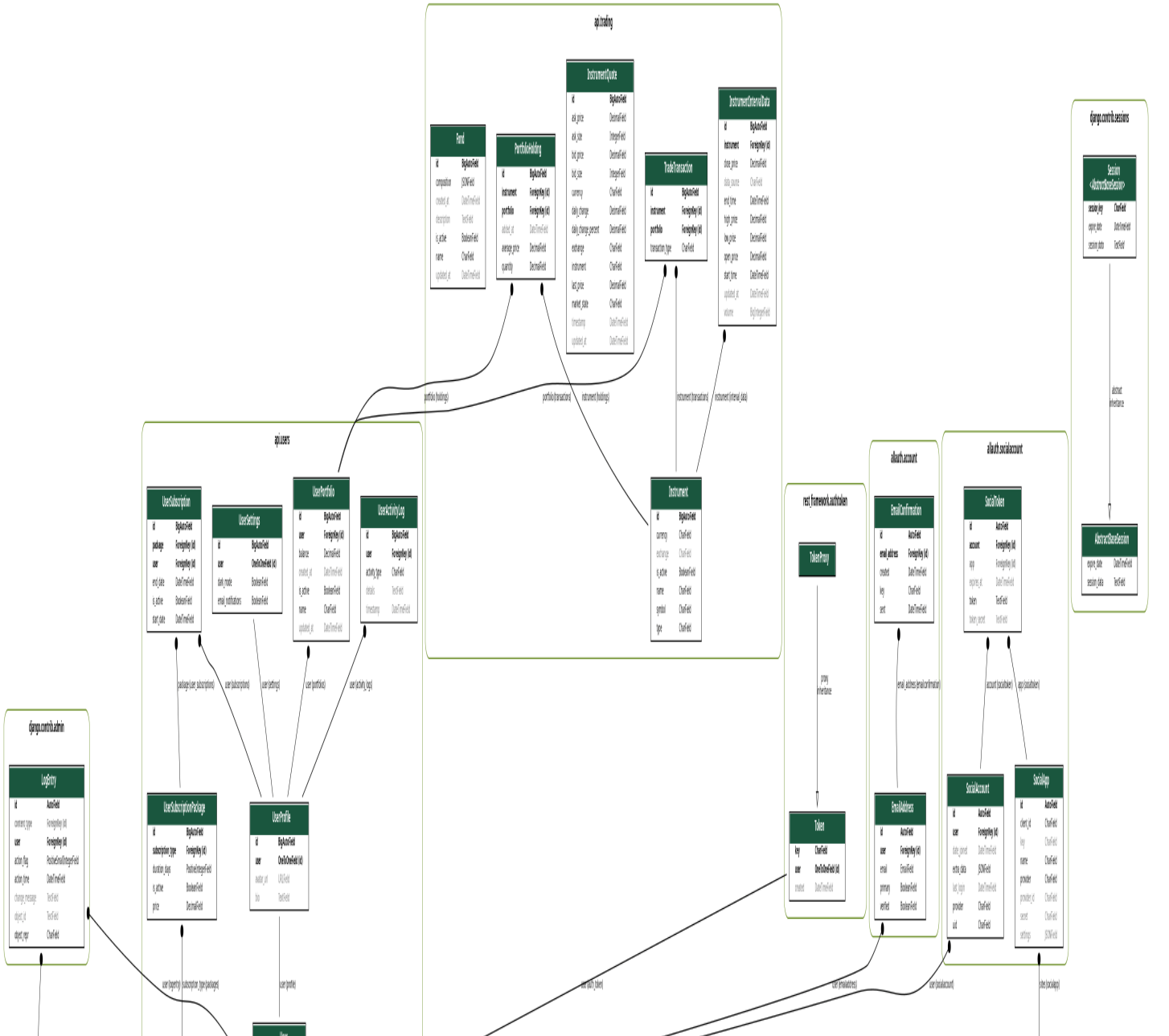
Atribut	Tip Podatka	Opis Varijable
last_login	DateTimeField	
provider	CharField	
uid	CharField	

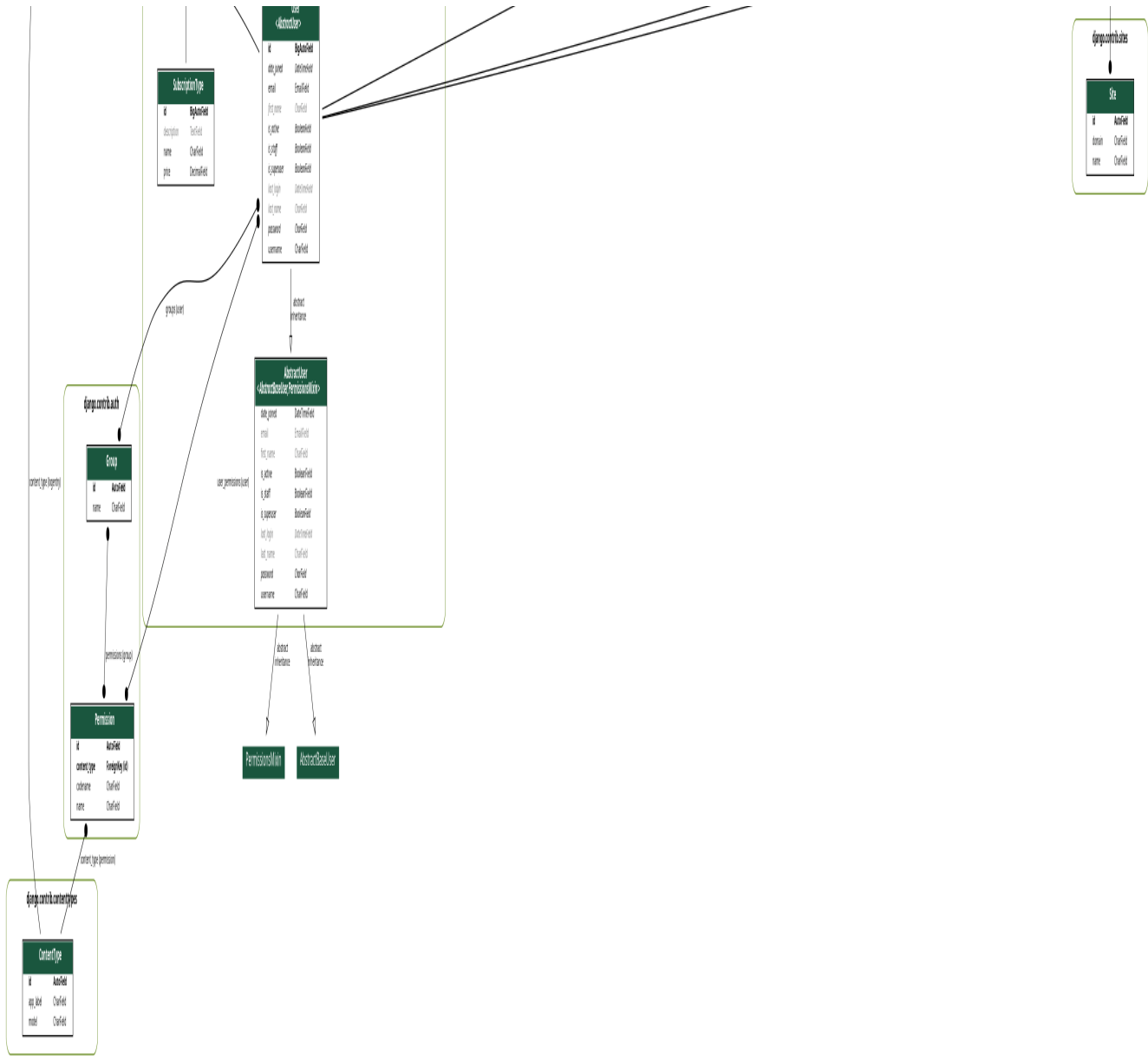
SocialToken

Atribut	Tip Podatka	Opis Varijable
id	AutoField	Primary Key
account	ForeignKey(id)	Foreign Key
app	ForeignKey(id)	Foreign Key
expires_at	DateTimeField	
token	TextField	
token_secret	TextField	

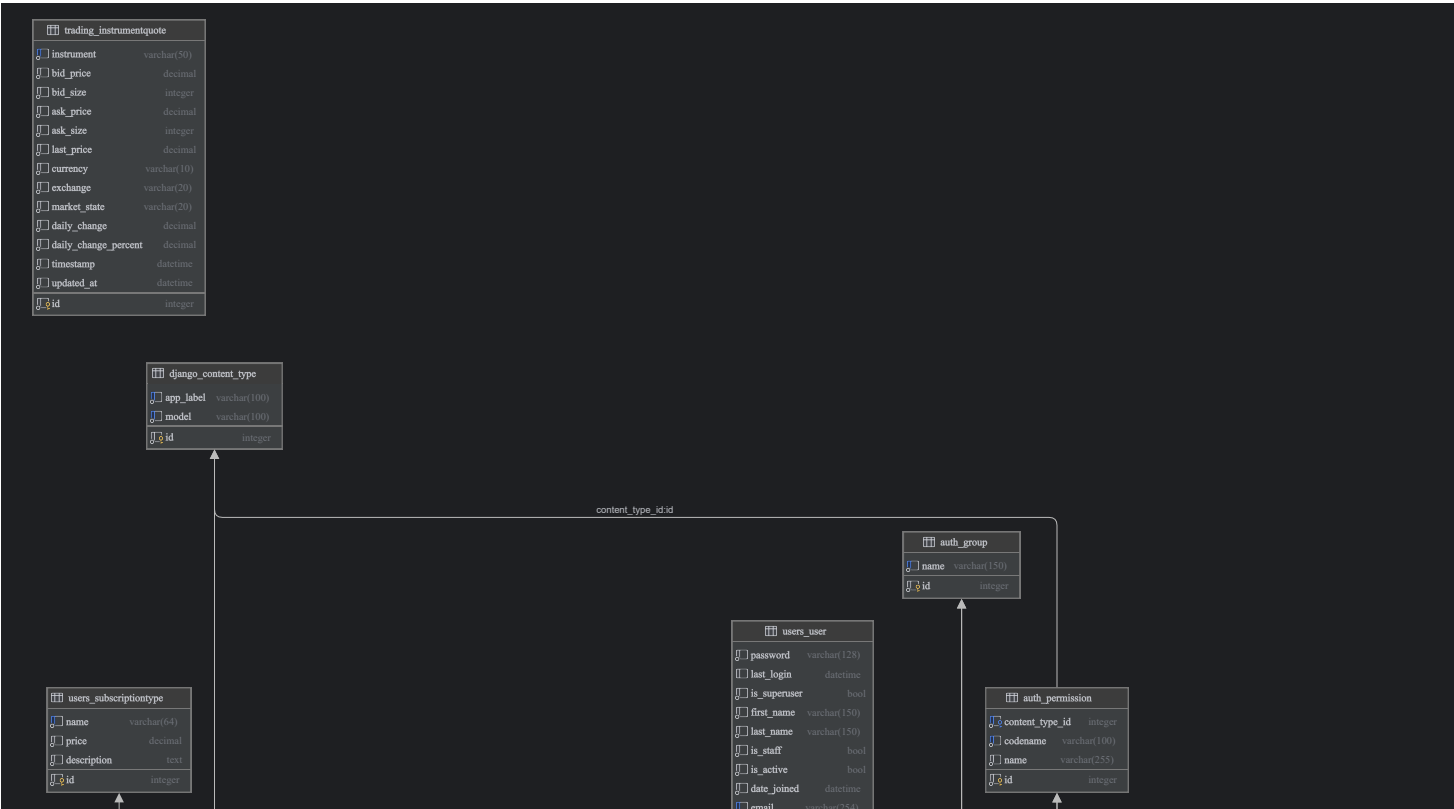
Dijagram baze podataka

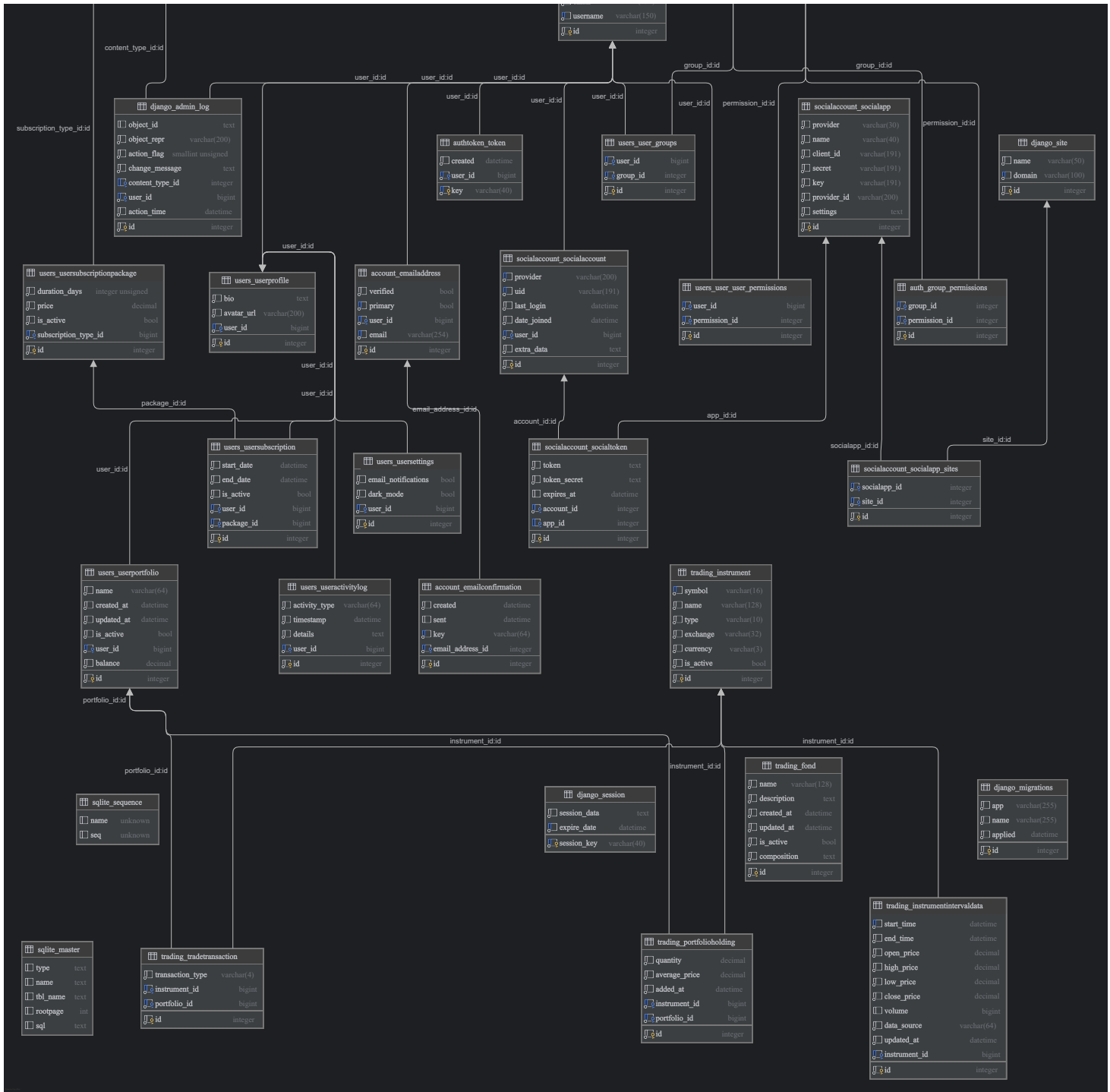
Pogled iz Django domene



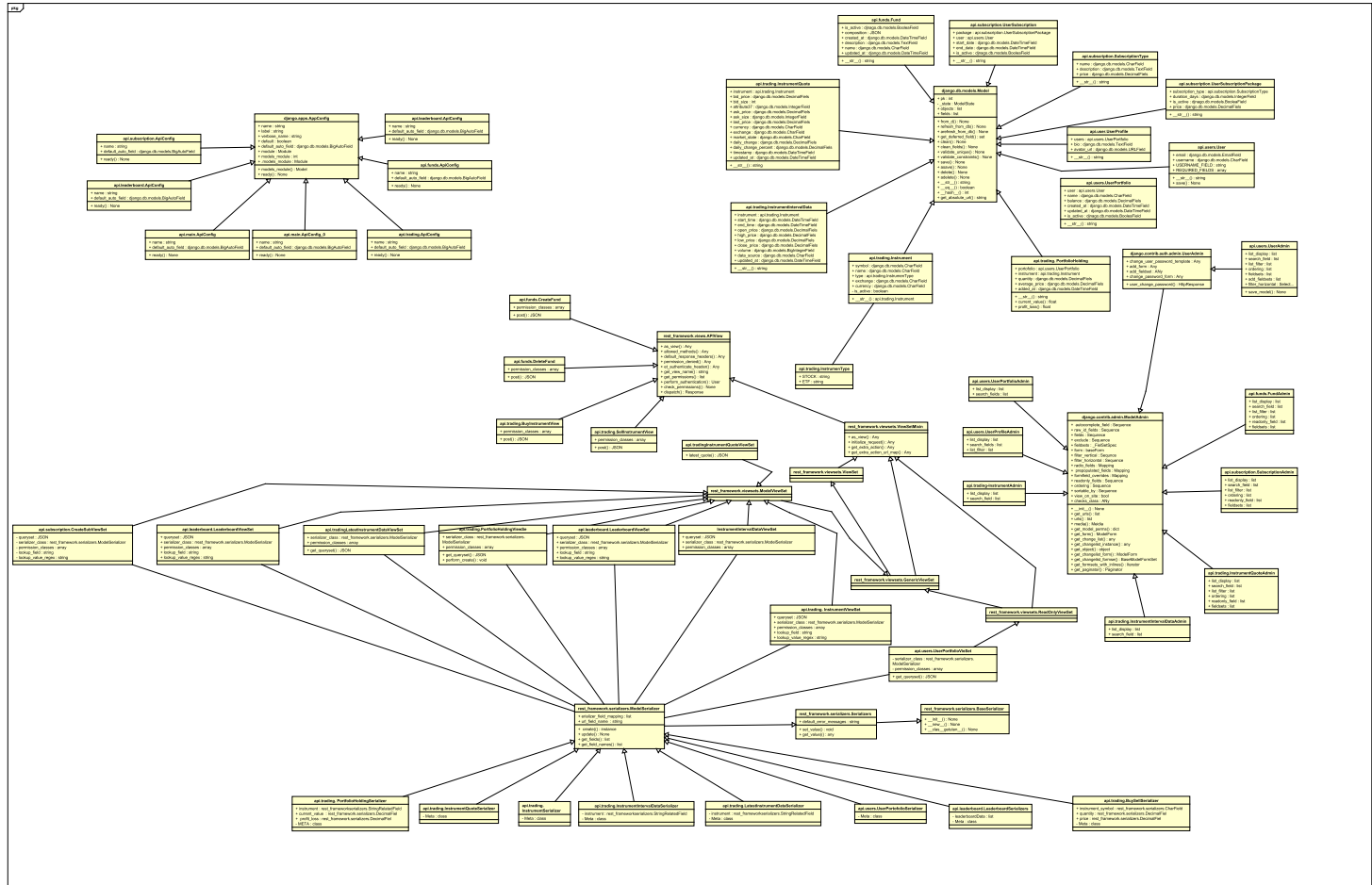


Pogled iz klasične domene baze podataka





Dijagram razreda



Napomena: klase iz 'django' ili 'rest_framework' paketa prikazane su samo u onome obimu koji je relevantna za njihovu implementaciju

Dinamičko ponašanje aplikacije

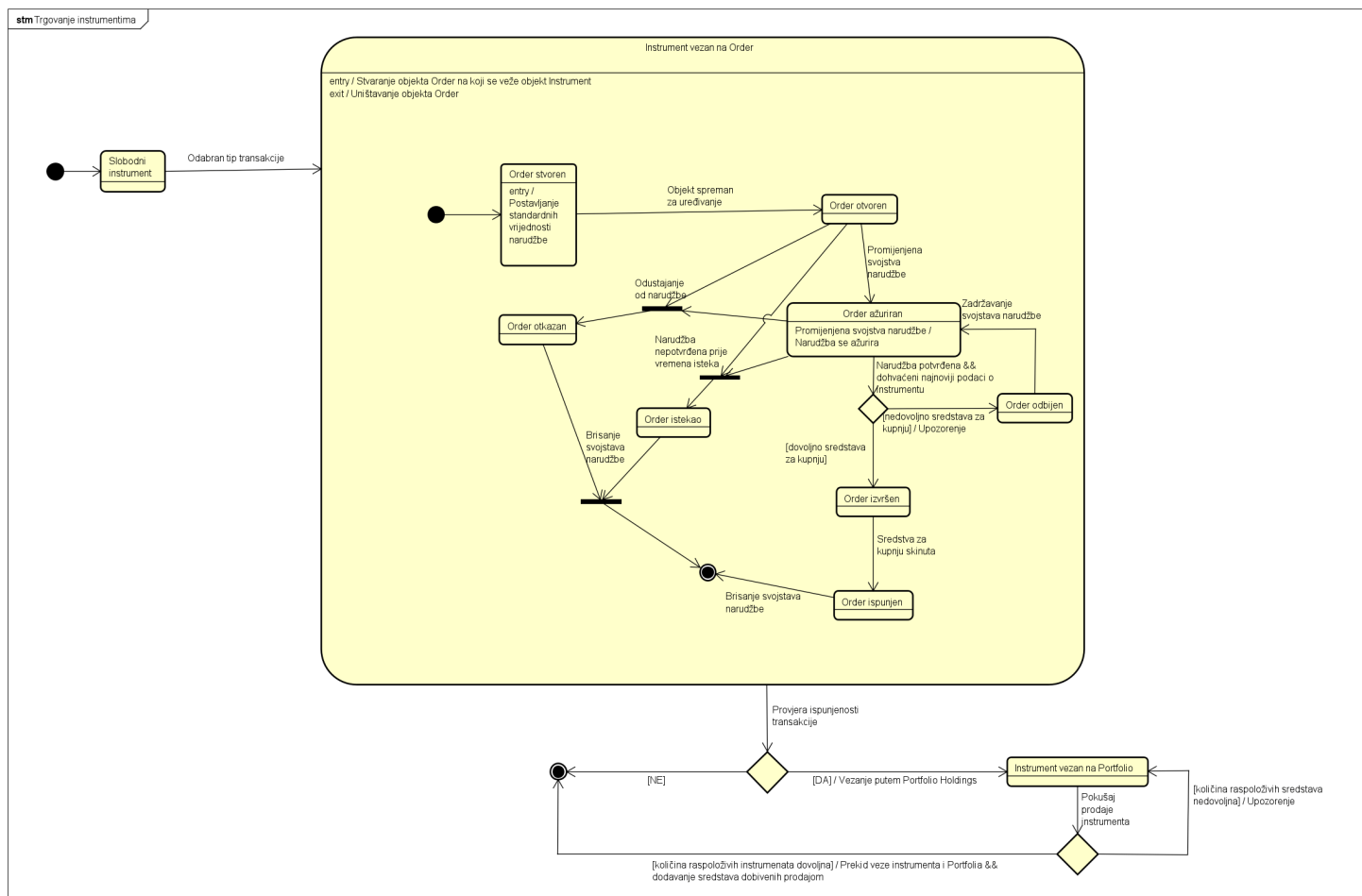
Dinamičko ponašanje aplikacije odnosi se na način na koji objekti u sustavu evoluiraju kroz vrijeme, uključujući prijelaze između različitih stanja. To uključuje aktivnosti, događaje, odluke i interakcije unutar aplikacije. UML dijagrami stanja omogućuju vizualizaciju tih promjena i olakšavaju razumijevanje dinamike sustava.

Razumijevanje promjena stanja neophodno je za pravilno funkcioniranje aplikacije jer pruža uvid u interakcije među objektima, komponentama i korisnicima tijekom rada sustava. Korištenjem UML dijagrama stanja i aktivnosti moguće je vizualizirati prijelaze i stanja objekata, identificirati potencijalne probleme, osigurati točnu implementaciju te poboljšati komunikaciju među članovima tima.

UML dijagrami stanja

UML dijagrami stanja nužni su za razumijevanje dinamičkog ponašanja sustava. Oni jasno prikazuju promjene stanja objekata tijekom vremena ovisno o događajima i uvjetima.

Prikaz UML dijagrama stanja za trgovanje instrumentima



Opis dijagrama:

Na početku procesa, instrument je slobodan objekt, nevezan na druge objekte.

Kad korisnik odluči obaviti transakciju s određenim instrumentom, stvara se objekt tipa Order na koji se veže traženi instrument. Pri stvaranju Ordera, u njemu se inicijaliziraju predodređene (default) vrijednosti narudžbe. Kada se one ispune, objekt je spreman za daljnje uređenje, čime Order prelazi u stanje otvoren.

Pretpostavlja se da korisnik želi podesiti svojstva narudžbe. Ako to napravi, Order prelazi u stanje ažuriran. U tom stanju korisnik i dalje može podešavati svojstva narudžbe, čime se neće promijeniti njegovo stanje.

Kad korisnik potvrdi da je zadovoljan podešenim postavkama i da želi obaviti narudžbu, mora se još pričekati da Massive websocket dohvati aktualne podatke o vrijednosti instrumenta. Kad se to dogodi, slijedi provjera raspoloživosti sredstava na računu korisnika.

Ako ima dovoljno sredstava, Order može prijeći u stanje izvršen, gdje samo mora pričekati da se sredstva za kupnju uistinu skinu s računa prije nego što prijeđe u stanje ispunjen. Iz stanja ispunjen, nakon brisanja svojstava obavljene narudžbe, Order završava svoj životni vijek.

U slučaju da na računu nema dovoljno sredstava za obavljanje željene narudžbe, sustav će korisniku poslati upozorenje o neuspjehu transakcije, a Order će nakratko prijeći u stanje odbijen. Od tamo će, nakon što se vrate postavke neuspješne narudžbe, otići natrag u stanje ažuriran, iz kojeg se kasnije može ponovno pokušati provesti transakcija.

Kad se Order nalazi u stanjima otvoren ili ažuriran, postoje još dva načina na koji može doći do kraja svog životnog vijeka. Ako korisnik odluči odustati od narudžbe, Order će prijeći u stanje otkazan. Drugi slučaj je da vremenski rok u kojem se narudžba mora izvršiti istekne prije nego što se narudžba potvrdi, čime Order prelazi u stanje istekao. Iz oba stanja, svojstva narudžbe se brišu, čime Order dolazi do kraja svog životnog vijeka.

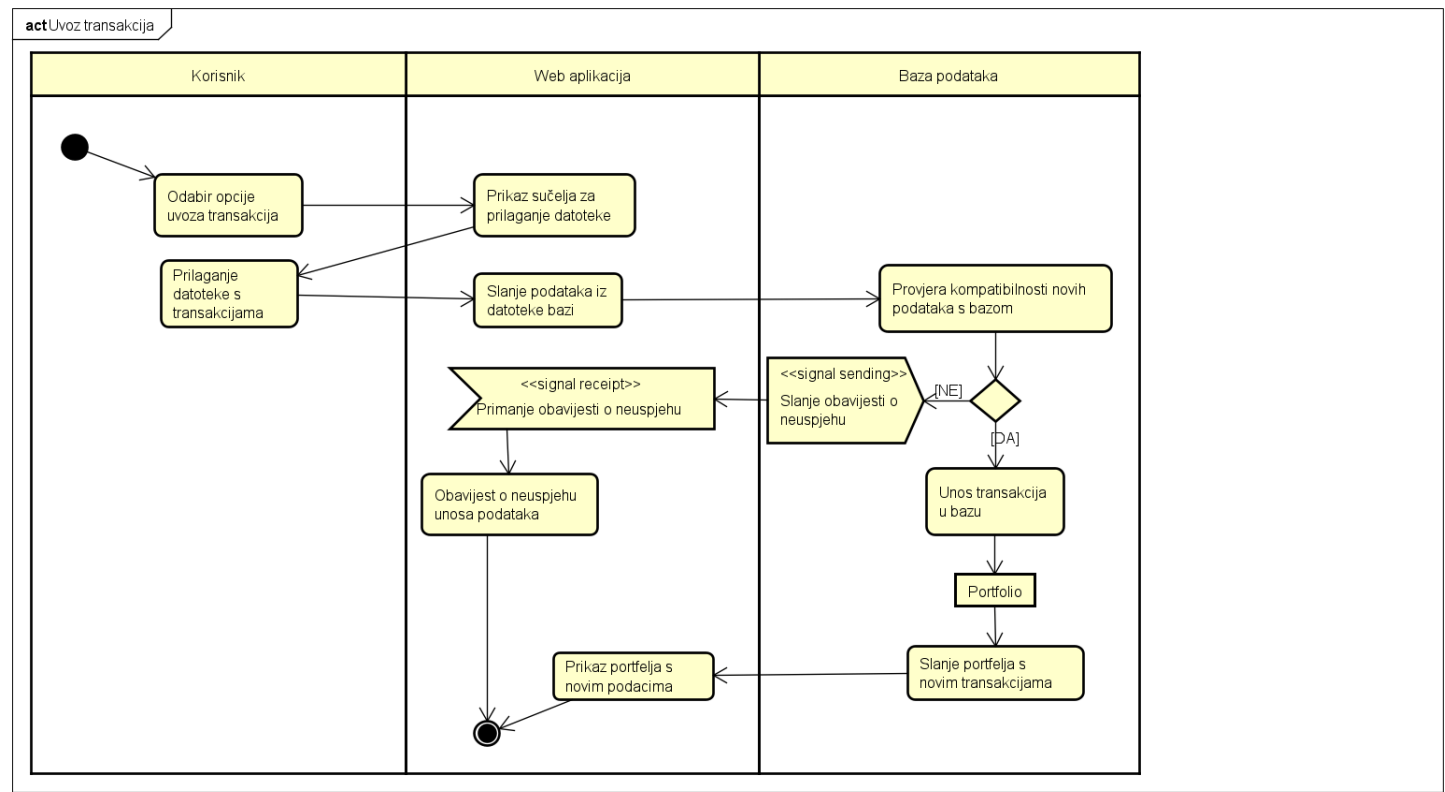
Na kraju životnog vijeka Ordera, neovisno o uspjehu ili neuspjehu transakcije, objekt Order se uništava, čime instrument više nije vezan za njega. Daljnji tijek ovisi o uspjehu ispunjenosti transakcije. Ako se transakcija nije ispunila, instrument odmah završava sa svojim životnim vijekom. Ako se transakcija ispunila, životni vijek instrumenta se nastavlja vezanjem na objekt Portfolio, kroz njegov element Portfolio Holdings.

Ako korisnik želi prodati instrumente iz svog portfelja, instrument prolazi kroz sličan proces vezanja na Order. Glavna razlika je u uvjetu koji se gleda pri odluci o uspjehu prodaje. Ako je količina raspoloživih dionica u portfelju dovoljna za traženu narudžbu, prekida se veza instrumenta i Portfolia, na račun se dodaju sredstva dobivenih prodajom, a instrument time završava svoj životni vijek. Ako u portfelju pak nema dovoljno dionica za narudžbu, korisnik se upozorava o neuspjehu narudžbe, a instrument ostaje vezan na Portfolio.

UML dijagrami aktivnosti

Dijagram aktivnosti prikazuje tijek izvršavanja određenog procesa. Osim za razumijevanje toka podataka unutar aplikacije, koristi se za analizu poslovnih procesa.

Prikaz UML dijagrama aktivnosti za uvoz transakcija



Opis dijagrama:

- Korisnik započinje proces odabirom opcije uvoza transakcija na sučelju. Web aplikacija tada prikazuje sučelje na kojem korisnik može priložiti datoteku s transakcijama.
- U trenutku kad korisnik priloži datoteku s transakcijama, web aplikacija njene podatke prosljeđuje bazi podataka. Baza podataka provjerava ima li u poslanim podacima nepravilnosti zbog kojih unos u baze ne bi bio moguć, poput neispravnog formata podataka ili nedozvoljenih vrijednosti.
- U slučaju da u podacima nema nepravilnosti, baza počinje unositi nove podatke u portfolio korisnika. Kada završi, portfolio s novim podacima šalje natrag web aplikaciji, koja nadalje u portfelju prikazuje i nove podatke, čime je unos transakcija uspješno završen.
- U slučaju nekompatibilnosti podataka i baze, baza podataka web aplikaciji šalje obavijest o neuspješnom unosu. Kad web aplikacija primi tu obavijest, na sličan način obavještava korisnika o neuspješnom unosu, čime unos transakcija neuspješno završava.

Arhitektura sustava predstavlja temeljni okvir za razumijevanje i implementaciju svih njegovih funkcionalnosti. U kontekstu razvojne dokumentacije aplikacija, dijagrami komponenata i razmještaja odlučujući su za prikaz povezanosti i rasporeda različitih komponenata sustava. Ovi dijagrami omogućuju sudionicima projekta razumijevanje i vizualizaciju fizičkog i logičkog dizajna sustava, uključujući interakcije između dijelova aplikacije, što je odlučujuće za efikasnu implementaciju i dugoročnu održivost sustava.

Arhitektura sustava, u kontekstu dijagrama komponenata i razmještaja, pruža uvid u strukturu i raspored ključnih dijelova aplikacije. Ovi dijagrami nisu korisni samo tijekom faza oblikovanja i implementacije, već služe i kao alati za održavanje i optimizaciju sustava u budućnosti.

Kao dio razvoja aplikacije, važno je osmisliti i dokumentirati arhitekturu sustava s naglaskom na dijagrame komponenata i razmještaja. Vaš zadatak je izraditi **dijagram komponenata** koji će jasno prikazivati ključne funkcionalne komponente aplikacije, njihovu međusobnu povezanost te sučelja za komunikaciju. Također, trebate izraditi **dijagram razmještaja** komponenata, koji treba detaljno prikazivati kako su te komponente raspoređene u infrastrukturi sustava,

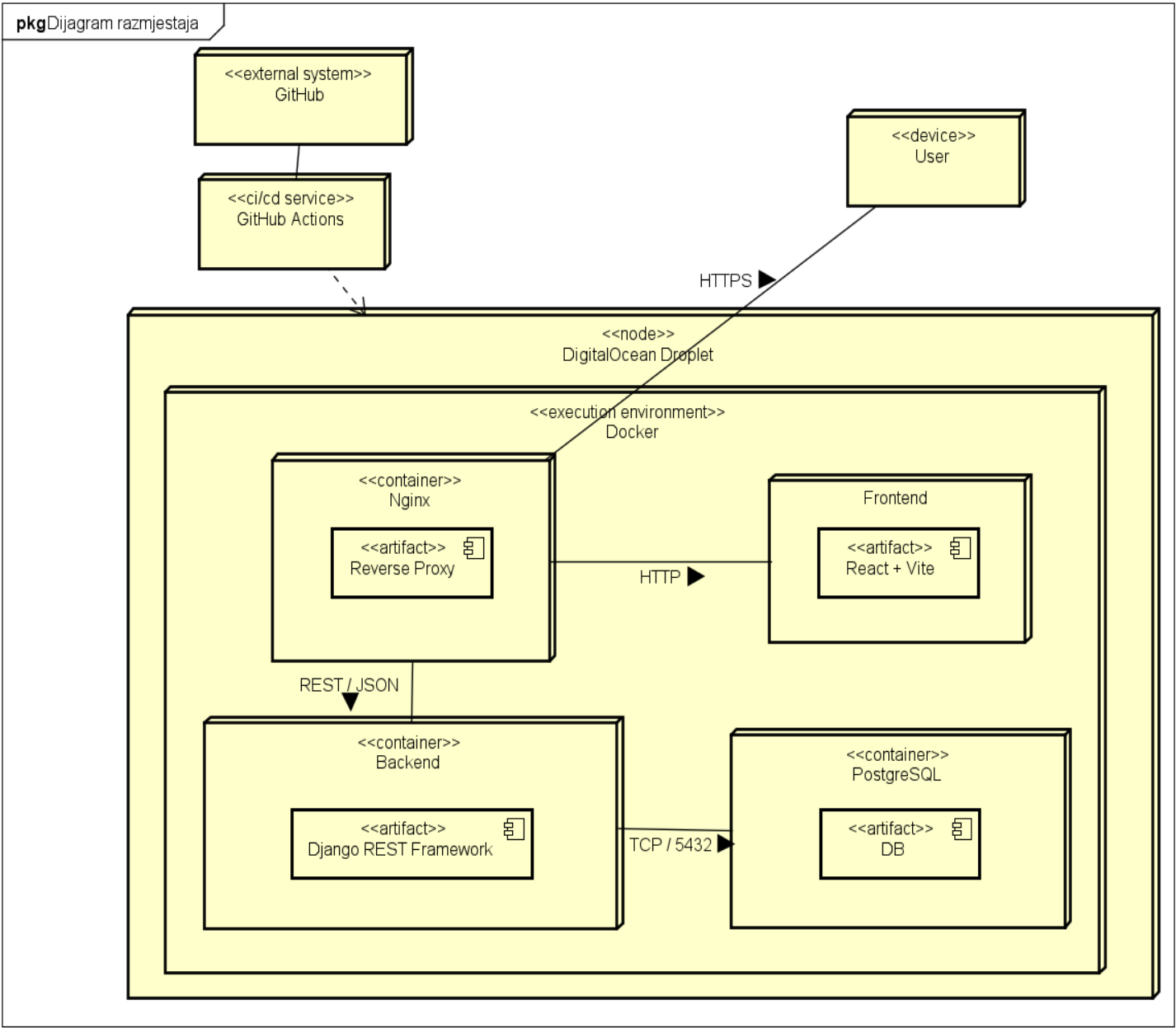
uključujući fizičke i virtualne resurse poput poslužitelja ili uređaja krajnjih korisnika.

Dijagram komponentata

Komponente sustava predstavljaju bitne dijelove aplikacije koji obavljaju specifične funkcije. Svaka komponenta je autonomna jedinica s vlastitim odgovornostima, ali je povezana s drugim komponentama kako bi sustav u cjelini funkcionirao. Komponente mogu biti elementi poput modula, servisa, razreda ili paketa, te komuniciraju putem jasno definiranih sučelja.

UML dijagram komponentata za vašu aplikaciju ovisi o njezinoj arhitekturi i složenosti, no općenito treba prikazivati važne funkcionalne komponente, njihovu međusobnu povezanost i sučelja za komunikaciju. Obzirom na namjenu projekta, dijagram treba biti jasan, organiziran i lako čitljiv kako bi olakšao razumijevanje strukture i suradnju unutar tima.

Dijagram razmještaja



Ispitivanje komponenti

TestCase: FundAPITests

Funkcionalnost koja se testira

Testira se API sloj za rad s fondovima (CRUD i dodatni endpointi):

- `GET /api/funds/funds/` (lista fondova korisnika)
- `GET /api/funds/funds/{id}/` (detalj fonda)
- `POST /api/funds/funds/` (kreiranje fonda)
- `PUT /api/funds/funds/{id}/` (ažuriranje fonda)
- `DELETE /api/funds/funds/{id}/` (brisanje fonda)
- `GET /api/funds/funds/all/` (lista svih aktivnih fondova)
- `GET /api/funds/funds/{id}/performance/` (NAV povijest / performanse fonda)

Test pokriva:

- autentikaciju i autorizaciju pristupa,
- pravilno filtriranje fondova po korisniku,
- CRUD ponašanje i status kodove,
- rad performansnog endpointa i filtriranje po broju dana.

Postupak provođenja ispitivanj

1. U `setUp()` se inicijalizira:

- `APIClient`
- dva korisnika (`user1`, `user2`)
- pripadajući `profile` i `portfolio`
- jedan instrument (`AAPL`) potreban za holdings
- osnovni URL: `/api/funds/funds/`

2. Autentikacija se provodi metodom `force_authenticate()` :

- `auth(user)` za prijavljenog korisnika
- `clear_auth()` za anonimni pristup

3. Za svaki `test_*` scenarij šalje se HTTP zahtjev (GET/POST/PUT/DELETE) i provjerava:

- `status_code`
- relevantni dio JSON odgovora
- stanje baze (kreirano/obrisano/filtrirano)

Ispitni slučajevi

1) `test_unauthenticated_user_is_forbidden`

Funkcionalnost: zabrana pristupa listi fondova bez autentikacije.

• **Ulazni podaci:**

- korisnik nije autentificiran
- zahtjev: `GET /api/funds/funds/`

• **Očekivani rezultat:**

- HTTP `401 UNAUTHORIZED`

• **Dobiveni rezultat:** prolaz (status 401)

• **Postupak:**

1. `clear_auth()`
2. `client.get(self.url)`

3. provjera `status_code == 401`

2) `test_authenticated_user_can_list_own_funds`

Funkcionalnost: autentificirani korisnik može dohvatiti listu svojih fondova.

- **Ulazni podaci:**

- autentikacija: `user1`
- zahtjev: `GET /api/funds/funds/`

- **Očekivani rezultat:**

- HTTP `200 OK`

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. `auth(user1)`
 2. `client.get(self.url)`
 3. provjera `status_code == 200`
-

3) `test_create_fund`

Funkcionalnost: kreiranje fonda s inicijalnim holdings.

- **Ulazni podaci:**

- autentikacija: `user1`
- zahtjev: `POST /api/funds/funds/`
- payload:
 - `creator_portfolio = portfolio1.id`
 - `name = "Test Fund"`
 - `description = "A test fund"`
 - `holdings = [{"instrument": AAPL.id, "weight_percent": "50.000"}]`

- **Očekivani rezultat:**

- HTTP `201 CREATED`
- u odgovoru: `name == "Test Fund"`
- broj holdings: `1`

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. `auth(user1)`
 2. `client.post(url, payload, format="json")`
 3. `assert status 201 + provjera JSON polja`
-

4) `test_create_fund_without_holdings`

Funkcionalnost: kreiranje fonda bez holdings.

- **Ulazni podaci:**

- autentikacija: `user1`
- POST payload bez `holdings`

- **Očekivani rezultat:**

- HTTP `201 CREATED`

- `holdings == []`

- **Dobiveni rezultat:** prolaz
 - **Postupak:** analogno prethodnom testu, bez holdings u payloadu
-

5) `test_list_only_own_funds`

Funkcionalnost: korisnik u listi vidi samo fondove povezane sa svojim portfeljem.

- **Ulazni podaci:**
 - kreira se fond za `user1` i fond za `user2`
 - autentikacija: `user1`
 - `GET /api/funds/funds/`
 - **Očekivani rezultat:**
 - HTTP `200 OK`
 - broj fondova u odgovoru: `1`
 - ime fonda: `"User1 Fund"`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. `Fund.objects.create(...)` za oba korisnika
 2. `auth(user1)`
 3. GET i provjera duljine liste i naziva fonda
-

6) `test_retrieve_any_fund_by_id`

Funkcionalnost: korisnik može dohvatiti detalje fonda po ID-u (view access).

- **Ulazni podaci:**
 - fond kreira `user2`
 - autentikacija: `user1`
 - `GET /api/funds/funds/{fund_id}/`
 - **Očekivani rezultat:**
 - HTTP `200 OK`
 - vraća se odgovarajući fond
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. kreirati fond za `user2`
 2. `auth user1`
 3. GET detail i provjera `name`
-

7) `test_retrieve_nonexistent_fund_returns_404`

Funkcionalnost: dohvat nepostojećeg fonda vraća 404.

- **Ulazni podaci:**
 - autentikacija: `user1`
 - `GET /api/funds/funds/99999/`
- **Očekivani rezultat:** HTTP `404 NOT FOUND`
- **Dobiveni rezultat:** prolaz
- **Postupak:** `auth` → `GET` → `assert 404`

8) `test_update_own_fund`

Funkcionalnost: vlasnik fonda može ažurirati vlastiti fond.

- **Ulazni podaci:**
 - fond pripada `portfolio1`
 - autentikacija: `user1`
 - `PUT /api/funds/funds/{id}/`
 - payload s novim `name` i `description`
 - **Očekivani rezultat:**
 - HTTP `200 OK`
 - `name == "My Fund Updated"`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create fund → auth user1 → PUT → provjera JSON
-

9) `test_delete_own_fund`

Funkcionalnost: vlasnik fonda može obrisati fond.

- **Ulazni podaci:**
 - fond pripada `user1`
 - autentikacija: `user1`
 - `DELETE /api/funds/funds/{id}/`
 - **Očekivani rezultat:**
 - HTTP `204 NO CONTENT`
 - fond više ne postoji u bazi
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create → auth → delete → `Fund.objects.filter(...).exists() == False`
-

10) `test_all_funds_returns_active_funds`

Funkcionalnost: endpoint `/all/` vraća samo aktivne fondove.

- **Ulazni podaci:**
 - kreiraju se 2 aktivna fonda + 1 neaktivan
 - autentikacija: `user1`
 - `GET /api/funds/funds/all/`
 - **Očekivani rezultat:**
 - HTTP `200`
 - sadrži aktivne fondove
 - ne sadrži neaktivni fond
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create funds → GET → provjera liste naziva
-

11) `test_performance_returns_nav_history`

Funkcionalnost: endpoint `performance/` vraća NAV povijest fonda.

- **Ulazni podaci:**
 - fond s dva `FundNAVHistory` zapisa

- autentikacija user1
 - `GET /api/funds/funds/{id}/performance/`
 - **Očekivani rezultat:**
 - HTTP 200
 - duljina liste = 2
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. create fund
 2. create dva NAV zapisa i ručno podesiti `recorded_at` kroz `.update()`
 3. GET i provjera duljine
-

12) `test_performance_filters_by_days`

Funkcionalnost: query param `days` filtrira rezultate NAV povijesti.

- **Ulazni podaci:**
 - NAV zapis star 45 dana
 - NAV zapis star 5 dana
 - `GET performance/` (default 30 dana)
 - `GET performance/?days=60`
 - **Očekivani rezultat:**
 - za default: 1 zapis
 - za 60 dana: 2 zapisa
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create NAV data → GET bez parametra → GET s `days=60`
-

TestCase: `FundSubscriptionAPITests`

Funkcionalnost koja se testira

Testira se API sloj za pretplate (subscriptions):

- `GET /api/funds/subscriptions/` (lista pretplata)
 - `GET /api/funds/subscriptions/{id}/` (detalji)
 - `POST /api/funds/subscriptions/` (kreiranje pretplate)
 - `PATCH /api/funds/subscriptions/{id}/` (ažuriranje pretplate)
 - `DELETE /api/funds/subscriptions/{id}/` (brisanje pretplate)
 - `GET /api/funds/subscriptions/unsubscribed/` (fondovi na koje korisnik nije pretplaćen)
 - `GET /api/funds/subscriptions/by-fund/{fund_id}/` (pretplata za određeni fond)
-

Postupak provođenja ispitivanja

1. `setUp()` inicijalizira:
 - `APIClient`
 - dva korisnika i njihove portfelje
 - jedan fond (`fund`) koji pripada `user2`
 - URL: `/api/funds/subscriptions/`
2. U testovima se koristi `force_authenticate()` za simulaciju prijave.
3. Svaki test izvršava odgovarajući HTTP poziv i provjerava:

- status kod
 - JSON sadržaj
 - stanje baze (postojanje/brisanje pretplate)
-

Ispitni slučajevi

1) `test_unauthenticated_user_is_forbidden`

Funkcionalnost: zabrana pristupa bez prijave.

- **Ulazni podaci:** `GET /api/funds/subscriptions/` bez autentikacije
 - **Očekivani rezultat:** `401 UNAUTHORIZED`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `clear_auth` → `GET` → `assert 401`
-

2) `test_authenticated_user_can_list_subscriptions`

Funkcionalnost: autentificirani korisnik može dohvatiti svoje pretplate.

- **Ulazni podaci:** `auth user1`, `GET list`
 - **Očekivani rezultat:** `200 OK`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `auth` → `GET` → `assert 200`
-

3) `test_create_subscription`

Funkcionalnost: kreiranje pretplate na fond.

- **Ulazni podaci:**
 - `auth user1`
 - `POST payload`:
 - `subscriber_portfolio`
 - `fund`
 - `invested_amount="100.00"`
 - **Očekivani rezultat:**
 - `201 CREATED`
 - vraćen `fund == self.fund.id`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `POST` → `assert 201` + provjera JSON
-

4) `test_list_only_own_subscriptions`

Funkcionalnost: korisnik u listi vidi samo vlastite pretplate.

- **Ulazni podaci:**
 - kreira se pretplata za `user1`
 - `auth user1`, `GET list`
 - **Očekivani rezultat:**
 - `HTTP 200`
 - `len == 1`
 - `subscriber_portfolio == portfolio1.id`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `create subscription` → `auth` → `GET` → provjere
-

5) `test_update_subscription`

Funkcionalnost: ažuriranje pretplate (PATCH).

- **Ulazni podaci:**
 - postojeća pretplata user1
 - PATCH payload s `invested_amount="200.00"`
 - **Očekivani rezultat:** 200 OK
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create → PATCH → assert 200
-

6) `test_delete_subscription`

Funkcionalnost: brisanje pretplate (unsubscribe).

- **Ulazni podaci:** postojeća subscription, auth user1, DELETE
 - **Očekivani rezultat:**
 - 204 NO CONTENT
 - zapis uklonjen iz baze
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create → DELETE → provjera da zapis ne postoji
-

7) `test_unsubscribed_returns_funds_not_subscribed_to`

Funkcionalnost: endpoint vraća fondove na koje korisnik nije pretplaćen.

- **Ulazni podaci:**
 - user1 je pretplaćen na `fund`
 - postoji `other_fund` (nije pretplaćen)
 - `GET /unsubscribed/`
 - **Očekivani rezultat:**
 - 200 OK
 - response sadrži `other_fund.id`
 - response ne sadrži `fund.id`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create other_fund → create subscription → GET unsubscribed → provjera ID-eva
-

8) `test_unsubscribed_excludes_own_funds`

Funkcionalnost: unsubscribed endpoint isključuje fondove čiji je korisnik vlasnik.

- **Ulazni podaci:**
 - fund owned by user1 (`own_fund`)
 - `GET /unsubscribed/`
 - **Očekivani rezultat:**
 - `own_fund` nije u listi
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create own_fund → GET → assert `own_fund.id not in response`
-

9) `test_by_fund_returns_subscription`

Funkcionalnost: endpoint vraća subscription za određeni fund.

- **Ulazni podaci:**
 - postoji subscription user1 → fund
 - `GET /by-fund/{fund_id}/`
 - **Očekivani rezultat:**
 - 200 OK
 - vraćen ID pretplate odgovara kreiranom zapisu
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create subscription → GET → assert response.id
-

10) `test_by_fund_returns_404_if_not_subscribed`

Funkcionalnost: ako korisnik nije pretplaćen na fond, endpoint vraća 404.

- **Ulazni podaci:**
 - auth user1
 - `GET /by-fund/{fund_id}/` bez subscription-a
 - **Očekivani rezultat:** `404 NOT FOUND`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET → assert 404
-

Ispitni slučajevi: Leaderboard API (Django tests)

Testovi provjeravaju:

- autentikaciju i posebne ovlasti (executor korisnik),
 - strukturu odgovora i sortiranje po vrijednosti portfelja,
 - izračun i kontinuitet rangiranja (rank),
 - paginaciju,
 - ponašanje vremenskih filtera (`time` parametar) kroz servis `get_leaderboard_queryset()` i API povezivanje parametra s servisom.
-

TestCase: `LeaderboardAPITests`

Funkcionalnost koja se testira

Testira se endpoint:

- `GET /api/leaderboard/`

Te pripadni servis:

- `api.ledgerboard.services.get_leaderboard_queryset(time_filter)`

Obuhvaćeno ponašanje:

- permission rules:
 - anonimni korisnik → odbijen
 - autentificirani korisnik → dozvoljen
 - executor korisnik → dozvoljen (specijalna rola)
- pravilno sortiranje po **najnovijem** `PortfolioSnapshot.total_value` (silazno)

- izračun **rank** vrijednosti (1 = najbolji)
 - paginacija (PageNumberPagination, očekivano `page_size=50`)
 - vremensko filtriranje snapshotova:
 - `"all"` bez cutoff-a
 - `"1D"` i `"1W"` primjenjuju cutoff
 - preferira se **najnoviji snapshot unutar cutoff-a** ako postoji više snapshotova
-

Postupak provođenja ispitivanja

1. Inicijalizacija (`setUp`)

- kreira se `APIClient`
- kreiraju se 3 korisnika: `user1` , `user2` , `user3`
- dohvaćaju se `profile` i pripadni `portfolio` za svakog korisnika
- brišu se inicijalni snapshotovi automatski generirani signalom:
 - `PortfolioSnapshot.objects.filter(portfolio__in=[...]).delete()`
- definira se URL:
 - `self.url = "/api/leaderboard/"`

2. Autentikacija

- `auth(user)` koristi `self.client.force_authenticate(user=user)`
- `clear_auth()` uklanja korisnika (`user=None`)

3. Testni pomoćni alati

- `create_snapshot(portfolio, total_value, ts)` kreira `PortfolioSnapshot`
- `create_latest_only()` kreira snapshot s `ts=timezone.now()`
- `assert_paginated(payload)` provjerava da API vraća paginiranu strukturu:
 - `count, next, previous, results`

4. Provođenje testova

- svaki test šalje zahtjev prema API-ju ili poziva servis, zatim provjerava:
 - status kod,
 - strukturu JSON odgovora,
 - poredak rezultata,
 - rank vrijednosti,
 - ponašanje filtera.
-

Ispitni slučajevi

A) Permission testovi

1) `test_unauthenticated_user_is_forbidden`

Funkcionalnost: anonimni korisnik ne smije pristupiti leaderboard endpointu.

- **Ulazni podaci:**
 - bez autentikacije
 - `GET /api/leaderboard/`

- **Očekivani rezultat:**

- HTTP 401 UNAUTHORIZED

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. `clear_auth()`
 2. `client.get(self.url)`
 3. `assert status_code == 401`
-

2) `test_authenticated_user_is_allowed`

Funkcionalnost: autentificirani korisnik smije pristupiti leaderboard endpointu.

- **Ulazni podaci:**

- autentikacija: `user1`
- `GET /api/leaderboard/`

- **Očekivani rezultat:**

- HTTP 200 OK

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. `auth(user1)`
 2. `client.get(self.url)`
 3. `assert status_code == 200`
-

3) `test_executor_user_is_allowed_even_if_not_authenticated`

Funkcionalnost: executor korisnik ima dopušten pristup.

- **Ulazni podaci:**

- `user2.is_executor = True`
- autentikacija: `user2`
- `GET /api/leaderboard/`

- **Očekivani rezultat:**

- HTTP 200 OK

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. postaviti `is_executor=True`
 2. `auth(user2)`
 3. GET i assert 200
-

B) Struktura odgovora, poredak i rangiranje

4) `test_returns_ordered_by_latest_total_value_desc`

Funkcionalnost: leaderboard se sortira po najnovijem `total_value` silazno + ispravan rank.

- **Ulazni podaci:**

- snapshotovi:

- portfolio1 = 150
- portfolio2 = 300
- portfolio3 = 200

- `GET /api/leaderboard/`

- **Očekivani rezultat:**

- HTTP 200
- paginiran odgovor
- poredak:
 1. user2(300)
 2. user3(200)
 3. user1(150)
- rankovi 1..3

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. kreirati snapshotove s `create_latest_only`
2. `auth(user1)`
3. GET i assert 200
4. `assert_paginated(payload)`
5. provjeriti `results[0..2]` (rank, username, total_value)

5) test_excludes_portfolios_without_snapshots

Funkcionalnost: portfelji bez snapshotova se ne prikazuju na leaderboardu.

- **Ulazni podaci:**

- snapshot postoji samo za portfolio1(999)
- `GET /api/leaderboard/`

- **Očekivani rezultat:**

- HTTP 200
- samo 1 rezultat u `results`
- rezultat pripada user1 i total_value=999

- **Dobiveni rezultat:** prolaz

- **Postupak:**

1. `create_latest_only(portfolio1, 999)`
2. auth user1
3. GET i provjera duljine results = 1 + provjera username/value

6) test_rank_is_continuous_across_pages

Funkcionalnost: rank mora biti kontinuiran kroz stranice paginacije (page 2 počinje rank=51).

- **Ulazni podaci:**

- kreira se 60 dodatnih korisnika/portfelja sa snapshotovima vrijednosti:
 - `1000 - i` (time se osigurava poredak)
- zahtjev: `GET /api/leaderboard/?page=2`

- **Očekivani rezultat:**

- HTTP 200

- paginirani odgovor
 - page 2 sadrži 10 rezultata (rankovi 51-60)
 - prvi username: `extra_50`
 - zadnji username: `extra_59`
- **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. petlja i kreiranje 60 korisnika + snapshotova
 2. `auth(user1)`
 3. GET s parametrom page=2
 4. `assert_paginated`
 5. provjera rank granica i username vrijednosti
-

C) Servisni testovi: `get_leaderboard_queryset()` (time filter)

7) `test_service_time_filter_all_includes_old_snapshots`

Funkcionalnost: `time=all` ne filtrira stare snapshotove.

- **Ulazni podaci:**
 - snapshot star 400 dana za portfolio1
 - poziv: `get_leaderboard_queryset("all")`
 - **Očekivani rezultat:**
 - portfolio1 je prisutan u querysetu
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. `create_snapshot(portfolio1, 123, old_ts)`
 2. `qs = get_leaderboard_queryset("all")`
 3. provjera da je `portfolio1.id` u `qs`
-

8) `test_service_time_filter_1d_excludes_older_than_1_day`

Funkcionalnost: filter `"1D"` odbacuje snapshotove starije od 1 dana.

- **Ulazni podaci:**
 - snapshot star 2 dana za portfolio1
 - poziv: `get_leaderboard_queryset("1D")`
 - **Očekivani rezultat:**
 - portfolio1 nije u querysetu
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create old snapshot → service call → assert not in ids
-

9) `test_service_time_filter_1w_excludes_older_than_7_days`

Funkcionalnost: filter `"1W"` odbacuje snapshotove starije od 7 dana.

- **Ulazni podaci:**
 - snapshot star 8 dana
 - `get_leaderboard_queryset("1W")`

- **Očekivani rezultat:**
 - portfolio1 nije uključen
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** analogno prethodnom, cutoff 7 dana
-

10) test_service_time_filter_prefers_latest_within_cutoff

Funkcionalnost: ako postoje snapshotovi stari i novi, servis mora uzeti **najnoviji unutar cutoff-a**.

- **Ulazni podaci:**
 - snapshot star 30 dana (999) — treba biti ignoriran kod `"1W"`
 - snapshot star 2 dana (111) — treba biti uzet
 - `get_leaderboard_queryset("1W")`
 - **Očekivani rezultat:**
 - `latest_total_value == 111`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. create old snapshot (999)
 2. create new snapshot (111)
 3. pozvati servis
 4. dohvatiti portfolio i provjeriti anotirano polje `latest_total_value`
-

11) test_service_orders_by_latest_total_value_desc

Funkcionalnost: servis vraća sortirane portfelje po `latest_total_value` silazno.

- **Ulazni podaci:**
 - portfolio1=10, portfolio2=30, portfolio3=20
 - `get_leaderboard_queryset("all")`
 - **Očekivani rezultat:**
 - redoslijed: portfolio2, portfolio3, portfolio1
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. create snapshots
 2. `qs = list(get_leaderboard_queryset("all"))`
 3. provjera poretka ID-eva
-

D) API testovi: wiring parametra `time` prema servisu

12) test_api_time_param_filters_results

Funkcionalnost: API parametar `time` filtrira podatke (prosljeđuje se servisu).

- **Ulazni podaci:**
 - portfolio1 snapshot star 10 dana (111)
 - portfolio2 snapshot star 1 dan (222)
 - `GET /api/leaderboard/?time=1W`
- **Očekivani rezultat:**

- prisutan user2
 - nije prisutan user1
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. kreirati snapshotove s različitim timestampovima
 2. `auth(user1)`
 3. GET s `{"time": "1W"}`
 4. provjeriti usernames listu
-

13) test_api_invalid_time_param_defaults_to_all

Funkcionalnost: nepoznata vrijednost `time` parametra se tretira kao `"all"` (bez cutoff-a).

- **Ulazni podaci:**
 - snapshot star 500 dana (123)
 - `GET /api/leaderboard/?time=nonsense`
 - **Očekivani rezultat:**
 - user1 mora biti uključen (jer nema cutoff)
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. create old snapshot
 2. auth user1
 3. GET s invalid parametrom
 4. provjera da se user1 nalazi u rezultatima
-

Ispitni slučajevi: Orders modul (Django tests)

TestCase: OrderAPITests

Funkcionalnost koja se testira

Testira se API sloj za rad s nalogima (Order):

- `GET /api/orders/orders/` (lista naloga)
- `GET /api/orders/orders/{id}/` (detalj naloga)
- `POST /api/orders/orders/` (kreiranje naloga)
- `PUT /api/orders/orders/{id}/` (ažuriranje naloga)
- `DELETE /api/orders/orders/{id}/` (brisanje / otkazivanje naloga)
- `POST /api/orders/orders/{id}/cancel/` (otkazivanje naloga akcijom)
- `GET /api/orders/orders/open/` (service endpoint za izvršitelja)
- `POST /api/orders/orders/{id}/execute/` (service endpoint za izvršitelja)

Obuhvaćeno ponašanje:

- autentikacija
 - validacija polja za različite tipove naloga (MARKET/LIMIT/STOP/STOP_LIMIT)
 - filtriranje naloga po korisniku
 - otkazivanje naloga pravilnim pravilima
 - servisni endpointi dostupni isključivo executor korisniku
 - izvršenje naloga i provjera statusa
-

Postupak provođenja ispitivanja

1. U `setUp()` se inicijalizira:

- `APIClient`
- korisnici `user1`, `user2`
- executor korisnik (`executor`) s `is_staff=True`, `is_superuser=True`
- portfelji korisnika (`portfolio1`, `portfolio2`)
- instrument `AAPL`
- quote za instrument (`bid/ask/last`)
- URL: `/api/orders/orders/`

2. Autentikacija se provodi metodom `force_authenticate()`:

- `auth(user)`
- `clear_auth()`

3. Helper metoda `create_order()` kreira naloge za testove s default vrijednostima.

4. Svaki test šalje HTTP zahtjev i provjerava:

- `status_code`
- JSON polja
- stanje baze i atributa modela nakon akcije (`status`, `cancelled_at`, `filled_quantity`, itd.)

Ispitni slučajevi

1) `test_unauthenticated_user_is_forbidden`

Funkcionalnost: anonimni korisnik ne smije pristupiti nalogima.

- **Ulazni podaci:** `GET /api/orders/orders/` bez autentikacije
- **Očekivani rezultat:** HTTP `401 UNAUTHORIZED`
- **Dobiveni rezultat:** prolaz
- **Postupak:** `clear_auth()` → GET → assert 401

2) `test_authenticated_user_can_list_orders`

Funkcionalnost: autentificirani korisnik smije dohvatiti listu naloga.

- **Ulazni podaci:** `auth user1`, `GET /api/orders/orders/`
- **Očekivani rezultat:** HTTP `200 OK`
- **Dobiveni rezultat:** prolaz
- **Postupak:** `auth(user1)` → GET → assert 200

3) `test_create_market_order`

Funkcionalnost: kreiranje MARKET naloga.

- **Ulazni podaci:**
 - `auth user1`
 - `POST /api/orders/orders/`
 - payload:
 - `instrument_symbol="AAPL"`
 - `side="BUY"`
 - `order_type="MARKET"`
 - `quantity="10.00"`

- **Očekivani rezultat:**

- HTTP 201 CREATED
- side="BUY"
- order_type="MARKET"
- quantity=10.00

- **Dobiveni rezultat:** prolaz

- **Postupak:** POST JSON → assert 201 → provjera JSON polja

4) test_create_limit_order_requires_limit_price

Funkcionalnost: LIMIT nalog zahtijeva limit_price .

- **Ulazni podaci:**

- auth user1
- POST payload bez limit_price

- **Očekivani rezultat:** HTTP 400 BAD REQUEST

- **Dobiveni rezultat:** prolaz

- **Postupak:** POST → assert 400

5) test_create_limit_order_with_limit_price

Funkcionalnost: LIMIT nalog s limit_price se uspješno kreira.

- **Ulazni podaci:**

- auth user1
- POST payload s limit_price="145.00"

- **Očekivani rezultat:**

- HTTP 201 CREATED
- limit_price=145.00

- **Dobiveni rezultat:** prolaz

- **Postupak:** POST → assert 201 → provjera limit_price

6) test_create_stop_order_requires_stop_price

Funkcionalnost: STOP nalog zahtijeva stop_price .

- **Ulazni podaci:**

- auth user1
- POST payload bez stop_price , order_type="STOP" , side="SELL"

- **Očekivani rezultat:** HTTP 400 BAD REQUEST

- **Dobiveni rezultat:** prolaz

- **Postupak:** POST → assert 400

7) test_create_stop_limit_order

Funkcionalnost: STOP_LIMIT nalog zahtijeva stop_price i limit_price .

- **Ulazni podaci:**

- auth user1

- `POST` payload:
 - `order_type="STOP_LIMIT"`
 - `stop_price="155.00"`
 - `limit_price="156.00"`
 - **Očekivani rezultat:** `HTTP 201 CREATED`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `POST` → `assert 201`
-

8) `test_list_only_own_orders`

Funkcionalnost: korisnik u listi vidi samo vlastite naloge.

- **Ulazni podaci:**
 - kreira se nalog za `user1` i nalog za `user2`
 - auth `user1`
 - `GET /api/orders/orders/`
 - **Očekivani rezultat:**
 - `HTTP 200`
 - duljina liste = 1
 - `user` u rezultatu odgovara `profile1.id`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create 2 naloga → auth `user1` → `GET` → `assert len` i `user` polje
-

9) `test_retrieve_own_order`

Funkcionalnost: korisnik može dohvatiti detalj svog naloga.

- **Ulazni podaci:**
 - nalog kreiran za `user1`
 - auth `user1`
 - `GET /api/orders/orders/{id}/`
 - **Očekivani rezultat:**
 - `HTTP 200`
 - `id` odgovara traženom nalogu
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create nalog → `GET` detail → `assert status` i `id`
-

Otkazivanje naloga

10) `test_cancel_order_via_delete`

Funkcionalnost: `DELETE` nad otvorenim nalogom postavlja nalog kao otkazan.

- **Ulazni podaci:**
 - nalog status `OPEN`
 - auth `user1`
 - `DELETE /api/orders/orders/{id}/`
- **Očekivani rezultat:**
 - `HTTP 204 NO CONTENT`

- `status=CANCELLED`
 - `cancelled_at` nije `None`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create OPEN → DELETE → refresh_from_db → provjere status/timestamp
-

11) `test_cancel_order_via_action`

Funkcionalnost: `POST /cancel/` otkazuje otvoreni nalog i sprema razlog.

- **Ulazni podaci:**
 - OPEN nalog
 - auth user1
 - `POST /cancel/` payload `{"reason": "Changed my mind"}`
 - **Očekivani rezultat:**
 - HTTP `200 OK`
 - `status=CANCELLED`
 - `cancel_reason="Changed my mind"`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create OPEN → POST cancel → refresh → provjere
-

12) `test_cannot_cancel_filled_order`

Funkcionalnost: ne smije se otkazati FILLED nalog.

- **Ulazni podaci:**
 - nalog status `FILLED`
 - auth user1
 - `DELETE /api/orders/orders/{id}/`
 - **Očekivani rezultat:** HTTP `400 BAD REQUEST`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create FILLED → DELETE → assert 400
-

13) `test_cannot_cancel_cancelled_order`

Funkcionalnost: ne smije se ponovno otkazati već otkazani nalog.

- **Ulazni podaci:**
 - nalog status `CANCELLED`
 - auth user1
 - `POST /cancel/`
 - **Očekivani rezultat:** HTTP `400 BAD REQUEST`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create CANCELLED → POST cancel → assert 400
-

Service endpoints (executor)

14) `test_open_orders_requires_executor`

Funkcionalnost: endpoint otvorenih naloga zahtijeva executor ovlasti.

- **Ulazni podaci:**

- auth user1 (nije executor)
 - `GET /api/orders/orders/open/`
 - **Očekivani rezultat:** HTTP `403 FORBIDDEN`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth user1 → GET open → assert 403
-

15) `test_open_orders_as_executor`

Funkcionalnost: executor može dohvatiti otvorene i djelomično ispunjene naloge.

- **Ulazni podaci:**
 - nalozi:
 - OPEN
 - PARTIALLY_FILLED
 - FILLED
 - auth executor
 - `GET /open/`
 - **Očekivani rezultat:**
 - HTTP 200
 - vraćena 2 naloga (OPEN + PARTIALLY_FILLED)
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create 3 naloga → auth executor → GET → assert len == 2
-

16) `test_execute_order_requires_executor`

Funkcionalnost: izvršavanje naloga dopušteno samo executoru.

- **Ulazni podaci:**
 - OPEN nalog
 - auth user1
 - `POST /execute/`
 - **Očekivani rezultat:** HTTP `403 FORBIDDEN`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create OPEN → auth user1 → POST execute → assert 403
-

17) `test_execute_market_order_as_executor`

Funkcionalnost: executor može izvršiti MARKET nalog.

- **Ulazni podaci:**
 - OPEN MARKET order, `quantity=5`
 - auth executor
 - `POST /execute/`
- **Očekivani rezultat:**
 - HTTP 200
 - nalog postaje `FILLED`
 - `filled_quantity=5`
- **Dobiveni rezultat:** prolaz

- **Postupak:** create order → auth executor → POST execute → refresh → provjere status i filled_quantity

18) test_execute_limit_order_condition_not_met

Funkcionalnost: LIMIT BUY nalog se ne izvršava ako tržišna cijena ne zadovoljava uvjet.

- **Ulazni podaci:**

- OPEN LIMIT BUY nalog:
 - `limit_price=140.00`
- quote last_price = `150.25`
- auth executor
- `POST /execute/`

- **Očekivani rezultat:**

- HTTP `409 CONFLICT`
- `detail` sadrži `"conditions not met"`

- **Dobiveni rezultat:** prolaz

- **Postupak:** create LIMIT BUY → POST execute → assert 409 i sadržaj poruke

TestCase: OrderEventAPITests

Funkcionalnost koja se testira

Read-only API za događaje naloga:

- `GET /api/orders/order-events/`
- `GET /api/orders/order-events/{id}/`

Obuhvaćeno ponašanje:

- autentikacija
- korisnik vidi samo događaje svojih naloga

Postupak provođenja ispitivanja

1. `setUp()` kreira:

- `APIClient`
- `user1`, `portfolio1`
- instrument `AAPL`
- nalog `Order` (OPEN)
- događaj `OrderEvent` tipa `CREATED`
- URL: `/api/orders/order-events/`

2. Testovi koriste `force_authenticate()` za autentikaciju.

Ispitni slučajevi

1) test_unauthenticated_user_is_forbidden

Funkcionalnost: anonimni korisnik ne može pristupiti događajima.

- **Ulazni podaci:** GET bez autentikacije
- **Očekivani rezultat:** HTTP `401 UNAUTHORIZED`
- **Dobiveni rezultat:** prolaz

- **Postupak:** GET → assert 401
-

2) `test_list_own_order_events`

Funkcionalnost: korisnik može listati događaje svojih naloga.

- **Ulazni podaci:**
 - auth user1
 - `GET /api/orders/order-events/`
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 1`
 - `type == "CREATED"`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET → provjera duljine i tipa eventa
-

TestCase: `OrderFillAPITests`

Funkcionalnost koja se testira

Read-only API za fillove naloga:

- `GET /api/orders/order-fills/`
- `GET /api/orders/order-fills/{id}/`

Obuhvaćeno ponašanje:

- autentikacija
 - korisnik vidi samo fillove svojih naloga
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:
 - `user1`, `user2` i njihove portfelje
 - instrument `AAPL`
 - nalog za `user1` sa statusom `FILLED`
 - `OrderFill` za taj nalog
 - URL: `/api/orders/order-fills/`
 2. Testovi autentificiraju korisnika po potrebi.
-

Ispitni slučajevi

1) `test_unauthenticated_user_is_forbidden`

Funkcionalnost: anonimni korisnik ne može pristupiti fillovima.

- **Ulazni podaci:** GET bez autentikacije
 - **Očekivani rezultat:** HTTP `401 UNAUTHORIZED`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 401
-

2) `test_list_own_order_fills`

Funkcionalnost: korisnik može listati fillove svojih naloga.

- **Ulazni podaci:**
 - auth user1
 - `GET /api/orders/order-fills/`
 - **Očekivani rezultat:** HTTP 200 OK
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET → assert 200
-

TestCase: OrderModelTests

Funkcionalnost koja se testira

Unit testovi za metode i svojstva modela `Order` :

- `remaining_quantity`
 - `apply_fill(quantity, price)`
 - `is_executable_at_price(price)` za različite tipove i strane naloga
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:
 - korisnika `testuser`, pripadni `profile` i `portfolio`
 - instrument `AAPL`
 2. Svaki test kreira `Order` instancu s potrebnim vrijednostima.
 3. Provjerava se rezultat izračuna ili promjena stanja naloga.
-

Ispitni slučajevi

1) `test_remaining_quantity`

Funkcionalnost: `remaining_quantity` mora ispravno izračunati preostalu količinu.

- **Ulazni podaci:**
 - `quantity=10`
 - `filled_quantity=3`
 - **Očekivani rezultat:** `remaining_quantity=7`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create order → assert `order.remaining_quantity == 7`
-

2) `test_apply_fill_partial`

Funkcionalnost: `apply_fill` kod djelomičnog ispuna ispravno ažurira nalog.

- **Ulazni podaci:**
 - OPEN nalog, `quantity=10`
 - `apply_fill(5, 150.00)`
- **Očekivani rezultat:**
 - `filled_quantity=5`
 - `avg_fill_price=150.00`

- `status=PARTIALLY_FILLED`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create OPEN → apply_fill → provjere polja
-

3) `test_apply_fill_complete`

Funkcionalnost: `apply_fill` kod potpunog ispunja zatvara nalog.

- **Ulazni podaci:** `apply_fill(10, 150.00)` na OPEN nalog
 - **Očekivani rezultat:**
 - `filled_quantity=10`
 - `status=FILLED`
 - `closed_at` nije `None`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create OPEN → apply_fill → provjera polja
-

4) `test_apply_fill_weighted_average_price`

Funkcionalnost: `apply_fill` mora izračunati ponderirani prosjek cijene.

- **Ulazni podaci:**
 - `apply_fill(4, 100.00)` i `apply_fill(6, 150.00)`
 - **Očekivani rezultat:**
 - ukupno 10
 - $\text{prosijek} = (4 \cdot 100 + 6 \cdot 150) / 10 = 130.00$
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create OPEN → apply_fill 2x → assert avg_fill_price == 130.00
-

5) `test_is_executable_at_price_market_order`

Funkcionalnost: MARKET nalog je uvijek izvršiv neovisno o cijeni.

- **Ulazni podaci:** MARKET order + cijena 999.99
 - **Očekivani rezultat:** `True`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create MARKET → `assertTrue(is_executable_at_price(...))`
-

6) `test_is_executable_at_price_limit_buy`

Funkcionalnost: LIMIT BUY je izvršiv kad je `price <= limit_price`.

- **Ulazni podaci:** LIMIT BUY `limit_price=150`
 - `price=149` → `True`
 - `price=150` → `True`
 - `price=151` → `False`
 - **Očekivani rezultat:** gore navedeno
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create LIMIT BUY → 3 provjere
-

7) `test_is_executable_at_price_limit_sell`

Funkcionalnost: LIMIT SELL je izvršiv kad je `price >= limit_price`.

- **Ulazni podaci:** LIMIT SELL `limit_price=150`
 - `price=151` → True
 - `price=150` → True
 - `price=149` → False
 - **Očekivani rezultat:** gore navedeno
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create LIMIT SELL → 3 provjere
-

8) `test_is_executable_at_price_stop_buy`

Funkcionalnost: STOP BUY je izvršiv kad je `price >= stop_price`.

- **Ulazni podaci:** STOP BUY `stop_price=160`
 - `161` → True
 - `160` → True
 - `159` → False
 - **Očekivani rezultat:** gore navedeno
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create STOP BUY → 3 provjere
-

9) `test_is_executable_at_price_stop_sell`

Funkcionalnost: STOP SELL je izvršiv kad je `price <= stop_price`.

- **Ulazni podaci:** STOP SELL `stop_price=140`
 - `139` → True
 - `140` → True
 - `141` → False
 - **Očekivani rezultat:** gore navedeno
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create STOP SELL → 3 provjere
-

Ispitni slučajevi: Portfolio snapshots i risk metrike (Django tests)

TestCase: `PortfolioRiskAPITests`

Funkcionalnost koja se testira

Testira se endpoint za izračun rizika portfelja:

- `GET /api/users/snapshots/risk/`

Obuhvaćeno ponašanje:

- validacija query parametara (`portfolio` , `range` , `interval`)
- provjera pristupa (vlasnik vs nevlasnik portfelja)
- ponašanje kad nema snapshotova
- korektnost izračuna povrata i drawdown-a
- osnovne konzistencije (volatility ≥ 0 , VaR/CVaR kod monotonog rasta)

- determinističnost izlaza (isti ulaz → isti rezultat)
-

Postupak provođenja ispitivanja

1. U `setUp()` se inicijalizira:

- `APIClient`
- dva korisnika `user1` i `user2`
- njihovi `profile` i `portfolio`
- brišu se inicijalni snapshotovi generirani signalom (`PortfolioSnapshot.objects.filter(...).delete()`)
- URL: `/api/users/snapshots/risk/`

2. Autentikacija se provodi s `force_authenticate()` preko metode `auth(user)`.

3. Helper `create_snapshots(portfolio, values, interval)` kreira determinističku seriju snapshotova po timestampovima.

4. Helper `assert_has_metrics(data)` provjerava da odgovor sadrži ključ `metrics` i numeričke vrijednosti za očekivane metrike.

5. Svaki test šalje `GET` i provjerava:

- status kod
 - strukturu i tipove metrika
 - specifične vrijednosti metrika kad je deterministički definirano
-

Ispitni slučajevi

1) `test_missing_portfolio_returns_400`

Funkcionalnost: parametar `portfolio` je obavezan.

- **Ulazni podaci:**
 - `auth user1`
 - `GET /api/users/snapshots/risk/` bez query parametara
 - **Očekivani rezultat:** HTTP `400 BAD REQUEST`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `auth` → `GET` → `assert 400`
-

2) `test_invalid_portfolio_returns_400`

Funkcionalnost: `portfolio` mora biti valjan ID (int).

- **Ulazni podaci:**
 - `auth user1`
 - `GET ...?portfolio=abc`
 - **Očekivani rezultat:** HTTP `400 BAD REQUEST`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `auth` → `GET` s `portfolio="abc"` → `assert 400`
-

3) `test_invalid_range_returns_400`

Funkcionalnost: `range` mora biti iz dozvoljenog skupa.

- **Ulazni podaci:**
 - `auth user1`
 - `GET ...?portfolio=<id>&range=9Y&interval=10m`

- **Očekivani rezultat:** HTTP 400 BAD REQUEST
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET s invalid `range` → assert 400
-

4) test_invalid_interval_returns_400

Funkcionalnost: `interval` mora biti iz dozvoljenog skupa.

- **Ulazni podaci:**
 - auth user1
 - GET `...?portfolio=<id>&range=1W&interval=5m`
 - **Očekivani rezultat:** HTTP 400 BAD REQUEST
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET s invalid `interval` → assert 400
-

5) test_non_owner_forbidden_or_empty

Funkcionalnost: ne vlasnik portfelja nema pristup metrikama.

- **Ulazni podaci:**
 - snapshotovi postoje za `portfolio1` (user1)
 - auth user2
 - GET `...?portfolio=<portfolio1>&range=1W&interval=10m`
 - **Očekivani rezultat:**
 - ili HTTP 403 FORBIDDEN
 - ili HTTP 200 OK uz valjanu strukturu metrika
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. kreirati snapshotove za `portfolio1`
 2. auth user2
 3. GET
 4. assert status u {403, 200}
 5. ako 200 → `assert_has_metrics(data)`
-

6) test_no_snapshots_returns_zero_metrics

Funkcionalnost: bez snapshotova endpoint vraća metrike s nulama.

- **Ulazni podaci:**
 - auth user1
 - GET `...?portfolio=<id>&range=1W&interval=10m`
 - portfelj nema snapshotove
- **Očekivani rezultat:**
 - HTTP 200
 - `return_pct=0.0, volatility_pct=0.0, max_drawdown_pct=0.0`
- **Dobiveni rezultat:** prolaz
- **Postupak:**
 1. auth user1
 2. GET

3. `assert_has_metrics(data)`
 4. provjera navedenih vrijednosti na 0.0
-

7) `test_return_pct_correct_simple`

Funkcionalnost: korektan izračun povrata.

- **Ulazni podaci:**
 - snapshotovi: `100 → 110`
 - auth user1
 - `GET ...?range=1W&interval=10m`
 - **Očekivani rezultat:** `return_pct = 10.0`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. kreirati snapshotove `[100, 110]`
 2. auth user1
 3. GET
 4. provjera `assertAlmostEqual(return_pct, 10.0)`
-

8) `test_max_drawdown_correct`

Funkcionalnost: korektan izračun maksimalnog drawdown-a.

- **Ulazni podaci:**
 - snapshotovi: `100 → 120 → 90 → 130`
 - peak=120, drop=90 → drawdown = $(90/120 - 1) = -25\%$
 - **Očekivani rezultat:** `max_drawdown_pct = -25.0`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:**
 1. kreirati snapshotove `[100,120,90,130]`
 2. auth user1
 3. GET
 4. `assertAlmostEqual(dd, -25.0)`
-

9) `test_volatility_non_negative`

Funkcionalnost: volatilnost nikada ne smije biti negativna.

- **Ulazni podaci:**
 - snapshotovi: `[100,101,99,100]`
 - auth user1
 - GET s valjanim parametrima
 - **Očekivani rezultat:** `volatility_pct >= 0.0`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create snapshotovi → GET → provjera `>= 0.0`
-

10) `test_var_cvar_consistency_monotonic_up`

Funkcionalnost: kod monotonog rasta vrijednosti, najgori povrati nisu negativni ⇒ VaR i CVaR su ≥ 0 .

- **Ulazni podaci:**

- snapshotovi: [100,101,102,103,104]
 - auth user1
 - GET s valjanim parametrima
 - **Očekivani rezultat:**
 - `var_95_pct >= 0.0`
 - `cvar_95_pct >= 0.0`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create snapshotovi → GET → provjera var/cvar
-

11) `test_endpoint_is_deterministic`

Funkcionalnost: isti ulazni podaci daju isti izlaz (determinističnost endpointa).

- **Ulazni podaci:**
 - snapshotovi: [100,120,90,130]
 - auth user1
 - dva identična GET zahtjeva
 - **Očekivani rezultat:**
 - oba odgovora `200`
 - `res1.json() == res2.json()`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create snapshotovi → GET x2 → usporedba JSON odgovora
-

TestCase: `PortfolioSnapshotAPITests`

Funkcionalnost koja se testira

Testira se API sloj za dohvat i kreiranje snapshotova portfelja:

- `GET /api/users/snapshots/`
- `POST /api/users/snapshots/`
- `GET /api/users/snapshots/latest/`

Obuhvaćeno ponašanje:

- vlasnik može listati snapshotove vlastitog portfelja
 - nevlasnik dobiva praznu listu (bez curenja podataka)
 - filtriranje po vremenu (`from` , `to`)
 - sortiranje (`order=asc/desc`)
 - limitiranje broja rezultata (`limit`)
 - robusnost na invalidne datume (`from/to`)
 - dohvat latest snapshot-a
 - latest endpoint vraća 404 kad nema snapshotova
 - kreiranje snapshot-a:
 - obični korisnik ne može kreirati snapshot za tuđi portfelj
 - executor korisnik može kreirati snapshot za bilo koji portfelj
-

Postupak provođenja ispitivanja

1. U `setUp()` se inicijalizira:

- korisnici `user1`, `user2`
- profili i portfelji (`portfolio1`, `portfolio2`)
- brišu se inicijalni snapshotovi
- kreiraju se 3 snapshot-a za `portfolio1` s timestampovima `t1<t2<t3`
- kreira se 1 snapshot za `portfolio2`
- kreira se executor korisnik (`executor`) s elevated pravima
- URL-ovi:
 - `snapshots_url="/api/users/snapshots/"`
 - `latest_url="/api/users/snapshots/latest/"`

2. Testovi koriste `force_authenticate()` u okviru `APITestCase`.

3. Svaki test šalje GET/POST zahtjeve i verificira:

- status kod
- duljinu i sadržaj liste
- filtriranje i sortiranje po timestampu
- pravila autorizacije kod kreiranja

Ispitni slučajevi

1) `test_owner_can_list_snapshots`

Funkcionalnost: vlasnik može listati snapshotove za svoj portfelj.

- **Ulazni podaci:**
 - auth user1
 - `GET /api/users/snapshots/?portfolio=<portfolio1>`
- **Očekivani rezultat:**
 - HTTP 200
 - vraćena 3 zapisa
 - svi zapisi imaju `portfolio == portfolio1.id`
- **Dobiveni rezultat:** prolaz
- **Postupak:** auth → GET → provjera duljine i atributa portfelja

2) `test_non_owner_sees_none`

Funkcionalnost: nevlasnik ne smije vidjeti snapshotove tuđeg portfelja.

- **Ulazni podaci:**
 - auth user2
 - `GET ...?portfolio=<portfolio1>`
- **Očekivani rezultat:**
 - HTTP 200
 - prazna lista `[]`
- **Dobiveni rezultat:** prolaz
- **Postupak:** auth user2 → GET → assert prazno

3) `test_filter_from_to`

Funkcionalnost: filtriranje snapshotova po vremenskom rasponu (`from` , `to`).

- **Ulazni podaci:**
 - `auth user1`
 - `from=t2` → očekuju se t2,t3
 - `to=t2` → očekuju se t1,t2
 - `from=t2&to=t2` → očekuje se samo t2
 - **Očekivani rezultat:** duljine 2, 2 i 1
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** poslati tri GET poziva s različitim parametrima → provjeriti duljine
-

4) `test_order_asc_desc`

Funkcionalnost: sortiranje snapshotova.

- **Ulazni podaci:**
 - `auth user1`
 - `order=asc` → prvi je najstariji (t1)
 - `order=desc` → prvi je najnoviji (t3)
 - **Očekivani rezultat:** `ts` prvog elementa odgovara t1 odnosno t3
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET `asc` i `desc` → provjera `ts` prvog elementa
-

5) `test_limit`

Funkcionalnost: `limit=N` vraća najviše N snapshotova.

- **Ulazni podaci:**
 - `auth user1`
 - `GET ...?portfolio=<id>&order=asc&limit=2`
 - **Očekivani rezultat:** `len == 2`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET s limit parametrom → assert `len 2`
-

6) `test_invalid_datetime_does_not_crash`

Funkcionalnost: invalidni `from/to` ne smiju uzrokovati 500 grešku.

- **Ulazni podaci:**
 - `auth user1`
 - `from=not-a-datetime , to=still-bad`
 - **Očekivani rezultat:**
 - HTTP 200
 - vraćeni svi snapshotovi (3)
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET s nevaljanim datumima → assert 200 i len 3
-

7) `test_latest_snapshot`

Funkcionalnost: dohvat latest snapshot-a za portfelj.

- **Ulazni podaci:**
 - auth user1
 - `GET /api/users/snapshots/latest/?portfolio=<portfolio1>`
 - **Očekivani rezultat:**
 - HTTP 200
 - `portfolio == portfolio1.id`
 - `ts == t3`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET latest → provjera `ts`
-

8) `test_latest_returns_404_if_none`

Funkcionalnost: latest endpoint vraća 404 ako portfelj nema snapshotova.

- **Ulazni podaci:**
 - auth user1
 - kreira se novi portfelj bez snapshotova
 - `GET latest` za taj portfelj
 - **Očekivani rezultat:** HTTP 404 NOT FOUND
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create empty portfolio → GET latest → assert 404
-

9) `test_user_cannot_create_snapshot_for_other_portfolio`

Funkcionalnost: korisnik ne smije kreirati snapshot za portfelj koji nije njegov.

- **Ulazni podaci:**
 - auth user1
 - `POST /api/users/snapshots/` s payload `portfolio=<portfolio2>` (owned by user2)
 - **Očekivani rezultat:** HTTP 403 FORBIDDEN
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth user1 → POST → assert 403
-

10) `test_executor_can_create_snapshot_any_portfolio`

Funkcionalnost: executor može kreirati snapshot za bilo koji portfelj.

- **Ulazni podaci:**
 - auth executor
 - `POST /api/users/snapshots/` za portfolio1 s timestampom `t`
 - **Očekivani rezultat:**
 - HTTP 200 OK ili 201 CREATED
 - zapis postoji u bazi: `PortfolioSnapshot(portfolio=portfolio1, ts=t)`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth executor → POST → provjera statusa → provjera `exists()` u bazi
-

Ispitni slučajevi: Trading modul (Buy/Sell + Instruments + Market data + Holdings)

TestCase: BuySellAPITests

Funkcionalnost koja se testira

Testiraju se endpointi za kupnju i prodaju instrumenta:

- `POST /api/trading/buy`
- `POST /api/trading/sell`

Obuhvaćeno ponašanje:

- autentikacija korisnika
- kreiranje i ažuriranje `PortfolioHolding`
- kupnja s automatskim dohvaćanjem cijene iz market data
- kupnja s eksplicitno navedenom cijenom
- prodaja koja smanjuje postojeći holding
- zabrana prodaje količine veće od posjedovane

Postupak provođenja ispitivanja

1. U `setUp()` se inicijalizira:

- `APIClient`
- korisnik `user1`, njegov `profile` i `portfolio`
- postavlja se dovoljno stanje (`balance=100000.00`)
- kreira se instrument `AAPL`
- kreira se `InstrumentIntervalData` radi dohvaćanja tržišne cijene
- URL-ovi:
 - `buy_url="/api/trading/buy"`
 - `sell_url="/api/trading/sell"`

2. Autentikacija se provodi metodom `force_authenticate()` (`auth(user)`).

3. Za svaki test izvrši se POST prema buy/sell endpointu i provjeri:

- status kod
- stanje i vrijednosti `PortfolioHolding` u bazi

Ispitni slučajevi

1) `test_buy_instrument`

Funkcionalnost: kupnja instrumenta kreira holding.

- **Ulazni podaci:**
 - `auth user1`
 - `POST /api/trading/buy`
 - `payload:`

- `instrument_symbol="AAPL"`
- `quantity="10"`
- `portfolio_id=<portfolio1>`

- **Očekivani rezultat:**

- HTTP 200 OK
- u bazi postoji `PortfolioHolding` za AAPL
- `holding.quantity == 10`

- **Dobiveni rezultat:** prolaz

- **Postupak:** auth → POST → dohvat holdinga → provjera quantity

2) `test_buy_instrument_with_explicit_price`

Funkcionalnost: kupnja s eksplicitno navedenom cijenom postavlja `average_price`.

- **Ulazni podaci:**

- auth user1
- `POST /api/trading/buy`
- payload:

- `instrument_symbol="AAPL"`
- `quantity="5"`
- `price="155.00"`
- `portfolio_id=<portfolio1>`

- **Očekivani rezultat:**

- HTTP 200 OK
- `holding.quantity == 5`
- `holding.average_price == 155.00`

- **Dobiveni rezultat:** prolaz

- **Postupak:** auth → POST → dohvat holdinga → provjera quantity i `average_price`

3) `test_sell_instrument`

Funkcionalnost: prodaja smanjuje holding.

- **Ulazni podaci:**

- početni holding: `quantity=20`
- auth user1
- `POST /api/trading/sell`
- payload:

- `instrument_symbol="AAPL"`
- `quantity="10"`
- `portfolio_id=<portfolio1>`

- **Očekivani rezultat:**

- HTTP 200 OK
- `holding.quantity == 10`

- **Dobiveni rezultat:** prolaz

- **Postupak:** create holding → auth → POST sell → dohvat holdinga → assert quantity

4) `test_sell_more_than_owned_fails`

Funkcionalnost: nije dopušteno prodati više nego što je posjedovano.

- **Ulazni podaci:**
 - početni holding: `quantity=5`
 - auth `user1`
 - `POST /api/trading/sell`
 - payload `quantity=10`
 - **Očekivani rezultat:** `HTTP 400 BAD REQUEST`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `create holding(5) → auth → POST sell(10) → assert 400`
-

TestCase: `BuySellServiceTests`

Funkcionalnost koja se testira

Unit testovi za servisne funkcije:

- `buy_instrument(portfolio, instrument_symbol, quantity, price)`
- `sell_instrument(portfolio, instrument_symbol, quantity, price)`

Obuhvaćeno ponašanje:

- kreiranje novog holdinga
 - ažuriranje postojećeg holdinga s ponderiranim prosjekom cijene
 - umanjenje/suplementacija balansa portfelja
 - greške kod nedostatnog balansa i kod prodaje bez posjedovanja
-

Postupak provođenja ispitivanja

1. U `setUp()` se inicijalizira:
 - korisnik `testuser` i portfelj s `balance=50000.00`
 - instrument `AAPL`
 - market data (`InstrumentIntervalData`) radi realističnog okruženja
 2. Testovi pozivaju servisne funkcije direktno i provjeravaju:
 - stanje holdinga (`quantity` , `average_price`)
 - stanje portfelja (`balance`)
 - iznimke (`with self.assertRaises( Exception )`)
-

Ispitni slučajevi

1) `test_buy_creates_new_holding`

Funkcionalnost: `buy_instrument` kreira novi holding.

- **Ulazni podaci:** `buy_instrument(... quantity=10, price=150)`
 - **Očekivani rezultat:**
 - `holding.quantity == 10`
 - `holding.average_price == 150`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** poziv servisa → provjera atributa holdinga
-

2) `test_buy_updates_existing_holding`

Funkcionalnost: `buy_instrument` ažurira postojeći holding i računa ponderirani prosjek cijene.

- **Ulazni podaci:**
 - kupnja 10 @ 100
 - kupnja 10 @ 200
 - **Očekivani rezultat:**
 - `quantity == 20`
 - `average_price == 150.00`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** dva poziva `buy_instrument` → provjera `quantity` i `average_price`
-

3) `test_buy_deducts_balance`

Funkcionalnost: kupnja umanjuje balance portfelja.

- **Ulazni podaci:**
 - `initial_balance`
 - buy: 10 @ 100 (cost=1000)
 - **Očekivani rezultat:**
 - novi balance = initial_balance - 1000
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** spremi init balance → buy → refresh portfolio → provjera balance
-

4) `test_buy_insufficient_balance_fails`

Funkcionalnost: kupnja ne prolazi kod nedostatnog balansa.

- **Ulazni podaci:**
 - portfolio.balance=100
 - buy: 10 @ 150 (cost=1500)
 - **Očekivani rezultat:** baca iznimku
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** postaviti mali balance → `assertRaises(Exception)` na `buy_instrument`
-

5) `test_sell_reduces_holding`

Funkcionalnost: `sell_instrument` smanjuje quantity holdinga.

- **Ulazni podaci:**
 - holding quantity=20
 - sell quantity=10 @ 160
 - **Očekivani rezultat:** preostalo `quantity == 10`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create holding → `sell_instrument` → provjera quantity
-

6) `test_sell_adds_to_balance`

Funkcionalnost: prodaja povećava balance portfelja.

- **Ulazni podaci:**
 - init balance
 - sell: 10 @ 160 (credit=1600)
 - **Očekivani rezultat:** balance = init + 1600
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** init balance → sell → refresh portfolio → provjera balance
-

7) test_sell_no_holding_fails

Funkcionalnost: prodaja bez holdinga nije dopuštena.

- **Ulazni podaci:** `sell_instrument(... quantity=10)` bez postojećeg holdinga
 - **Očekivani rezultat:** baca iznimku
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `assertRaises(Exception)` oko poziva `sell_instrument`
-

8) test_sell_insufficient_quantity_fails

Funkcionalnost: prodaja veće količine od posjedovane nije dopuštena.

- **Ulazni podaci:**
 - holding quantity=5
 - sell quantity=10
- **Očekivani rezultat:** baca iznimku
- **Dobiveni rezultat:** prolaz
- **Postupak:** create holding (5) → `assertRaises(Exception)` za sell

TestCase: InstrumentAPITests

Funkcionalnost koja se testira

Read-only API za instrumente:

- GET `/api/trading/instruments/`
- GET `/api/trading/instruments/{symbol}/`

Obuhvaćeno ponašanje:

- listanje bez autentikacije
 - filtriranje neaktivnih instrumenata (`is_active=False`)
 - dohvat po simbolu
 - 404 za nepostojeći simbol
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:
 - dva aktivna instrumenta (AAPL, GOOGL)
 - jedan neaktivan instrument (DEAD)
 - URL: `/api/trading/instruments/`
 2. Testovi izvršavaju GET zahtjeve i provjeravaju status, listu simbola i polja odgovora.
-

Ispitni slučajevi

1) `test_list_instruments_unauthenticated`

Funkcionalnost: listanje instrumenata bez autentikacije.

- **Ulazni podaci:** `GET /api/trading/instruments/` bez auth
 - **Očekivani rezultat:**
 - HTTP 200
 - uključuje simbole AAPL i GOOGL
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 200 → provjera simbola u listi
-

2) `test_list_excludes_inactive_instruments`

Funkcionalnost: neaktivni instrumenti su isključeni.

- **Ulazni podaci:** `GET /api/trading/instruments/`
 - **Očekivani rezultat:** simbol DEAD nije prisutan
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → provjera da DEAD nije u listi
-

3) `test_retrieve_instrument_by_symbol`

Funkcionalnost: dohvat instrumenta po simbolu.

- **Ulazni podaci:** `GET /api/trading/instruments/AAPL/`
 - **Očekivani rezultat:**
 - HTTP 200
 - `symbol="AAPL"`
 - `name="Apple Inc."`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 200 → provjera JSON polja
-

4) `test_retrieve_nonexistent_instrument_returns_404`

Funkcionalnost: dohvat nepostojećeg simbola vraća 404.

- **Ulazni podaci:** `GET /api/trading/instruments/NONEXIST/`
 - **Očekivani rezultat:** HTTP 404 NOT FOUND
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 404
-

TestCase: `InstrumentIntervalDataAPITests`

Funkcionalnost koja se testira

Read-only API za intervalne tržišne podatke:

- `GET /api/trading/instrument-data/`
- `GET /api/trading/instrument-data/?instrument=<symbol>`

Obuhvaćeno ponašanje:

- dohvat svih intervalnih podataka
 - filtriranje po simbolu instrumenta
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:

- instrumente AAPL i GOOGL
- ukupno 3 intervalna zapisa (2 za AAPL, 1 za GOOGL)
- URL: `/api/trading/instrument-data/`

2. Testovi izvršavaju GET i provjeravaju duljine i sadržaj rezultata.

Ispitni slučajevi

1) `test_list_all_interval_data`

Funkcionalnost: listanje svih intervalnih zapisa.

- **Ulazni podaci:** `GET /api/trading/instrument-data/`
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 3`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 200 → assert len 3
-

2) `test_filter_by_instrument`

Funkcionalnost: filtriranje intervalnih podataka po simbolu.

- **Ulazni podaci:** `GET /api/trading/instrument-data/?instrument=AAPL`
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 2`
 - svaki item pripada AAPL
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 200 → provjera duljine i sadržaja instrument polja
-

TestCase: `InstrumentQuoteAPITests`

Funkcionalnost koja se testira

Dohvat posljednjeg quote-a za instrument:

- `GET /api/trading/latest-instrument-quote/{symbol}/quote/`

Obuhvaćeno ponašanje:

- ispravan povrat cijena za postojeći simbol
 - 404 za nepostojeći simbol
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:

- instrument AAPL
- `InstrumentQuote` s poznatim vrijednostima
- URL prefix: `/api/trading/latest-instrument-quote/`

2. Testovi šalju GET zahtjeve i provjeravaju vrijednosti decimalnih polja.

Ispitni slučajevi

1) `test_get_quote_by_symbol`

Funkcionalnost: dohvat quote-a za postojeći simbol.

- **Ulazni podaci:** `GET /api/trading/latest-instrument-quote/AAPL/quote/`
 - **Očekivani rezultat:**
 - HTTP 200
 - `bid_price=150.00`
 - `ask_price=150.50`
 - `last_price=150.25`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → provjera statusa → provjera cijena
-

2) `test_get_quote_nonexistent_returns_404`

Funkcionalnost: nepostojeći simbol vraća 404.

- **Ulazni podaci:** `GET .../NONEXIST/quote/`
 - **Očekivani rezultat:** HTTP 404 NOT FOUND
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 404
-

TestCase: `LatestInstrumentDataAPITests`

Funkcionalnost koja se testira

Read-only API za dohvat posljednjeg interval data zapisa po instrumentu:

- `GET /api/trading/latest-instrument-data/`
- `GET /api/trading/latest-instrument-data/?instrument=<symbol>`

Obuhvaćeno ponašanje:

- vraća 1 zapis po instrumentu (najnoviji)
 - filtriranje po instrumentu vraća samo 1 zapis (najnoviji)
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:
 - AAPL i GOOGL
 - za AAPL: stariji i noviji interval zapis
 - za GOOGL: jedan zapis
 - URL: `/api/trading/latest-instrument-data/`
 2. Testovi provjeravaju duljinu odgovora i provjeravaju da se vraća noviji zapis.
-

Ispitni slučajevi

1) `test_list_latest_data_for_all_instruments`

Funkcionalnost: listanje latest podataka za sve instrumente.

- **Ulazni podaci:** `GET /api/trading/latest-instrument-data/`
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 2` (po jedan instrument)
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 200 → assert len 2
-

2) `test_filter_latest_by_instrument`

Funkcionalnost: filtriranje latest podataka po instrumentu vraća najnoviji zapis.

- **Ulazni podaci:** `GET ...?instrument=AAPL`
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 1`
 - `close_price == 152.50` (najnoviji zapis)
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** GET s filterom → assert len 1 → provjera close_price
-

TestCase: `PortfolioHoldingAPITests`

Funkcionalnost koja se testira

Endpoint za dohvat holdings:

- `GET /api/trading/portfolio-holdings/`

Obuhvaćeno ponašanje:

- autentikacija je obavezna
 - korisnik vidi samo svoje holding zapise
 - drugi korisnik vidi praznu listu
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:
 - user1 i user2
 - njihove portfelje
 - instrument AAPL
 - jedan holding za portfolio1
 - URL: `/api/trading/portfolio-holdings/`
 2. Testovi provjeravaju:
 - status kodove za auth/no-auth
 - filtriranje rezultata po vlasništvu
-

Ispitni slučajevi

1) `test_unauthenticated_user_is_forbidden`

Funkcionalnost: anonimni korisnik ne smije dohvatiti holdings.

- **Ulazni podaci:** GET bez auth
- **Očekivani rezultat:** HTTP `401 UNAUTHORIZED`

- **Dobiveni rezultat:** prolaz
 - **Postupak:** GET → assert 401
-

2) test_list_own_holdings

Funkcionalnost: korisnik može dohvatiti vlastite holdings.

- **Ulazni podaci:**
 - auth user1
 - GET `/api/trading/portfolio-holdings/`
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 1`
 - `quantity == 10`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth → GET → provjera duljine i quantity
-

3) test_cannot_see_other_user_holdings

Funkcionalnost: korisnik ne vidi holdings drugog korisnika.

- **Ulazni podaci:**
 - auth user2
 - GET holdings
 - **Očekivani rezultat:**
 - HTTP 200
 - `len == 0`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** auth user2 → GET → assert len 0
-

TestCase: PortfolioHoldingModelTests

Funkcionalnost koja se testira

Unit testovi za properties modela `PortfolioHolding` :

- `current_value`
 - `profit_loss`
 - ponašanje kad ne postoji market data (`current_value == 0`)
-

Postupak provođenja ispitivanja

1. `setUp()` kreira:
 - korisnika i portfelj
 - instrument AAPL s `InstrumentIntervalData` gdje `close_price=160.00`
 - holding `quantity=10, average_price=150`
 2. Testovi direktno čitaju property-je modela i provjeravaju izračune.
-

Ispitni slučajevi

1) test_current_value_property

Funkcionalnost: `current_value` koristi zadnju `close_price` za izračun.

- **Ulazni podaci:** holding 10 dionica, close_price=160
 - **Očekivani rezultat:** `current_value = 1600.00`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `assert holding.current_value == 1600.00`
-

2) test_profit_loss_property

Funkcionalnost: `profit_loss` računa nerealizirani profit/loss.

- **Ulazni podaci:** (160 - 150)*10
 - **Očekivani rezultat:** `profit_loss = 100.00`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** `assert holding.profit_loss == 100.00`
-

3) test_current_value_no_data

Funkcionalnost: ako nema market data za instrument, `current_value` je 0.

- **Ulazni podaci:**
 - instrument bez `InstrumentIntervalData`
 - holding quantity=10 avg=100
 - **Očekivani rezultat:** `current_value == 0`
 - **Dobiveni rezultat:** prolaz
 - **Postupak:** create instrument bez data → create holding → assert current_value == 0
-

Creating test database for alias 'default' ('test_bullbear_local_test')...

System check identified 0 issues (0 silenced).

test_all_funds_returns_active_funds

(src.backend.api.funds.tests.test_api.FundAPITests.test_all_funds_returns_active_funds)

GET /api/funds/funds/all/ returns all active funds. ... ok

test_authenticated_user_can_list_own_funds

(src.backend.api.funds.tests.test_api.FundAPITests.test_authenticated_user_can_list_own_funds)

Authenticated user can list their own funds. ... ok

test_create_fund (src.backend.api.funds.tests.test_api.FundAPITests.test_create_fund)

Create a new fund. ... ok

test_create_fund_without_holdings

(src.backend.api.funds.tests.test_api.FundAPITests.test_create_fund_without_holdings)

Create a fund without initial holdings. ... ok

test_delete_own_fund (src.backend.api.funds.tests.test_api.FundAPITests.test_delete_own_fund)

Owner can delete their fund. ... ok

test_list_only_own_funds (src.backend.api.funds.tests.test_api.FundAPITests.test_list_only_own_funds)

User can only list their own funds. ... ok

test_performance_filters_by_days

(src.backend.api.funds.tests.test_api.FundAPITests.test_performance_filters_by_days)

Performance endpoint respects days query param. ... ok

test_performance_returns_nav_history

(src.backend.api.funds.tests.test_api.FundAPITests.test_performance_returns_nav_history)

GET /api/funds/funds/{id}/performance/ returns NAV history. ... ok

test_retrieve_any_fund_by_id (src.backend.api.funds.tests.test_api.FundAPITests.test_retrieve_any_fund_by_id)

User can retrieve any fund by ID (for viewing). ... ok

test_retrieve_nonexistent_fund_returns_404

(src.backend.api.funds.tests.test_api.FundAPITests.test_retrieve_nonexistent_fund_returns_404)

```
Retrieving a non-existent fund returns 404. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.funds.tests.test_api.FundAPITests.test_unauthenticated_user_is_forbidden)
Unauthenticated user cannot access funds. ... ok
test_update_own_fund (src.backend.api.funds.tests.test_api.FundAPITests.test_update_own_fund)
Owner can update their fund. ... ok
test_authenticated_user_can_list_subscriptions
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_authenticated_user_can_list_subscrip
tions)
Authenticated user can list their subscriptions. ... ok
test_by_fund_returns_404_if_not_subscribed
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_by_fund_returns_404_if_not_subscribe
d)
By-fund endpoint returns 404 if user is not subscribed. ... ok
test_by_fund_returns_subscription
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_by_fund_returns_subscription)
GET /api/funds/subscriptions/by-fund/{fund_id}/ returns user's subscription. ... ok
test_create_subscription
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_create_subscription)
Create a new subscription. ... ok
test_delete_subscription
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_delete_subscription)
User can delete (unsubscribe) their subscription. ... ok
test_list_only_own_subscriptions
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_list_only_own_subscriptions)
User can only list their own subscriptions. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_unauthenticated_user_is_forbidden)
Unauthenticated user cannot access subscriptions. ... ok
test_unsubscribed_excludes_own_funds
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_unsubscribed_excludes_own_funds)
Unsubscribed endpoint excludes user's own funds. ... ok
test_unsubscribed_returns_funds_not_subscribed_to
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_unsubscribed_returns_funds_not_subsc
ribed_to)
GET /api/funds/subscriptions/unsubscribed/ returns funds user is not subscribed to. ... ok
test_update_subscription
(src.backend.api.funds.tests.test_subscription.FundSubscriptionAPITests.test_update_subscription)
User can update their subscription. ... ok
test_api_invalid_time_param_defaults_to_all
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_api_invalid_time_param_defaults_to_a
ll)
Unknown time string => else fallback branch => cutoff None => behaves like all ... ok
test_api_time_param_filters_results
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_api_time_param_filters_results)
API passes time filter to get_leaderboard_queryset(). ... ok
test_authenticated_user_is_allowed
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_authenticated_user_is_allowed)
auth => 200 ... ok
test_excludes_portfolios_without_snapshots
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_excludes_portfolios_without_snapshot
s)
empty portfolios ... ok
test_executor_user_is_allowed_even_if_not_authenticated
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_executor_user_is_allowed_even_if_not
_authenticated)
executor special user => 200 ... ok
test_rank_is_continuous_across_pages
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_rank_is_continuous_across_pages)
PageNumberPagination page_size=50. ... ok
test_returns_ordered_by_latest_total_value_desc
(src.backend.api.leaderboard.tests.test_leaderboard.LeaderboardAPITests.test_returns_ordered_by_latest_total_valu
```

```
e_desc)
Ensure ordering: highest total_value first. ... ok
test_service_orders_by_latest_total_value_desc
(src.backend.api.leaderboard.tests.test_leaderboard.LeadersboardAPITests.test_service_orders_by_latest_total_value_desc)
Service returns sorted result. ... ok
test_service_time_filter_1d_excludes_older_than_1_day
(src.backend.api.leaderboard.tests.test_leaderboard.LeadersboardAPITests.test_service_time_filter_1d_excludes_older_than_1_day)
Snapshot older than 1 day should be ignored; if only old snapshots exist => portfolio excluded. ... ok
test_service_time_filter_1w_excludes_older_than_7_days
(src.backend.api.leaderboard.tests.test_leaderboard.LeadersboardAPITests.test_service_time_filter_1w_excludes_older_than_7_days) ... ok
test_service_time_filter_all_includes_old_snapshots
(src.backend.api.leaderboard.tests.test_leaderboard.LeadersboardAPITests.test_service_time_filter_all_includes_old_snapshots)
time=all should not apply cutoff. ... ok
test_service_time_filter_prefers_latest_within_cutoff
(src.backend.api.leaderboard.tests.test_leaderboard.LeadersboardAPITests.test_service_time_filter_prefers_latest_within_cutoff)
If there are snapshots both old and new, must take latest within cutoff. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.leaderboard.tests.test_leaderboard.LeadersboardAPITests.test_unauthenticated_user_is_forbidden)
No auth + not executor => 403 ... ok
test_authenticated_user_can_list_orders
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_authenticated_user_can_list_orders)
Authenticated user can list their orders. ... ok
test_cancel_order_via_action
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_cancel_order_via_action)
POST /orders/{id}/cancel/ cancels an open order. ... ok
test_cancel_order_via_delete
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_cancel_order_via_delete)
DELETE cancels an open order. ... ok
test_cannot_cancel_cancelled_order
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_cannot_cancel_cancelled_order)
Cannot cancel an already cancelled order. ... ok
test_cannot_cancel_filled_order
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_cannot_cancel_filled_order)
Cannot cancel a filled order. ... ok
test_create_limit_order_requires_limit_price
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_create_limit_order_requires_limit_price)
LIMIT order requires limit_price. ... ok
test_create_limit_order_with_limit_price
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_create_limit_order_with_limit_price)
LIMIT order with limit_price succeeds. ... ok
test_create_market_order (src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_create_market_order)
Create a new MARKET order. ... ok
test_create_stop_limit_order
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_create_stop_limit_order)
STOP_LIMIT order requires both stop_price and limit_price. ... ok
test_create_stop_order_requires_stop_price
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_create_stop_order_requires_stop_price)
STOP order requires stop_price. ... ok
test_execute_limit_order_condition_not_met
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_execute_limit_order_condition_not_met)
LIMIT order not executed if price condition not met. ... ok
test_execute_market_order_as_executor
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_execute_market_order_as_executor)
Executor can execute a MARKET order. ... ok
test_execute_order_requires_executor
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_execute_order_requires_executor)
POST /orders/{id}/execute/ requires executor service user. ... ok
```



```
test_list_only_own_orders (src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_list_only_own_orders)
User can only list their own orders. ... ok
test_open_orders_as_executor
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_open_orders_as_executor)
Executor can access open orders. ... ok
test_open_orders_requires_executor
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_open_orders_requires_executor)
GET /orders/open/ requires executor service user. ... ok
test_retrieve_own_order (src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_retrieve_own_order)
User can retrieve their own order. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.orders.tests.test_orderAPI.OrderAPITests.test_unauthenticated_user_is_forbidden)
Unauthenticated user cannot access orders. ... ok
test_list_own_order_events
(src.backend.api.orders.tests.test_orderEventAPI.OrderEventAPITests.test_list_own_order_events)
User can list events for their own orders. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.orders.tests.test_orderEventAPI.OrderEventAPITests.test_unauthenticated_user_is_forbidden)
Unauthenticated user cannot access order events. ... ok
test_list_own_order_fills
(src.backend.api.orders.tests.test_orderFillAPI.OrderFillAPITests.test_list_own_order_fills)
User can list fills for their own orders. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.orders.tests.test_orderFillAPI.OrderFillAPITests.test_unauthenticated_user_is_forbidden)
Unauthenticated user cannot access order fills. ... ok
test_apply_fill_complete (src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_apply_fill_complete)
apply_fill correctly updates complete fill. ... ok
test_apply_fill_partial (src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_apply_fill_partial)
apply_fill correctly updates partial fill. ... ok
test_apply_fill_weighted_average_price
(src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_apply_fill_weighted_average_price)
apply_fill correctly calculates weighted average price. ... ok
test_is_executable_at_price_limit_buy
(src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_is_executable_at_price_limit_buy)
LIMIT BUY executes when price <= limit_price. ... ok
test_is_executable_at_price_limit_sell
(src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_is_executable_at_price_limit_sell)
LIMIT SELL executes when price >= limit_price. ... ok
test_is_executable_at_price_market_order
(src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_is_executable_at_price_market_order)
MARKET orders are always executable. ... ok
test_is_executable_at_price_stop_buy
(src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_is_executable_at_price_stop_buy)
STOP BUY executes when price >= stop_price. ... ok
test_is_executable_at_price_stop_sell
(src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_is_executable_at_price_stop_sell)
STOP SELL executes when price <= stop_price. ... ok
test_remaining_quantity (src.backend.api.orders.tests.test_orderModel.OrderModelTests.test_remaining_quantity)
remaining_quantity property calculates correctly. ... ok
test_buy_instrument (src.backend.api.trading.tests.test_buySellLogic.BuySellAPITests.test_buy_instrument)
Buy instrument creates holding. ... ok
test_buy_instrument_with_explicit_price
(src.backend.api.trading.tests.test_buySellLogic.BuySellAPITests.test_buy_instrument_with_explicit_price)
Buy instrument with explicit price. ... ok
test_sell_instrument (src.backend.api.trading.tests.test_buySellLogic.BuySellAPITests.test_sell_instrument)
Sell instrument reduces holding. ... ok
test_sell_more_than_owned_fails
(src.backend.api.trading.tests.test_buySellLogic.BuySellAPITests.test_sell_more_than_owned_fails)
Cannot sell more than owned. ... ok
test_buy_creates_new_holding
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_buy_creates_new_holding)
buy_instrument creates a new holding. ... ok
```

```
test_buy_deducts_balance
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_buy_deducts_balance)
buy_instrument deducts from portfolio balance. ... ok
test_buy_insufficient_balance_fails
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_buy_insufficient_balance_fails)
buy_instrument fails with insufficient balance. ... ok
test_buy_updates_existing_holding
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_buy_updates_existing_holding)
buy_instrument updates existing holding with weighted average. ... ok
test_sell_adds_to_balance
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_sell_adds_to_balance)
sell_instrument adds to portfolio balance. ... ok
test_sell_insufficient_quantity_fails
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_sell_insufficient_quantity_fails)
sell_instrument fails if insufficient quantity. ... ok
test_sell_no_holding_fails
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_sell_no_holding_fails)
sell_instrument fails if no holding exists. ... ok
test_sell_reduces_holding
(src.backend.api.trading.tests.test_buySellLogic.BuySellServiceTests.test_sell_reduces_holding)
sell_instrument reduces holding quantity. ... ok
test_list_excludes_inactive_instruments
(src.backend.api.trading.tests.test_instrumentAPI.InstrumentAPITests.test_list_excludes_inactive_instruments)
Inactive instruments are excluded from list. ... ok
test_list_instruments_unauthenticated
(src.backend.api.trading.tests.test_instrumentAPI.InstrumentAPITests.test_list_instruments_unauthenticated)
Instruments can be listed without authentication. ... ok
test_retrieve_instrument_by_symbol
(src.backend.api.trading.tests.test_instrumentAPI.InstrumentAPITests.test_retrieve_instrument_by_symbol)
Retrieve instrument by symbol. ... ok
test_retrieve_nonexistent_instrument_returns_404
(src.backend.api.trading.tests.test_instrumentAPI.InstrumentAPITests.test_retrieve_nonexistent_instrument_returns_404)
Retrieve nonexistent instrument returns 404. ... ok
test_filter_by_instrument
(src.backend.api.trading.tests.test_instrumentIntervalDataAPI.InstrumentIntervalDataAPITests.test_filter_by_instrument)
Filter interval data by instrument symbol. ... ok
test_list_all_interval_data
(src.backend.api.trading.tests.test_instrumentIntervalDataAPI.InstrumentIntervalDataAPITests.test_list_all_interval_data)
List all interval data without filter. ... ok
test_get_quote_by_symbol
(src.backend.api.trading.tests.test_instrumentQuoteAPI.InstrumentQuoteAPITests.test_get_quote_by_symbol)
Get quote for instrument by symbol. ... ok
test_get_quote_nonexistent_returns_404
(src.backend.api.trading.tests.test_instrumentQuoteAPI.InstrumentQuoteAPITests.test_get_quote_nonexistent_returns_404)
Get quote for nonexistent symbol returns 404. ... ok
test_filter_latest_by_instrument
(src.backend.api.trading.tests.test_latestInstrumentDataAPI.LatestInstrumentDataAPITests.test_filter_latest_by_instrument)
Filter latest data by instrument. ... ok
test_list_latest_data_for_all_instruments
(src.backend.api.trading.tests.test_latestInstrumentDataAPI.LatestInstrumentDataAPITests.test_list_latest_data_for_all_instruments)
List latest data for all instruments. ... ok
test_cannot_see_other_user_holdings
(src.backend.api.trading.tests.test_portfolioHoldingAPI.PortfolioHoldingAPITests.test_cannot_see_other_user_holdings)
User cannot see other user's holdings. ... ok
test_list_own_holdings
```

```

(src.backend.api.trading.tests.test_portfolioHoldingAPI.PortfolioHoldingAPITests.test_list_own_holdings)
User can list their own holdings. ... ok
test_unauthenticated_user_is_forbidden
(src.backend.api.trading.tests.test_portfolioHoldingAPI.PortfolioHoldingAPITests.test_unauthenticated_user_is_forbidden)
Unauthenticated user cannot access holdings. ... ok
test_endpoint_is_deterministic
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_endpoint_is_deterministic)
Multiple calls produce same result. ... ok
test_invalid_interval_returns_400
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_invalid_interval_returns_400) ... ok
test_invalid_portfolio_returns_400
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_invalid_portfolio_returns_400) ... ok
test_invalid_range_returns_400
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_invalid_range_returns_400) ... ok
test_max_drawdown_correct
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_max_drawdown_correct)
values: 100 -> 120 -> 90 -> 130 ... ok
test_missing_portfolio_returns_400
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_missing_portfolio_returns_400) ... ok
test_no_snapshots_returns_zero_metrics
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_no_snapshots_returns_zero_metrics)
No snapshots -> all metrics zero ... ok
test_non_owner_forbidden_or_empty
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_non_owner_forbidden_or_empty)
- either 403 ... ok
test_return_pct_correct_simple
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_return_pct_correct_simple)
values: 100 -> 110 => return_pct = 10% ... ok
test_var_cvar_consistency_monotonic_up
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_var_cvar_consistency_monotonic_up)
If total_value increases monotonically, returns are >= 0, ... ok
test_volatility_non_negative
(src.backend.api.users.tests.test_portfolio_snapshoting_risk.PortfolioRiskAPITests.test_volatility_non_negative)
Volatility should never be negative. ... ok
test_executor_can_create_snapshot_any_portfolio
(src.backend.api.users.tests.test_portfolio_snapshoting.PortfolioSnapshotAPITests.test_executor_can_create_snapshot_any_portfolio)
Executor service can create snapshot for any portfolio. ... ok
test_filter_from_to
(src.backend.api.users.tests.test_portfolio_snapshoting.PortfolioSnapshotAPITests.test_filter_from_to)
from excludes earlier snapshots, to excludes later snapshots. ... ok
test_invalid_datetime_does_not_crash
(src.backend.api.users.tests.test_portfolio_snapshoting.PortfolioSnapshotAPITests.test_invalid_datetime_does_not_crash)
Bad from/to should not crash or 500. ... ok
test_latest_returns_404_if_none
(src.backend.api.users.tests.test_portfolio_snapshoting.PortfolioSnapshotAPITests.test_latest_returns_404_if_none)
GET latest snapshot for a portfolio with no snapshots -> 404 ... ok
test_latest_snapshot
(src.backend.api.users.tests.test_portfolio_snapshoting.PortfolioSnapshotAPITests.test_latest_snapshot)
GET latest snapshot for portfolio1 ... ok

```

```
test_limit (src.backend.api.users.tests.tests_portfolio_snapshoting.PortfolioSnapshotAPITests.test_limit)
limit=N returns at most N snapshots. ... ok
test_non_owner_sees_none
(src.backend.api.users.tests.tests_portfolio_snapshoting.PortfolioSnapshotAPITests.test_non_owner_sees_none)
User2 tries to list snapshots of portfolio1 -> should return empty. ... ok
test_order_asc_desc
(src.backend.api.users.tests.tests_portfolio_snapshoting.PortfolioSnapshotAPITests.test_order_asc_desc)
order=asc returns earliest first, order=desc returns latest first. ... ok
test_owner_can_list_snapshots
(src.backend.api.users.tests.tests_portfolio_snapshoting.PortfolioSnapshotAPITests.test_owner_can_list_snapshots)
... ok
test_user_cannot_create_snapshot_for_other_portfolio
(src.backend.api.users.tests.tests_portfolio_snapshoting.PortfolioSnapshotAPITests.test_user_cannot_create_snapsh
ot_for_other_portfolio)
User1 tries to create snapshot for portfolio2 (owned by user2) -> 403 Forbidden ... ok

-----
Ran 112 tests in 76.166s

OK
Destroying test database for alias 'default' ('test_bullbear_local_test')...
```

Ispitni slučajevi (Selenium WebDriver, PyTest)

Opće informacije

- **Alat:** Selenium WebDriver (Python)+ PyTest
- **Aplikacija:** BullBear (frontend)
- **Osnovni URL:** <http://localhost:5173/>
- **Timeout:** 60s

TEST CASE 1 – Redovni slučaj: Navigacija i učitavanje početne stranice

Ulazi

- URL: <http://localhost:5173/>
- Korisničke akcije:
 - Otvaranje aplikacije u pregledniku
 - Automatsko učitavanje stranice
 - Provjera elemenata (logo, navigacija, hero tekst)
 - Navigacija preko više stranica

Koraci ispitivanja (Selenium)

1. Otvoriti aplikaciju u pregledniku:

- `driver.get(BASE_URL)`

2. Provjeriti naslov stranice:

- `assert "BullBear" in driver.title or driver.title != ""`

3. Provjeriti da postoji logo:

- `EC.presence_of_element_located((By.CLASS_NAME, "logo-text"))`

4. Provjeriti da postoje navigacijski linkovi:

- `.nav-links a`

5. Provjeriti "hero" sekciju:

- `EC.presence_of_element_located((By.CLASS_NAME, "hero-title"))`

6. Iterativno testirati navigaciju na podstranice:

- `/features`, `/pricing`, `/faq`, `/contact`, `/about`, `/login`

Očekivani izlaz

- Početna stranica se ispravno učitava.
- Naslov stranice nije prazan i sadrži "BullBear" (ili barem postoji title).
- Logo s tekstom "BullBear" je vidljiv.
- Navigacija sadrži minimalno 2 linka.
- "Hero" sekcija nije prazna i sadrži očekivani sadržaj.
- Navigacija na podstranice radi bez greške.

TEST CASE 2 – Redovni slučaj: Navigacija na Login stranicu

Ulazi

- Početna stranica: `http://localhost:5173/`
- Korisničke akcije:
 - Klik na "Login" gumb/link u navigaciji

Koraci ispitivanja (Selenium)

1. Otvoriti homepage:

- `driver.get(BASE_URL)`

2. Pronaći login element:

- `EC.element_to_be_clickable((By.CSS_SELECTOR, "a[href='/login']", ".nav-btn"))`

3. Kliknuti login gumb:

- `login_btn.click()`

4. Provjeriti URL:

- `assert "/login" in driver.current_url`

5. Provjeriti OAuth gumbe:

- `.oauth-buttons button, .oauth-btn`

Očekivani izlaz

- Korisnik se preusmjerava na `/login`
- Login stranica se učitava bez greške
- OAuth gumbi (Google/Microsoft) su vidljivi

TEST CASE 3 – Rubni uvjet: Pristup zaštićenoj ruti bez autentifikacije

Ulazi

- URL: `http://localhost:5173/positions`
- Stanje sustava: korisnik **nije prijavljen**

- Korisničke akcije:
 - Direktno učitavanje zaštićene rute (bez login-a)

Koraci ispitivanja (Selenium)

1. Direktno otvoriti:

- `driver.get(f"{BASE_URL}/positions")`

2. Očitati trenutni URL:

- `current_url = driver.current_url`

3. Provjeriti reakciju:

- Je li došlo do redirecta na `/login` ili `/`
- Alternativno, provjera poruka u page source:
 - `"authentication", "unauthorized", "access denied", "login"`

4. Provjeriti da osjetljiv sadržaj nije vidljiv:

- traži se `.portfolio-table`, `.positions-list`, `.stock-holdings`
- očekuje se da je rezultat prazan

Očekivani izlaz

- Sustav blokira prikaz sadržaja "positions".
- Korisnik je:
 - preusmjeren na `/login` ili početnu stranicu, ili
 - dobiva poruku "authentication required".
- Zaštićeni elementi se **ne prikazuju**.

TEST CASE 4 – Poziv nepostojeće funkcionalnosti: Pristup nepostojećoj stranici (404)

Ulazi

- URL: `http://localhost:5173/nonexistent-page-xyz-123`
- Korisničke akcije:
 - Ručni unos URL-a za stranicu koja nije implementirana

Koraci ispitivanja (Selenium)

1. Otvoriti nepostojeći URL:

- `driver.get(nonexistent_url)`

2. Provjeriti sadržaj stranice:

- tražiti indikatore 404:
 - `"404", "not found", "invalid url", "page does not exist"`

3. Provjeriti redirect:

- je li korisnik vraćen na homepage

4. Provjeriti stabilnost aplikacije:

- `body = driver.find_element(By.TAG_NAME, "body")`
- tijelo nije prazno i DOM postoji

Očekivani izlaz

- Sustav prikazuje 404 ili informativnu poruku “Not found”, ili
- Sustav radi redirect na početnu stranicu.
- Aplikacija se **ne ruši** (nema prazne stranice / white screen).

```
=====
===== test session starts
=====
=====
platform linux -- Python 3.13.7, pytest-7.4.3, pluggy-1.6.0 --
/home/vhrabar/Programing/FER/SEM5/PROINZ/.venv/bin/python
cachedir: .pytest_cache
metadata: {'Python': '3.13.7', 'Platform': 'Linux-6.17.0-8-generic-x86_64-with-glibc2.42', 'Packages': {'pytest': '7.4.3', 'pluggy': '1.6.0'}, 'Plugins': {'metadata': '3.1.1', 'html': '4.1.1'}}
rootdir: /home/vhrabar/Programing/FER/SEM5/PROINZ/src/selenium
plugins: metadata-3.1.1, html-4.1.1
collected 8 items

test_bullbear.py::TestBullBearApplication::test_01_homepage_navigation PASSED
[ 12%]
test_bullbear.py::TestBullBearApplication::test_02_login_page_navigation PASSED
[ 25%]
test_bullbear.py::TestBullBearApplication::test_03_protected_route_without_auth PASSED
[ 37%]
test_bullbear.py::TestBullBearApplication::test_04_nonexistent_page PASSED
[ 50%]
test_bullbear.py::TestBullBearApplication::test_05_browser_navigation PASSED
[ 62%]
test_bullbear.py::TestBullBearApplication::test_06_responsive_elements PASSED
[ 75%]
test_bullbear.py::TestEdgeCases::test_07_rapid_clicking PASSED
[ 87%]
test_bullbear.py::TestEdgeCases::test_08_nonexistent_api_route PASSED
[100%]

-----
----- Generated html report: file:///home/vhrabar/Programing/FER/SEM5/PROINZ/src/selenium/report.html -----
-----
=====
===== 8 passed in 72.34s (0:01:12)
=====
=====
```

Korištene tehnologije i alati

Cilj: Jasno i precizno opisati tehnologije korištene u projektu kako bi se olakšalo održavanje, proširenje i suradnja u timu. Uključite informacije:

1. **Programski jezici:** Navesti korištene jezike i njihove verzije (npr. JavaScript 16.13).
2. **Radni okviri i biblioteke:** Detaljno opisati alate za frontend i backend (npr. React 18, Node.js 16).

3. **Baza podataka:** Navesti vrstu baze (npr. PostgreSQL 13).
4. **Razvojni alati:** Popis korištenih IDE-ova, alata za verzioniranje (npr. VS Code, Git 2.34).
5. **Alati za ispitivanje:** Jedinični, integracijski ili UI ispitni sljučajevi (npr. Jest 27, Selenium 4.0).
6. **Alati za razmještaj:** Korišteni alati za implementaciju (npr. Docker 20.10).
7. **Cloud platforma:** Ako je aplikacija hostana, navesti platformu (npr. AWS).

Preporuke za opis:

- **Jasno i precizno:** Izbjegavati tehnički žargon i navesti točne verzije.
- **Obrazloženje izbora:** Objasniti zašto su odabrane određene tehnologije.
- **Opis konfiguracije:** Istaknuti specifične postavke alata i baza.

Primjer:

Za razvoj klijentskog dijela aplikacije korišten je **React** (verzija 18)[ref.], popularna JavaScript biblioteka za izgradnju interaktivnih korisničkih sučelja. React omogućuje stvaranje samostalnih komponenti koje se mogu ponovno koristiti i lako ažurirati, čime se poboljšava učinkovitost razvoja. Za stiliziranje su upotrijebljene **styled-components** (verzija 5.3), što omogućava integraciju stilova izravno unutar Ručat komponenti koristeći **CSS-in-JS** pristup.

1. Instalacija

- Preduvjeti: Popis potrebnog softvera i njihovih verzija

- Python 3.12 +
- PostgreSQL 18+
- Node.js 20 +

- Preuzimanje:

```
git clone git@github.com:vhrabar/BullBear.git #ssh
git clone https://github.com/vhrabar/BullBear.git #https
```

```
cd BullBear
```

Instalacija ovisnosti: Upute za instaliranje ovisnosti.

Backend

```
cd src/backend
```

Linux / Mac

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows (PS)

```
py -m venv .venv
..venv\Scripts\Activate.ps1
```


Windows (CMD)

```
py -m venv .venv
..venv\Scripts\activate.bat
```

Instalacija komponenti

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Scheduler

```
cd src/scheduler
```

Linux / Mac

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows (PS)

```
py -m venv .venv
..venv\Scripts\Activate.ps1
```

Windows (CMD)

```
py -m venv .venv
..venv\Scripts\activate.bat
```

Instalacija komponenti

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Frontend

```
cd src/frontend
npm install
```

2. Postavke

Backend (src/backend/.env.example)

Potrebno je ispuniti sva polja s ogovarajućim podacima

```
DB_NAME=          # naziv baze
DB_USER=          # korisnik s ovlaštenjima na bazi
DB_PASSWORD=      # lozinka korisnika s ovlaštenjima na bazi
DB_HOST=          # host na kojemu se baza nalazi
DB_PORT=          # port iza kojega se baza nalazi
DB_SSLMODE=       # required / no
```

```
PRODUCTION=      # true/false
```

```
STRIPE_SECRET_KEY=      # Stripe key
```

```
PAYPAL_CLIENT_ID=      # Paypal key
PAYPAL_CLIENT_SECRET=  # Paypal secret
PAYPAL_MODE=           # sandbox / live
```

- migracija

```
mv .env.example .env
```

Frontend (`src/frontend/.env.example`)

Potrebno je ispuniti sva polja s ogovarajućim podacima

```
REACT_APP_ENV=      # development / production
VITE_ENV=            # development / production
```

- migracija

```
mv .env.example .env
```

```
MASSIVE_API_KEY=     # Massive API key
```

```
DB_NAME=             # naziv baze
DB_USER=              # korisnik s ovlaštenjima na bazi
DB_PASSWORD=         # lozinka korisnika s ovlaštenjima na bazi
DB_HOST=             # host na kojemu se baza nalazi
DB_PORT=             # port iza kojega se baza nalazi
DB_SSLMODE=          # required / no
```

```
TESTING=             # true / false
```

```
DJANGO_SERVICE_TOKEN= # DJ token
DJANGO_API_BASE_URL=  # URL na kojemu počiva backend
```

Postavke baze podataka: Upute za inicijalizaciju baze podataka,

uključujući migracije i postavljanje inicijalnih podataka.

- login u postgres servis

```
sudo -u postgres psql
```

- kreiranje korisnika i baze podataka

```
CREATE USER django_user WITH PASSWORD 'STRONG_PASSWORD_HERE';
```

```
CREATE DATABASE django_db OWNER django_user;
```

```
ALTER ROLE django_user SET client_encoding TO 'utf8';
ALTER ROLE django_user SET default_transaction_isolation TO 'read committed';
ALTER ROLE django_user SET timezone TO 'UTC';
```

```
GRANT ALL PRIVILEGES ON DATABASE django_db TO django_user;
```

Migracija baze

```
cd src/backend
python manage.py makemigrations
python manage.py migrate
python manage.py populate_db
```

3. Pokretanje aplikacije

Dev

```
python src/backend/manage.py runserver
python src/scheduler/ingestion_service.py
```

```
cd src/frontend && npm run dev
```

Prod

```
python src/backend/manage.py runserver
python src/scheduler/ingestion_service.py
cd src/frontend && npm build && npm run
```

Alternativno (Docker)

```
docker compose build --no-cache
docker compose up -d
```

5. Upute za administratore

Smjernice za administratore aplikacije nakon puštanja u pogon:

- **Pristup administratorskom sučelju:**
 - URL za admin panel: `localhost:8000/admin`.
- **Redovito održavanje:**
 - Arhiviranje baze podataka.
 - Pregled logova.
 - Ažuriranje aplikacije (primjer: povlačenje novih verzija iz Git repozitorija i ponovno pokretanje aplikacije).

****5. Primjer za D0 platformu ****

Digital Ocean je popularna cloud platforma za jednostavno smještanje aplikacija. Deployment je posve automatiziran putem Github Actions-a, no moguće ga je izvršiti i ručno.

Inicijalizacija (vidi sekciju I)

Postavke (vidi sekciju II)

Generiranje SSH ključa

- spremanje ključa

```
mkdir -p ~/.ssh
nano ~/.ssh/do_bullbear
chmod 600 ~/.ssh/do_bullbear
ssh-keyscan -H >> ~/.ssh/known_hosts
```

Deployment

```
rsync -avz --delete
-e "ssh -i ~/.ssh/do_bullbear"
./
root@:/root/bullbear/
```

Pokretanje / Restart /

```
ssh -i ~/.ssh/do_bullbear root@<DO_DROPLET_IP> << 'EOF'
cd /root/bullbear
docker compose down
docker compose build --no-cache
docker compose up -d
EOF
```

Pristup aplikaciji

- Pristup je moguć preko: bull-bear.app
- admin sučelje na admin.bull-bear.app/admin

1. Programsko inženjerstvo, FER ZEMRIS, <http://www.fer.hr/predmet/proinz>
2. I. Sommerville, "Software engineering", 8th ed, Addison Wesley, 2007.
3. T.C.Lethbridge, R.Langaniere, "Object-Oriented Software Engineering", 2nd ed. McGraw-Hill, 2005.
4. I. Marsic, Software engineering book", Department of Electrical and Computer Engineering, Rutgers University, <http://www.ece.rutgers.edu/~marsic/books/SE>
5. The Unified Modeling Language, <https://www.uml-diagrams.org/>
6. Astah Community, <http://astah.net/editions/uml-new> Rev. | Opis promjene/dodatka | Autori | Datum |
| ——— |:————— |:——— |:——— |
| 0.0.1 | Ažuriranje predloška Home stranice za wiki. ([1983135](#)) | Antun Silov | 16.10.2025. |
| 0.0.2 | Ubacivanje profesorovog predloška u wiki projekta, uskoro ažuriranje s projektnim informacijama ([d192e9e](#)) | Antun Silov | 16.10.2025. |
| 0.0.3 | Stvaranje Home stranice za wiki. ([fdbaf70](#)) | Antun Silov | 16.10.2025. |
| 0.0.4 | Ažuriranje predloška podnožja za wiki. ([d4a1e8a](#)) | Antun Silov | 16.10.2025. |
| 0.0.5 | Inicijalni FZ ([0b14a9f](#)) | Vedran Hrabar | 17.10.2025. |
| 0.0.6 | Izmjena FZ ([596e562](#)) | Vedran Hrabar | 17.10.2025. |
| 0.0.7 | Izmjena FZ ([1e83c37](#)) | Vedran Hrabar | 17.10.2025. |
| 0.0.8 | Izmjena FZ ([54a296f](#)) | Viktor Lazić | 17.10.2025. |
| 0.0.9 | Izmjena FZ ([54a296f](#)) | Vedran Hrabar | 28.10.2025. |
| 0.0.10 | Izmjena FZ ([d4e59c2](#)) | Vedran Hrabar | 28.10.2025. |
| 0.0.11 | Izmjena FZ ([50ed086](#)) | Vedran Hrabar | 29.10.2025. |
| 0.0.12 | Dodani Highlevel i VrsteKorisnika UC dijagram ([771325f](#)) | Vedran Hrabar | 30.10.2025. |
| 0.0.13. | Dodani UC dijagrami za ključne značajke i vanjske integracije ([019529b](#)) | Vedran Hrabar | 30.10.2025. |
| 0.0.14 | Dodan prikaz arhitekture ([9bd1c58](#)) | Vedran Hrabar | 30.10.2025. |
| 0.0.15 | Revizija UC dijagrama ([8d18cd3](#)) | Vedran Hrabar | 5.11.2025. |
| 0.0.16 | Revizija UC dijagrama ([9cfb829](#)) | Vedran Hrabar | 5.11.2025. |
| 0.0.17 | Revizija UC dijagrama ([5b1adab](#)) | Vedran Hrabar | 5.11.2025. |
| 0.0.18 | Promjena formatiranja tablice ([746e2be](#)) | Vedran Hrabar | 6.11.2025. |
| 0.0.19 | Dodani akteri i dionici ([c1f4414](#)) | Vedran Hrabar | 6.11.2025. |
| 0.0.20 | Opis projektnoga zadatka ([70746aa](#)) | Vedran Hrabar | 6.11.2025. |
| 0.0.21 | Dodan opis arhitekture i podaci o deploymentu ([ba5b86c](#)) | Vedran Hrabar | 6.11.2025. |
| 0.0.22 | Dodani opisi upotrijebljenih protokola ([6d2f730](#)) | Vedran Hrabar | 6.11.2025. |
| 0.0.23 | Dodan opis UC obrazaca ([93f7eaa](#)) | Vedran Hrabar | 7.11.2025. |
| 0.0.24 | inicijalna verzija AI sekcije ([d0146b6](#)) | Vedran Hrabar | 12.11.2025. |

Dnevnik sastajanja

([Nulti sastanak](#))

Prisustvovali:

- Antun Silov (@AntunSilov)
- Leon Zorko (@LeonZorko)
- Viktor Lazić (@ViktorLazic3)
- Luka Varga (@MegaNoris)
- Vedran Radojčić (@VedranRadojicic) [remote]
- Vedran Hrabar (@vhrabar)

Odluke:

- usuglašena tema
- odabran voditelja tima (@vhrabar)

- Antun Silov (@AntunSilov) zadužen za kreaciju Discord servera

(Prvi sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Viktor Lazić (@ViktorLazic3) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Antun Silov (@AntunSilov) zadužen za izradu incijalne prezentacije na temelju FZ/NFZ i trenutalnoga stanja projekta
- konačni draft prezentacije odobren
- podijeljene uloge
- Viktor Lazić (@ViktorLazic3) zadužen za izradu početne stranice

(Drugi sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Viktor Lazić (@ViktorLazic3) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Vedran Hrabar (@vhrabar) zadužen za OAuth integraciju
- bull-bear.app izabrana kao domena aplikacije
- kreirani UC dijagrami

(Treći sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Viktor Lazić (@ViktorLazic3) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Antun Silov (@AntunSilov) i Vedran Hrabar (@vhrabar) zaduženi za Djnago-REST API obrasca za simuliranu trgovinu instrumentima
- Leon Zorko (@LeonZorko), Viktor Lazić (@ViktorLazic3) i Luka Varga (@MegaNoris) zaduženi za REACT view portofolia
- Leon Zorko (@LeonZorko), Viktor Lazić (@ViktorLazic3) i Vedran Radojčić (@VedranRadojcic) zaduženi za REACT view obrasca za trgovinu

(Četvri sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Viktor Lazić (@ViktorLazic3) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]

- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Testiranje Google/MS logina, te osnovne funkcionalnosti
- Usuglašavanje s dokumentacijom

(Peti sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov)
- Leon Zorko (@LeonZorko) [remote]
- Viktor Lazić (@ViktorLazic3)
- Luka Varga (@MegaNoris)
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar)

Odluke:

- prezentacija inicijalne funkcionalnosti
- Antun Silov (@AntunSilov) zadužen za opis UC dijagrama
- Luka Varga (@MegaNoris) zadužen za prijepis dijagrama
- Vedran Hrabar (@vhrabar) zadužen za otklanjanje problema s deplymentom
- Vedran Radojčić (@VedranRadojcic) zadzžen za organizaciju sustava

(Šesti sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Viktor Lazić (@ViktorLazic3) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Introspekcija rada u prvome ciklusu
- sastavljanje prijedloga žalbe
- usvajanje distribucije bodova na prijedlog Vedrana Hrabara (@vhrabar):

Ime i prezime	base	korekcija	konačni bodovi
Antun Silov	8,5	1,5	10
Leon Zorko	8,5	1,5	10
Luka Varga	8,5	1,5	10
Vedran Hrabar	8,5	-3	5,5
Vedran Radojčić	8,5	1,5	10
Viktor Lazić	8,5	-3	5,5

([Sedmi sastanak]

(<https://github.com/vhrabar/BullBear/issues/53>)) Prisustvovali: - Antun Silov (@AntunSilov) [remote] - Leon Zorko (@LeonZorko) [remote] - Viktor Lazić (@ViktorLazic3) [remote] - Luka Varga (@MegaNoris) [remote] - Vedran Radojčić (@VedranRadojcic) [remote] - Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Antun Silov (@AntunSilov) postaje vršiteljem dužnosti voditelja tima do daljnjega
- Usuglašenoga je da će razvoj funkcionalnosti korisničkih profila biti slijedećom fazom rada
- Antun Silov (@AntunSilov) i Vedran Hrabar (@vhrabar) preuzimaju REST API te funkcionalnosti
- Viktor Lazić (@ViktorLazic3) i Leon Zorko (@MegaNoris) preuzimaju korisničko sučelje
- Vedran Hrabar (@vhrabar) zadužen je za auto-updater (scheduler) vrijednosti u bazi

(Osmi sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Vedran Hrabar (@vhrabar) nastavlja s družnostima vodtelja grupe
- Antun Silov (@AntunSilov) zadužen za dokumentiranej dinamičkoga ponašanja aplikacije
- Luka Varga (@MegaNoris) zadužen za dokumentiranje dijagrama komponenata
- Leon Zorko (@LeonZorko) zadužen za dokumentiranje dijagrama razmještaja
- Vedran Radojčić (@VedranRadojcic) zadužen za dokumentiranje upotrijebljenih alata i tehnologija
- vedran Hrabar (@vhrabar) zaduđen za dokumentiranje testova, te upite za puštanje u pogon i korištenje

(Deveti sastanak)

Prisustvovali:

- Antun Silov (@AntunSilov) [remote]
- Leon Zorko (@LeonZorko) [remote]
- Luka Varga (@MegaNoris) [remote]
- Vedran Radojčić (@VedranRadojcic) [remote]
- Vedran Hrabar (@vhrabar) [remote]

Odluke:

- Antun Silov (@AntunSilov), Leon Zorko (@LeonZorko) i Vedran Radojčić (@VedranRadojcic) zaduženi za leaderboard stranicu (backend + frontend) po prinosu
- Leon Zorko (@LeonZorko) zaduđen za profil stranica i ažuriranje profila
- Vedran radojčić (@VedranRadojcic) zadužen za stranicu za uvoz i izvoz CSV datoteka.
- Vedran Hrabar (@vhrabar) i Luka Varga (@MegaNoris) zaduženi za pregled, kreiranje i upravljanje mini-fondovima.

Tablica aktivnosti

Stavka	Vedran Hrabar	Luka Varga	Leon Zorko
Upravljanje projektom	10h	0h	0h
Opis projektnog zadatka	2h	0h	0h
Funkcionalni zahtjevi	2h	0h	0h
Opis pojedinih obrazaca	0h	0h	0h
Dijagram obrazaca	0h	1.5h	0h
Sekvencijski dijagrami	1h	0h	0h
Opis ostalih zahtjeva	0h	0h	2h
Arhitektura i dizajn sustava	1h	0h	0h
Baza podataka	2h	0h	0h
Dijagram razreda	3h	0h	0h
Dijagram stanja	0h	0h	0h
Dijagram aktivnosti	0h	0h	0h
Dijagram komponenti	0h	0h	0h
Korištene tehnologije i alati	0h	1.5h	0h
Ispitivanje programskog rješenja	10h	3h	0h

Stavka	Vedran Hrabar	Luka Varga	Leon Zorko
Dijagram razmještaja	0h	0h	4h
Upute za puštanje u pogon	1h	0h	0h
Dnevnik sastajanja	1.5h	0h	0h
Zaključak i budući rad	0.5h	0h	0h
Popis literature	0.5h	0h	0h
izrada baze podataka	10 min	0h	0h
spajanje s bazom podataka	10 min	0h	0h
izrada prezentacije	0h	0h	0h
frontend	15h	35h	50h
backend	100 h	0h	0h
scheduler	10 h	0h	0h

Dijagram pregleda promjena

Prenijeti dijagram pregleda promjena nad datotekama projekta. Potrebno je na kraju projekta generirane grafove s githuba prenijeti u ovo poglavlje dokumentacije. Dijagrami za vlastiti projekt se mogu preuzeti s github.com stranice, u izborniku Repository, pritiskom na stavku Contributors.

Ključni izazovi i rješenja

- Zaključno
- Opis izazova: Glavni izazovi tijekom projekta (npr. kašnjenje u razvoju, tehnički problemi).
- Rješenja: Način na koji su izazovi riješeni, kao i naučene lekcije koje su doprinijele napretku tima.

Uporaba LLM alata

Ova sekcija navodi sve slučajeve u kojima su upotrijebljeni LLM alati, njihove prednosti, te pojedinosti o načinu korištenja.

Ispravak pravopisnih i jezičnih pogrešaka u Wiki sekciji

Google Gemini 2.5 Flash upotrijebljen je za ispravak jezičnih i pravopisnih pogrešaka u Wikiju. Alat je omogućio automatsko prepoznavanje gramatičkih nepravilnosti, te neusklađenosti u stilu pisanja. Korištenjem ovakvog sustava znatno je smanjeno vrijeme potrebno za lekturu, a osigurana je i viša razina jezične preciznosti. Uporaba alata nipošto ne utječe na razvijanje sposobnosti pisanje dokumentacije, već ubrzava postupak lektoriranja kakav pružaju servisi poput ispravi.me

Automatski PR Code Review

GitHub Copilot konfiguriran je da pruža automatski Code Review unutar svakoga Pull Requesta. Sustav analizira prijavljene promjene u kodu, predlaže optimizacije, upozorava na potencijalne logičke ili sigurnosne nedostatke, te osigurava usklađenost s postojećim standardima. Ovakav je pristup u zadnje vrijeme postao djelom standardne prakse, te nipošto ne utječe negativno na osnovne ciljeva projekta.

Ispravak konflikata tijekom merganja PR-ova

GitHub Copilot (Claude Opus 4.5 agent) korišten je za automatizirani ispravak konflikata prilikom merganja PR-ova. Ta se primjena nije pokazala osobito uspješnom te je izazvala više problema nego koristi, izvođenjem rebasea koji je uništio velik dio koda. Većina je štete oporavljena pomoću alata `git reflog` i `git fsck`, ali je svejedno izvučena pouka o korištenju agentskih LLM alata. Posljedično, svi kasniji PR-ovi obrađeni su ručno.

Pisanje komentara

Dio komentara napisan je pomoću lokalnog JetBrains single-line autocomplete LLM-a koji je omogućio brzo umetanje korisnih simbola (poput strelica, headera i sličnog) u svrhu boljeg stiliziranja komentara te veće preglednosti.

Ispravak bugova

Za pronalaženje bugova (prvenstveno), te njihov ispravak (u manjoj mjeri), korišten je GitHub Copilot (Claude Opus 4.5 i GPT Codex) u agentskom načinu rada. Ti su alati zgodni za brzo pronalaženje nesukladnosti između različitih komponenti koje su kreirali različiti članovi tima. Manje su bili korisni za ispravke te su često unosili dodatne probleme, pa je njihova uporaba svedena prvenstveno na detekciju nesukladnosti (najčešće u REST–React prikazu), a ne na njihov ispravak.

Test dokumentacija

Većina opisa testnih slučajeva napisana je ručno, no u nedostatku vremena manji je dio generiran pomoću modela GPT-5.2 na temelju:

- prošlih opisa
- koda testnih slučajeva
- izlaza Django testnog alata

Nažalost, zbog nedostatka vremena odlučili smo se za ovaj pristup kako bismo smanjili mogućnost pogrešaka (nakon problema s prvom predajom). Napisali smo veći broj testova koji su bili posebno korisni za otkrivanje grešaka i nesukladnosti koje bi se teško detektirale ručnim ispitivanjem u web pregledniku.

Alternativa je bila opisati samo dio testova, no s obzirom na njihovu repetitivnu narav, za manji dio testova upotrijebljen je LLM alat, koji je prikladan za takve zadatke. Budući da je većina testova ipak opisana ručno, nije se smatralo da bi se time ugrozila neka komponenta.